



Universidad Loyola

Grado en Ingeniería Informática y Tecnologías Virtuales

TRABAJO FIN DE GRADO

Aprendizaje por refuerzo para la ciberseguridad: un defensor basado en flujos para decisiones binarias de permitir/bloquear

Autor: Javier Rivero Iglesias

Tutor: Alfonso Carlos Martínez Estudillo

Julio de 2026

Agradecimientos

Estas líneas cierran una etapa que no habría sido posible recorrer solo, y es de justicia dedicárselas a quienes la han hecho más llevadera.

A mis padres, Juan y Esmeralda, por su apoyo incondicional, por sostenerme en los momentos de duda y por aguantarme (que no es poco) durante todos estos años de estudio. Todo lo bueno que haya en este trabajo empezó, sin duda, en casa.

A mi hermano, Carlos, por soportarme en el día a día, por las conversaciones a deshoras y por recordarme, cuando más falta hacía, que siempre hay vida más allá de una pantalla.

A mi tutor, Alfonso Carlos, por guiarme durante todo este tiempo con paciencia y criterio, por sus consejos en cada reunión y por la confianza depositada en este proyecto desde el primer día.

A todos ellos, gracias de corazón: este trabajo también es vuestro.

Resumen

Este Trabajo Fin de Grado estudia la detección de intrusiones en red como una decisión binaria, sensible al coste y realizada flujo a flujo: PERMIT o BLOCK. Se ha construido un pipeline reproducible que transforma CICIDS2017 en una observación canónica de 152 dimensiones —76 características de flujo y 76 indicadores de presencia—, excluye campos susceptibles de fuga, ajusta el preprocesamiento únicamente con la partición de entrenamiento y entrena un agente *Quantile Regression Deep Q-Network* (QRDQN). La recompensa penaliza con mayor intensidad permitir un ataque que bloquear tráfico benigno. Como el factor de descuento se fija en $\gamma = 0$, el problema se interpreta expresamente como un bandido contextual y no como una defensa secuencial que modifica una red real.

La contribución metodológica incluye un perfil experimental congelado, semillas separadas para partición y modelo, una caché canónica sin escalar, contratos completos de artefactos y una campaña final diseñada para distinguir aprendizaje intradistribución de generalización. Esta campaña comprende una ejecución MAIN nueva, validación directa, intervalos *bootstrap*, análisis de duplicados, control con etiquetas barajadas, partición completa por días, escalera de tamaños, sensibilidad a la semilla del modelo, cuatro escenarios retenidos dirigidos, comparaciones equivalentes con *Random Forest* e inferencia offline sobre tráfico de laboratorio.

En el momento de cerrar este borrador de supervisión, la campaña final todavía no ha producido resultados científicos. Los artefactos históricos muestran que el pipeline aprende con una partición aleatoria y, a la vez, que los duplicados y el cambio de día obligan a limitar cualquier afirmación de generalización; se presentan únicamente como evidencia previa a la campaña. Por ello no se anticipa una conclusión numérica final. El resultado ya defendible es un método auditable para determinar, cuando existan los artefactos finales, qué comportamiento persiste al variar distribución, volumen de entrenamiento, semilla, escenario y familia de modelo. El sistema permanece restringido a experimentación e inferencia offline y no se considera preparado para despliegue operativo.

Palabras clave: aprendizaje por refuerzo; detección de intrusiones en red; CICIDS2017; generalización; QRDQN; reproducibilidad.

Índice general

Agradecimientos	i
Resumen	ii
Índice de figuras	xi
Índice de tablas	xii
Índice de algoritmos	xiv
Glosario de acrónimos y términos	xv
1. Introducción	1
1.1. Motivación	1
1.2. Definición del problema	3
1.2.1. Desafíos metodológicos	3
1.2.2. Especificación verificable del proyecto	4
1.3. Enfoque propuesto	5
1.4. Contribución de la tesis	7
1.5. Alcance y limitaciones	8
1.6. Estructura del documento	9

2. Objetivos, alcance y planificación	10
2.1. Objetivo general	10
2.2. Objetivos contractuales	11
2.3. Objetivos específicos	13
2.3.1. Representación del tráfico basada en flujos (OBJ-01) . .	13
2.3.2. Preprocesamiento canónico y esquema de características (OBJ-02)	14
2.3.3. Entorno de aprendizaje por refuerzo (OBJ-03)	15
2.3.4. Agente de decisión binaria basado en QRDQN (OBJ-04)	16
2.3.5. Comparación con línea base supervisada (OBJ-05)	17
2.3.6. Evaluación con control de fugas (OBJ-06)	18
2.3.7. Análisis de escala de entrenamiento y eficiencia de datos (OBJ-07)	19
2.3.8. Validación complementaria con tráfico de laboratorio (OBJ-08)	20
2.4. Alcance del trabajo	21
2.5. No objetivos	23
2.6. Resultados esperados	24
2.7. Restricciones	24
2.8. Recursos	25
2.9. Planificación temporal	26
3. Estado del arte	29
3.1. Introducción y alcance del capítulo	29
3.2. Detección de intrusiones y monitorización de seguridad en red .	31
3.3. Representación del tráfico de red	33
3.4. Conjuntos de datos públicos para la detección de intrusiones en red	36
3.5. CICIDS2017 como benchmark interno principal	37
3.6. Aprendizaje supervisado y aprendizaje profundo para NIDS . . .	41

3.7. Fundamentos del aprendizaje por refuerzo	43
3.8. Q-Learning profundo y aprendizaje por refuerzo distribucional .	45
3.9. Aprendizaje por refuerzo para la detección de intrusiones y la defensa cibernética	47
3.10. Clasificación como RL y decisiones de seguridad binarias	50
3.11. Riesgos metodológicos en NIDS basado en ML/DL/RL	52
3.12. Protocolos de evaluación, métricas y reproducibilidad	54
3.13. Eficiencia de datos y evaluación a escala de entrenamiento . . .	56
3.14. Brecha de investigación y posicionamiento de esta tesis	57
4. Diseño del sistema	60
4.1. Visión general metodológica	60
4.2. Fuente de datos y representación de la entrada	62
4.2.1. CSVs de flujos de CICIDS2017	63
4.2.2. Mapeo de etiquetas binarias	63
4.2.3. Representación tabular a nivel de flujo	64
4.3. Limpieza de datos y preprocesamiento	64
4.3.1. Normalización de columnas y coerción numérica	64
4.3.2. Valores inválidos, valores ausentes e imputación	65
4.3.3. Exclusión de campos susceptibles de fuga	66
4.4. Esquema canónico de características	66
4.4.1. Características canónicas de flujo	66
4.4.2. Máscara de presencia/ausencia	67
4.4.3. Vector de observación final	69
4.5. Formulación del aprendizaje por refuerzo	72
4.5.1. Espacio de observación	72
4.5.2. Espacio de acciones	72
4.5.3. Función de recompensa	73
4.5.4. Estructura del episodio	74
4.5.5. Estado del entorno y protocolo de transición	75

4.5.6.	Limitaciones de la formulación conjunto-de-datos-como-entorno	76
4.6.	Agente QRDQN	78
4.6.1.	Papel algorítmico	78
4.6.2.	Función de pérdida de regresión cuantílica	79
4.6.3.	Factor de descuento $\gamma = 0$: formulación de un solo paso	80
4.6.4.	Estrategia de exploración	81
4.6.5.	Configuración del entrenamiento	82
4.6.6.	Procedencia de los hiperparámetros	82
4.6.7.	Persistencia del modelo y el preprocesamiento	84
4.7.	Marco de implementación	84
4.7.1.	Pila de procesamiento de datos y aprendizaje supervisado	86
4.7.2.	Gestión de artefactos	86
4.7.3.	Controles de reproducibilidad	87
5.	Protocolo experimental	88
5.1.	Preguntas experimentales	88
5.2.	Datos, unidad de análisis y fases	90
5.3.	Preprocesamiento y control de fuga	90
5.4.	Perfil congelado main-v1	91
5.5.	Matriz primaria	92
5.5.1.	Partición aleatoria y día completo	92
5.5.2.	Escalera de tamaños	92
5.5.3.	Sensibilidad a la semilla del modelo	92
5.5.4.	Cuatro escenarios retenidos dirigidos	93
5.6.	Random Forest emparejado	93
5.7.	Controles auxiliares	93
5.8.	Jerarquía de evaluación	94
5.9.	Métricas y reglas de lectura	94
5.10.	Fase 2: inferencia offline y diagnósticos	95

5.11. Contrato de artefactos y cierre de campaña	96
5.12. Plataforma experimental y preflight	97
5.13. Estado del protocolo	97
6. Resultados	99
6.1. Condiciones generales y trazabilidad	99
6.2. Evidencia histórica previa a la campaña final	100
6.2.1. MAIN histórica: capacidad de aprendizaje intradistribución	100
6.2.2. Precisión de muestreo y duplicados históricos	103
6.2.3. Controles A, B y proxy temporal histórico	104
6.2.4. Random Forest histórico como referencia interpretativa .	104
6.2.5. Inferencia histórica de Fase 2	106
6.3. Inserción de la campaña final	106
6.4. Síntesis provisional	107
7. Discusión	109
7.1. Rendimiento intradistribución frente a generalización	109
7.2. Duplicados y continuidad entre particiones	110
7.3. Lectura sensible al coste	110
7.4. QRDQN frente a Random Forest	111
7.5. Conjunto de datos como entorno y $\gamma = 0$	112
7.6. Eficiencia de datos y sensibilidad a la semilla	112
7.7. Sensibilidad por escenario	113
7.8. Fase 2 y cambio de dominio	113
7.9. Amenazas a la validez y evidencia más fuerte	114
8. Limitaciones	115
8.1. Benchmark y particiones	115
8.2. Formulación binaria y contextual	116
8.3. Variabilidad experimental	116
8.4. Cobertura de modelos y escenarios	117

8.5. Fase 2 y validez externa	117
8.6. Evidencia todavía pendiente	117
8.7. Alcance operacional	118
9. Consideraciones éticas y legales	119
9.1. Uso dual y alcance estrictamente defensivo	119
9.2. Privacidad y tratamiento de los datos	120
9.3. Riesgo operativo y asimetría de errores	121
9.4. Reproducibilidad como práctica responsable	122
10. Conclusiones	123
10.1. Problema abordado y sistema construido	123
10.2. Contribución metodológica	124
10.3. Qué respalda la evidencia disponible	124
10.4. Qué no respalda todavía	125
10.5. Por qué la partición aleatoria no basta	125
10.6. Papel de Random Forest	126
10.7. Aprendizajes sobre RL con un conjunto de datos como entorno .	126
10.8. Cuestiones reservadas a la campaña final	127
10.9. Cierre provisional	127
Bibliografía	129
A. Manual de reproducibilidad	145
A.1. Código, revisión y datos	145
A.2. Entorno de software	146
A.3. Caché canónica sin escalar	146
A.4. Preflight de la plataforma experimental	147
A.5. Campaña seca y ejecución controlada	148
A.6. Semillas y subconjuntos	148
A.7. Artefactos por ejecución	149

A.8. Exportación y recuperación	149
A.9. Agregación y generación de figuras	150
A.10.Reproducción de la memoria	150
B. Entorno hardware y software	152
B.1. Contrato de la plataforma final	152
B.2. Evidencia ambiental pendiente	153
B.3. Entorno histórico de la MAIN previa	153
B.4. Contratos de dependencias	154
B.5. Entorno de construcción de la memoria	154
C. Esquema canónico completo	156
D. Catálogo de artefactos	164
D.1. Esquema de ejecución final	164
D.2. Artefactos auxiliares	166
D.3. Productos agregados	166
D.4. Catálogo histórico previo a campaña	166
D.5. Git, LFS y exportación	168
D.6. Estado antes de la campaña	168

Índice de figuras

2.1. Diagrama de Gantt del proyecto	28
4.1. Pipeline general del sistema	62
4.2. Construcción del vector de observación	70
4.3. Interacción agente-entorno	74
4.4. Arquitectura QRDQN del perfil main-v1	79
5.1. Visión general de la campaña experimental final	89
5.2. Composición por día de CICIDS2017	90
5.3. Jerarquía de evaluación y generalización	95
5.4. Pipeline de inferencia offline de la Fase 2	96
6.1. Trazabilidad desde el diseño hasta la tesis	100
6.2. Curvas de entrenamiento de la MAIN histórica	102
6.3. Matriz de confusión de la MAIN histórica	103
6.4. Intervalos bootstrap históricos	103
6.5. Duplicados en la partición aleatoria histórica	104
6.6. Matriz histórica del proxy por días	105
6.7. Comparación histórica entre QRDQN y Random Forest	105
6.8. Matriz histórica de Random Forest por día	106
6.9. Resultados históricos de Fase 2	107

Índice de tablas

1.1. Especificación verificable del proyecto	5
2.1. Objetivo contractual C1	11
2.2. Objetivo contractual C2	12
2.3. Objetivo contractual C3	12
2.4. Objetivo contractual C4	13
2.5. Matriz de trazabilidad de objetivos	22
2.6. Catálogo de restricciones del proyecto	25
2.7. Recursos hardware y software del proyecto	26
2.8. Hitos y fases del proyecto	27
3.1. Comparación de conjuntos de datos públicos de NIDS	38
3.2. Trabajos de RL para NIDS	49
4.1. Agrupación lógica de las 76 características canónicas	68
4.2. Matriz de recompensa del entorno del defensor binario	73
4.3. Lectura multiobjetivo de la función de recompensa	75
4.4. Hiperparámetros congelados del perfil main-v1	85
6.1. Estructura de los resultados finales	101
6.2. Métricas de la MAIN histórica	102

B.1. Procedencia del entorno experimental final	153
B.2. Runtime de la MAIN histórica	154
B.3. Contratos de dependencias	154
B.4. Entorno de construcción documental	155
C.1. Esquema canónico completo	157
D.1. Contrato de artefactos por ejecución	165
D.2. Trabajos auxiliares de la campaña	165
D.3. Catálogo de evidencia histórica	167

Índice de algoritmos

1.	Mapeo al esquema canónico y construcción de la observación . .	71
2.	Paso de decisión del entorno	77
3.	Entrenamiento del defensor QRDQN	83

Glosario de acrónimos y términos

Acrónimos

ACK	<i>Acknowledgement</i> , flag TCP que confirma recepción de datos.
CIC	Canadian Institute for Cybersecurity (Universidad de Nuevo Brunswick), creador del conjunto de datos CICIDS2017.
CICIDS2017	Conjunto de datos de referencia para la detección de intrusiones en red, publicado por el CIC en 2017.
CPU	Unidad central de procesamiento (<i>Central Processing Unit</i>).
CSE	Communications Security Establishment, organismo coautor del conjunto CSE-CIC-IDS2018.
CSV	Valores separados por comas (<i>comma-separated values</i>), formato de los ficheros tabulares usados por el proyecto.
DDoS	Denegación de servicio distribuida (<i>Distributed Denial of Service</i>).
DDPG	<i>Deep Deterministic Policy Gradient</i> , algoritmo actor-crítico citado como antecedente de RL continuo.
DL	Aprendizaje profundo (<i>Deep Learning</i>).

DQN	<i>Deep Q-Network</i> , algoritmo de aprendizaje por refuerzo profundo basado en valores.
DRL	Aprendizaje por refuerzo profundo (<i>Deep Reinforcement Learning</i>).
ECE	<i>Explicit Congestion Notification Echo</i> , flag TCP incluido entre las estadísticas de flujo.
F1	Media armónica de la precisión y el recall.
FIN	Flag TCP que indica cierre ordenado de una conexión.
FN / FNR	Falso negativo (ataque no bloqueado) / tasa de falsos negativos.
FP / FPR	Falso positivo (tráfico benigno bloqueado) / tasa de falsos positivos.
GPU	Unidad de procesamiento gráfico (<i>Graphics Processing Unit</i>).
HIDS	Sistema de detección de intrusiones en host (<i>Host Intrusion Detection System</i>).
HTTP	<i>Hypertext Transfer Protocol</i> , protocolo de aplicación citado como ejemplo de tráfico de red.
IC	Intervalo de confianza.
ID	Identificador; en esta tesis se evita como característica cuando puede introducir fuga de información.
IDS	Sistema de detección de intrusiones (<i>Intrusion Detection System</i>).
IoT	Internet de las cosas (<i>Internet of Things</i>).
IP	Protocolo de Internet; las direcciones IP se excluyen de las características del modelo por riesgo de fuga.
IPFIX	<i>IP Flow Information Export</i> , estándar de exportación de registros de flujo.

IPS	Sistema de prevención de intrusiones (<i>Intrusion Prevention System</i>).
JSON	<i>JavaScript Object Notation</i> , formato usado para configuraciones, métricas y manifiestos de artefactos.
KDD	KDD Cup 1999, conjunto de datos histórico de detección de intrusiones.
MCC	Coefficiente de correlación de Matthews (<i>Matthews Correlation Coefficient</i>).
ML	Aprendizaje automático (<i>Machine Learning</i>).
MLP	Perceptrón multicapa (<i>Multilayer Perceptron</i>).
NIDS	Sistema de detección de intrusiones en red (<i>Network Intrusion Detection System</i>).
NIST	National Institute of Standards and Technology, organismo de referencia en guías técnicas de seguridad.
NSL-KDD	Revisión depurada del conjunto KDD Cup 1999.
POMDP	Proceso de decisión de Markov parcialmente observable (<i>Partially Observable Markov Decision Process</i>).
PSH	<i>Push</i> , flag TCP incluido entre las estadísticas de flujo.
QRDQN	<i>Quantile Regression Deep Q-Network</i> , variante distribucional de DQN empleada en este trabajo.
RF	<i>Random Forest</i> , línea base supervisada de comparación.
RL	Aprendizaje por refuerzo (<i>Reinforcement Learning</i>).
RST	<i>Reset</i> , flag TCP que indica reinicio abrupto de una conexión.
RUN_ID	Identificador único de ejecución usado para localizar configuración, métricas y artefactos reproducibles.
SB3	Stable-Baselines3, biblioteca de algoritmos de RL; en el proyecto se usa junto con <code>sb3-contrib</code> .
SDN	Redes definidas por software (<i>Software-Defined Networking</i>).

SHA-256	Función hash criptográfica usada para verificar identidad de artefactos o particiones.
SIEM	Gestión de eventos e información de seguridad (<i>Security Information and Event Management</i>).
SOAR	Orquestación, automatización y respuesta de seguridad (<i>Security Orchestration, Automation and Response</i>).
SYN	Flag TCP usado en el establecimiento de conexión.
TCP	Protocolo de control de transmisión (<i>Transmission Control Protocol</i>).
TFG	Trabajo Fin de Grado.
TLS	Seguridad de la capa de transporte (<i>Transport Layer Security</i>).
TN / TP	Verdadero negativo / verdadero positivo.
UNSW-NB15	Conjunto de datos de detección de intrusiones de la Universidad de Nueva Gales del Sur (2015).
URG	<i>Urgent</i> , flag TCP incluido entre las estadísticas de flujo.

Términos

Bandido	contextual	Formulación de decisión de un solo paso en la que la acción no modifica los estados futuros. En esta tesis describe la consecuencia de usar $\gamma = 0$ sobre un conjunto de datos estático.
Benchmark		Conjunto de datos y protocolo usados como referencia de evaluación. CICIDS2017 funciona aquí como benchmark interno, no como prueba de rendimiento en producción.

Bootstrap	Técnica de remuestreo usada para estimar intervalos de confianza sobre el conjunto de prueba fijo. En esta memoria cuantifica precisión de muestreo, no varianza entre semillas de entrenamiento.
Buffer de repetición	Memoria de transiciones usada por DQN/QRDQN para muestrear minibatches durante el entrenamiento y reducir correlaciones inmediatas entre pasos.
Cambio de dominio	Diferencia entre la distribución usada para entrenar el modelo y la distribución sobre la que se aplica después; por ejemplo, CICIDS2017 frente a tráfico de laboratorio.
Despliegue activo	Integración operativa que modifica tráfico real, por ejemplo bloqueando paquetes o flujos. Este trabajo no implementa despliegue activo.
Escalador	Transformación de normalización ajustada sobre entrenamiento, aquí <code>StandardScaler</code> ; debe reutilizarse en inferencia sin recalcularse sobre datos de prueba o laboratorio.
F1	Métrica que combina precisión y recall mediante su media armónica; resulta útil cuando hay desbalanceo, pero no sustituye a la lectura de falsos positivos y falsos negativos.
Falso negativo	Error en el que un ataque se clasifica como benigno o se permite. En la tesis es el error más penalizado por la recompensa.
Falso positivo	Error en el que tráfico benigno se clasifica como ataque o se bloquea. Afecta a disponibilidad y carga operativa.

Flujo	Resumen tabular de una comunicación de red, normalmente bidireccional, descrita por estadísticas agregadas de paquetes, bytes, tiempos y flags.
Inferencia offline	Aplicación del modelo sobre ficheros ya capturados y almacenados, sin actuar sobre la red en vivo.
Holdout dirigido	Protocolo que retiene un CSV completo como prueba y entrena con los restantes. La campaña selecciona cuatro escenarios concretos y no ejecuta un leave-one-CSV-out exhaustivo.
Línea base	Modelo comparativo usado para contextualizar los resultados del método principal; en esta tesis, Random Forest es la línea base supervisada principal.
Máscara de presencia	Vector binario que indica si cada característica canónica procede de una columna fuente disponible o ha sido imputada por ausencia de columna.
Partición aleatoria	División entrenamiento/prueba por filas, normalmente estratificada. Es útil como comprobación de aprendizaje, pero puede sobreestimar generalización cuando hay duplicados o dependencia entre filas.
Partición por día	División que separa ficheros completos de CICIDS2017 según día o escenario de captura. Evalúa generalización de forma más estricta que la partición aleatoria.
Pipeline	Secuencia reproducible de pasos de procesamiento, entrenamiento, evaluación o inferencia. En esta tesis incluye carga, limpieza, mapeo canónico, escalado, modelo y artefactos de salida.
Precisión	Proporción de predicciones positivas que son correctas. Para la clase ataque, mide cuántos bloqueos o alertas corresponden realmente a ataques.

Recall	Proporción de ejemplos reales de una clase que el modelo detecta. Para la clase ataque, mide qué fracción de ataques se bloquea o identifica.
Recorte por percentiles	Limitación de valores crudos a rangos observados en entrenamiento, por ejemplo entre los percentiles 0.5 y 99.5. Sirve para acotar valores extremos en inferencia.
Recorte por z-score	Limitación de valores escalados a un umbral absoluto de desviaciones estándar. En Fase 2 ayuda a controlar valores fuera de distribución después del escalado.
Red objetivo	Copia estabilizadora de la red de valores usada en DQN/QRDQN para calcular objetivos temporales. Con $\gamma = 0$ no contribuye al objetivo de esta tesis.
Semilla de modelo	Semilla que controla inicialización y aprendizaje. Se registra como <code>model_seed</code> y puede variar sin cambiar el test.
Semilla de partición	Semilla que controla selección y división aleatoria de filas. Se registra como <code>split_seed</code> .

Introducción

Este capítulo sitúa el problema de investigación, delimita una especificación verificable para el prototipo y adelanta las contribuciones, el alcance y la organización de la memoria. La intención es distinguir desde el inicio entre la evidencia que puede obtenerse mediante experimentación fuera de línea (*offline*) y cualquier afirmación de preparación para un despliegue operativo.

1.1 Motivación

Las infraestructuras de red modernas generan volúmenes de tráfico que superan con creces lo que los analistas humanos pueden inspeccionar manualmente. Una única red empresarial puede producir millones de registros de flujo al día, y los despliegues a gran escala agregan flujos procedentes de miles de endpoints, servicios en la nube y enlaces entre centros de datos (Scarfone et al. 2007). Distinguir la comunicación legítima de la actividad maliciosa dentro de este volumen es uno de los desafíos operativos centrales en la seguridad de redes. Las respuestas tradicionales se apoyan en sistemas de detección de intrusiones en red (NIDS, del inglés *Network Intrusion Detection Systems*) que aplican coincidencia de firmas, umbrales estadísticos o reglas específicas de protocolo

para señalar actividad sospechosa. Estos enfoques son efectivos frente a patrones de amenaza conocidos, pero requieren actualizaciones manuales constantes de reglas, resultan en gran medida ineficaces frente a variantes novedosas de ataque y generan volúmenes de alertas que habitualmente desbordan a los equipos de seguridad.

El aprendizaje automático (ML, del inglés *Machine Learning*) ofrece una perspectiva complementaria. En lugar de codificar el conocimiento experto como reglas estáticas, los modelos de ML pueden aprender patrones estadísticos a partir de datos de tráfico etiquetados y generalizar hacia instancias de ataque no vistas previamente. A medida que el campo ha madurado, las representaciones a nivel de flujo se han convertido en un sustrato práctico para este aprendizaje: cada conexión de red se resume como un conjunto fijo de características estadísticas que cubren duración, conteos de paquetes, totales de bytes, tiempos entre llegadas, distribuciones de flags TCP y métricas basadas en tasas (Sperotto et al. 2010; Umer et al. 2017). Esta reducción hace abordable el aprendizaje a gran escala y evita las complicaciones legales y técnicas de la inspección profunda de paquetes, que resulta cada vez más ineficaz de todos modos a medida que crece la fracción de tráfico cifrado en internet.

La limitación clave del aprendizaje supervisado estándar en este contexto es que trata la detección de intrusiones como un problema de clasificación simétrico, minimizando el error de clasificación global sin tener en cuenta las consecuencias operativas de los distintos tipos de error (Axelsson 2000). En la práctica, esas consecuencias son profundamente asimétricas. Un falso negativo permite a un actor de amenazas avanzar sin ser detectado y puede conducir a exfiltración de datos, despliegue de ransomware o interrupción del servicio. Un falso positivo genera una alerta o un bloqueo de tráfico que consume tiempo del analista, pero rara vez causa daños duraderos. El aprendizaje por refuerzo (RL, del inglés *Reinforcement Learning*) proporciona un marco natural para codificar esta asimetría directamente en el objetivo de aprendizaje: al diseñar una función de recompensa que penaliza los ataques no detectados más severamente que las

falsas alarmas, un agente de RL puede ser guiado hacia políticas que reflejen las prioridades de riesgo operativo real en lugar de la exactitud agregada (Sutton et al. 2018; Lee et al. 2002). Explorar cómo un agente basado en la *Quantile Regression Deep Q-Network* (QRDQN) navega estos compromisos sobre datos a nivel de flujo es la motivación central de esta tesis (Dabney et al. 2018).

1.2 Definición del problema

1.2.1 Desafíos metodológicos

Aplicar el aprendizaje por refuerzo a la detección de intrusiones en red plantea varios desafíos metodológicos que van más allá de sustituir un algoritmo de RL por un clasificador estándar. El primer desafío es la calidad de los datos. Los bancos de pruebas públicos (*benchmarks*) de NIDS como CICIDS2017 son ampliamente utilizados, pero contienen artefactos conocidos de su flujo de procesamiento (*pipeline*) de extracción de flujos, entre ellos ruido en las etiquetas, características duplicadas, fallos de implementación de CICFlowMeter y campos estadísticos que codifican inadvertidamente el contexto del día de captura (Ring et al. 2019; Engelen et al. 2021; Lanvin et al. 2023). Los modelos entrenados sin auditar estos artefactos pueden aprender a reconocer firmas de extracción o identificadores específicos de escenario en lugar de comportamiento de ataque genuino, produciendo puntuaciones infladas en el *benchmark* que no se transfieren a ningún entorno nuevo.

El segundo desafío es la fuga de información. Muchos estudios publicados de NIDS evalúan los modelos utilizando particiones aleatorias por filas sobre todo el tráfico disponible, lo que puede situar flujos altamente correlacionados a ambos lados del límite entrenamiento-prueba. Más allá de las filas duplicadas, la fuga puede introducirse a través de direcciones IP, marcas temporales absolutas, identificadores del flujo (*Flow IDs*) y números de puerto que son específicos de

un escenario de ataque programado en lugar de indicativos del comportamiento malicioso en general (Arp et al. 2020; Kaufman et al. 2011). Los pasos de preprocesamiento como el escalado de características, aplicados globalmente antes de la partición, pueden contaminar adicionalmente la evaluación al permitir que la información distribucional de la partición de prueba influya en la normalización del entrenamiento. Un estudio riguroso no puede limitarse a elegir un algoritmo de alto rendimiento y reportar su exactitud sobre un conjunto de datos público dividido aleatoriamente.

El tercer desafío es la comparación justa contra una línea base (*baseline*) de un método estándar. Los métodos de aprendizaje por refuerzo profundo (DRL, del inglés *Deep Reinforcement Learning*) pueden ser costosos de ajustar, sensibles a los hiperparámetros y más lentos en converger que los modelos supervisados tabulares. El bosque aleatorio (*Random Forest*) y los ensamblados de árboles con potenciación de gradiente (*gradient boosting*) son líneas base sólidas para la detección de intrusiones a nivel de flujo y pueden alcanzar puntuaciones casi perfectas en CICIDS2017 bajo condiciones de evaluación favorables (H. Liu et al. 2019; Apruzzese et al. 2022). Un defensor basado en RL no puede evaluarse de manera significativa sin una comparación bajo el mismo protocolo frente a dichas líneas base. La pregunta no es si QRDQN alcanza alta exactitud en una sola ejecución, sino si su formulación sensible al coste ofrece alguna ventaja empírica sobre un modelo supervisado bien ajustado bajo esquemas de características, protocolos de partición y métricas de evaluación equivalentes.

1.2.2 Especificación verificable del proyecto

Como adaptación al contexto de investigación de la especificación propia de un proyecto de desarrollo de software (PDS), esta tesis formula el trabajo como un sistema experimental verificable y no como un producto de bloqueo listo para operar. La Tabla 1.1 fija el problema que se aborda, los límites que

preservan la seguridad metodológica y la evidencia exigida para considerar cubierto cada compromiso.

Especificación verificable del prototipo experimental	
Necesidad	Construir y evaluar un defensor offline que clasifique flujos de red como PERMIT o BLOCK, haciendo explícita la asimetría entre permitir un ataque y bloquear tráfico legítimo.
Entradas	CSV de flujos de CICIDS2017 y, para la comprobación complementaria, flujos extraídos de tráfico de laboratorio; todos se adaptan al contrato canónico y se procesan sin usar identificadores ni otros campos propensos a fuga.
Proceso	Limpieza y escalado ajustados solo con entrenamiento, representación de $76 + 76 = 152$ dimensiones, entrenamiento QRDQN, comparación con Random Forest y escalera de validación con artefactos persistidos.
Salidas	Predicciones binarias experimentales, métricas por clase y artefactos de ejecución trazables; BLOCK permanece como salida de inferencia offline y no acciona un cortafuegos ni una red real.
Verificación	Los compromisos C1–C4, especificados en las Tablas 2.1 a 2.4, requieren código auditable, ejecuciones comprometidas y resultados interpretados dentro de las restricciones declaradas.
Límite de la afirmación	El rendimiento sobre CICIDS2017 y sobre una captura concreta de laboratorio constituye evidencia experimental acotada; no certifica generalización a producción ni bloqueo activo seguro.

Tabla 1.1: Especificación verificable del prototipo experimental, adaptada al contexto de un proyecto de desarrollo de software. Fuente: elaboración propia.

1.3 Enfoque propuesto

Esta tesis formula la tarea defensiva como un problema de aprendizaje por refuerzo offline con el conjunto de datos como entorno (Levine et al. 2020). La elección del entorno offline es deliberada: desplegar un agente de RL que deba interactuar con una red en vivo para aprender generaría riesgos de seguridad inaceptables, ya que un agente en exploración tomaría inevitablemente acciones

subóptimas durante el entrenamiento. Al tratar un conjunto de datos etiquetado estático como el entorno, el agente puede entrenarse y evaluarse sin ninguna exposición a una red en vivo. El tráfico de red se procesa en un esquema canónico de flujo de 76 características fijo, independiente del formato nativo de cualquier conjunto de datos específico. Este contrato canónico separa la nomenclatura de columnas propia de la extracción del espacio de observación del modelo y hace posible procesar tanto flujos de CICIDS2017 como tráfico capturado en un laboratorio doméstico cerrado a través del mismo pipeline sin modificación alguna.

Para gestionar esquemas heterogéneos al transitar entre herramientas de extracción, se añade una máscara de presencia/ausencia de 76 valores, produciendo una observación de 152 dimensiones. Cada posición indica si existe una columna fuente mapeada para la característica canónica correspondiente. En CICIDS2017 las 76 columnas canónicas están cubiertas y la máscara es constante; en la Fase 2 puede revelar columnas no disponibles. Los valores no finitos de una columna existente se sanean antes del mapeo y no cambian su indicador. Esta semántica, más estrecha que una máscara de validez fila a fila, coincide con la implementación actual y hace auditable la compatibilidad estructural entre extractores.

La tarea de decisión se mapea a un espacio de acciones binario: **PERMIT** para el tráfico clasificado como benigno, y **BLOCK** para el tráfico clasificado como ataque. Un agente *Quantile Regression Deep Q-Network* (Dabney et al. 2018), implementado a través de la biblioteca **sb3-contrib** (Stable-Baselines3 Contributors 2023), se entrena dentro de un entorno compatible con Gymnasium (Farama Foundation 2023). El entorno aplica una función de recompensa asimétrica que penaliza los falsos negativos más severamente que los falsos positivos, codificando directamente las prioridades de riesgo operativo descritas anteriormente (Cárdenas et al. 2006). El conjunto de datos CICIDS2017 (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017) sirve como benchmark interno primario, procesado a través de un pipeline de carga curado

que excluye los campos propensos a fuga de información antes de que comience cualquier entrenamiento del modelo. El rendimiento se evalúa frente a una línea base de Random Forest bajo el mismo esquema de características y protocolo de partición (Engelen et al. 2021). Un segundo nivel de validación extiende la evaluación ejecutando inferencia offline sobre características de flujo extraídas de tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial, ofreciendo una indicación parcial y de validez externa limitada del comportamiento del sistema ante un cambio de dominio.

1.4 Contribución de la tesis

Las contribuciones se expresan mediante los cuatro compromisos contractuales del Capítulo 2. Esta formulación permite separar el producto construido, la evidencia de evaluación y los límites de cada conclusión, en lugar de presentar el rendimiento de una sola partición como una demostración global.

C1 — Pipeline experimental reproducible con contrato de datos canónico. La primera contribución es un pipeline offline de extremo a extremo que fija una representación de $76 + 76 = 152$ dimensiones y conserva los artefactos necesarios para reconstruir cada resultado. La Tabla 2.1 especifica este compromiso y el Anexo D documenta el contrato de evidencia.

C2 — Defensor RL y jerarquía de validación con control de fugas. La segunda contribución es la implementación de QRDQN como defensor binario y el diseño de una campaña que separa MAIN aleatoria, día completo, tamaño, semilla y cuatro escenarios retenidos, junto con controles directos, barajado de etiquetas y duplicados. La Tabla 2.2 fija el compromiso; el Capítulo 5 distingue el diseño aprobado de la evidencia todavía pendiente.

C3 — Decisión sensible al coste con lectura multiobjetivo. La tercera contribución convierte la asimetría entre ataques permitidos y tráfico legítimo bloqueado en un elemento auditable del diseño, no en una interpretación

posterior de la exactitud. La Tabla 2.3 enlaza esta contribución con la matriz de recompensa y su lectura de seguridad e impacto operativo (Tabla 4.3).

C4 — Comparación honesta y transferencia offline acotada. La cuarta contribución incorpora Random Forest bajo particiones emparejadas y una Fase 2 que reutiliza modelo y preprocesamiento sin reajuste. La evidencia histórica se conserva como antecedente; la comparación final y la inferencia fresca dependen de la campaña. La Tabla 2.4 fija el compromiso y la Tabla 6.1 define la evidencia necesaria, sin atribuir a BLOCK ningún efecto operativo.

1.5 Alcance y limitaciones

El alcance de esta investigación se limita estrictamente a la evaluación offline, tabular y a nivel de flujo. Las salidas PERMIT y BLOCK son clasificaciones binarias experimentales sobre registros de flujo estáticos; el sistema no realiza bloqueo en vivo de cortafuegos, descarte de paquetes, manipulación de `iptables` ni prevención de intrusiones en línea. El entorno no simula un adversario activo cuyas acciones posteriores dependan de las decisiones del defensor, lo que significa que el sistema no opera como un plano de control de toma de decisiones secuenciales completo. Estas restricciones son decisiones de diseño deliberadas que hacen que la evaluación sea abordable, reproducible y segura. No son omisiones que deban subsanarse en iteraciones futuras de este mismo prototipo.

CICIDS2017 se utiliza como benchmark interno primario para la experimentación controlada. Los resultados sobre este conjunto de datos deben interpretarse como evidencia sobre el comportamiento bajo un benchmark específico y bien caracterizado, no como prueba de generalización a redes de producción arbitrarias. El tráfico de laboratorio, generado por el operador en un entorno doméstico cerrado y no adversarial, se utiliza como nivel de evaluación secundario para examinar el cambio de dominio; dada su procedencia, su validez externa es limitada y, además, también es inferencia offline y no

despliegue en vivo. La tesis ocupa una posición metodológica entre la evaluación pura sobre benchmark y la validación de despliegue completo, apuntando a la reproducibilidad e interpretabilidad del marco experimental más que a la operatividad en producción.

1.6 Estructura del documento

Tras esta introducción, el Capítulo 2 fija los objetivos contractuales y específicos, el alcance, las restricciones y la planificación. El Capítulo 3 revisa el estado del arte de los NIDS basados en flujos, los conjuntos de datos públicos, el aprendizaje automático y el aprendizaje por refuerzo aplicados a detección de intrusiones, junto con sus riesgos metodológicos. El Capítulo 4 presenta el diseño del sistema: datos, limpieza anti-fuga, contrato canónico, formulación del entorno y agente QRDQN. El Capítulo 5 define el protocolo experimental, las particiones, las métricas, la línea base y la inferencia offline de la Fase 2.

El Capítulo 6 fija la estructura de incorporación de la campaña final y conserva, claramente separada, la evidencia histórica previa. El Capítulo 7 organiza la discusión por generalización, duplicados, coste, comparación de modelos, escala, semillas, escenarios y cambio de dominio; el Capítulo 8 expone las limitaciones y el Capítulo 9 delimita las implicaciones éticas. El Capítulo 10 presenta una conclusión provisional que podrá cerrarse desde los artefactos finales. Los anexos reúnen reproducción, entorno, esquema canónico y catálogo de artefactos.

Objetivos, alcance y planificación

Este capítulo concreta los objetivos verificables de la tesis, delimita su alcance y documenta los recursos y la planificación que sostienen la evaluación experimental.

2.1 Objetivo general

El objetivo general de esta tesis es diseñar, implementar y evaluar rigurosamente un defensor de ciberseguridad basado en aprendizaje por refuerzo, de carácter fuera de línea (*offline*), que toma decisiones binarias sensibles al coste (PERMIT o BLOCK) sobre observaciones de flujos de red estandarizados. Este objetivo se persigue mediante un flujo de procesamiento (*pipeline*) experimental controlado que opera íntegramente sobre datos de flujo etiquetados y estáticos, priorizando el rigor metodológico y la reproducibilidad sobre cualquier pretensión de preparación para el despliegue.

El enfoque se fundamenta en la observación de que la detección de intrusiones implica costes inherentemente asimétricos: permitir un ataque conlleva consecuencias que son, en general, considerablemente más graves que las de generar una falsa alarma. Una formulación basada en recompensas hace explícita

esta asimetría en el objetivo de aprendizaje, en lugar de dejarla implícita en un ajuste posterior de umbrales. La pregunta central que guía el trabajo es si un algoritmo de RL distribucional como QRDQN puede configurarse, entrenarse y evaluarse de forma que demuestre un comportamiento sensible al coste y medible sobre un benchmark público bien estudiado, manteniéndose a la vez comparable con sólidas líneas base supervisadas bajo las mismas condiciones controladas.

2.2 Objetivos contractuales

El objetivo general se concreta en cuatro objetivos contractuales, C1 a C4, que constituyen el compromiso verificable de esta tesis. Siguiendo el estilo de especificación de un proyecto de desarrollo de software, cada uno se formula con una descripción cerrada, un tipo, un criterio de verificación anclado en artefactos concretos y los capítulos en los que se materializa (Tablas 2.1 a 2.4). La cobertura efectiva de estos compromisos se retoma en la síntesis de resultados y en las conclusiones.

C1 — Pipeline experimental reproducible con contrato de datos canónico	
Descripción	Construir un pipeline offline de extremo a extremo (ingesta de CICIDS2017, limpieza anti-fuga, esquema canónico de $76 + 76 = 152$ dimensiones, entrenamiento y evaluación) cuyas cifras sean reproducibles a partir de los artefactos persistidos por ejecución: RUN_ID, configuración, métricas, escalador y percentiles de entrenamiento.
Tipo	Producto (infraestructura experimental).
Verificación	Esquema de artefactos 3.0 y catálogo del Anexo D; Algoritmos 3 y 1 contrastados con el código fuente; pruebas automatizadas del repositorio.
Capítulos	4, 5 y 6; anexos A y D.

Tabla 2.1: Especificación del objetivo contractual C1. Fuente: elaboración propia.

C2 — Defensor RL evaluado mediante una escalera de validación con control de fugas

Descripción	Entrenar un agente QRDQN distribucional sobre la representación canónica —con $\gamma = 0$, es decir, una formulación de bandido contextual sensible al coste— y evaluarlo mediante MAIN aleatoria, día completo, escalera de tamaños, semillas 42–46, cuatro escenarios retenidos y controles auxiliares, haciendo explícito qué pregunta resuelve cada bloque.
Tipo	Producto y evaluación.
Verificación	Especificación congelada de campaña, artefactos completos por ejecución y agregado validado. En este borrador el diseño está completado y la ejecución final permanece pendiente.
Capítulos	4, 5 y 6.

Tabla 2.2: Especificación del objetivo contractual C2. Fuente: elaboración propia.

C3 — Decisión sensible al coste con lectura multiobjetivo

Descripción	Codificar en la función de recompensa la asimetría de costes del dominio con una lectura multiobjetivo explícita entre <i>seguridad</i> (minimizar ataques admitidos; el falso negativo, con recompensa -5.0 , es el coste dominante) e <i>impacto operativo</i> (minimizar el bloqueo de tráfico legítimo; falso positivo con recompensa -2.0), y analizar los resultados por clase bajo esa lectura.
Tipo	Diseño y evaluación.
Verificación	Tablas 4.2 y 4.3 contrastadas con <code>src/rl_defender_env.py</code> ; tasas de falsos positivos y falsos negativos reportadas por clase en el capítulo de resultados y leídas en clave multiobjetivo en la discusión.
Capítulos	4, 6 y 7.

Tabla 2.3: Especificación del objetivo contractual C3. Fuente: elaboración propia.

C4 — Comparación honesta con línea base y transferencia a tráfico externo	
Descripción	Comparar QRDQN con una línea base Random Forest bajo el mismo protocolo (misma representación canónica, mismas particiones y mismas métricas) y comprobar la transferencia del pipeline mediante la inferencia offline de la Fase 2 sobre tráfico de laboratorio generado por el operador, declarando su validez externa limitada.
Tipo	Evaluación.
Verificación	Pares QRDQN–Random Forest en MAIN, un millón de filas, día completo y cuatro holdouts; inferencia de Fase 2 vinculada a la MAIN fresca. En este borrador estos artefactos finales están pendientes.
Capítulos	5, 6, 7 y 8.

Tabla 2.4: Especificación del objetivo contractual C4. Fuente: elaboración propia.

2.3 Objetivos específicos

2.3.1 Representación del tráfico basada en flujos (OBJ-01)

El primer objetivo específico es extraer y normalizar datos de tráfico de red en registros tabulares de flujos, garantizando que el formato de entrada sea compatible tanto con benchmarks públicos como CICIDS2017 como con capturas de laboratorio offline. Los flujos de red son resúmenes bidireccionales de intercambios de paquetes que comparten una quintupla común, y su estructura tabular compacta y de anchura fija los hace muy adecuados como entradas para modelos de aprendizaje automático (Sperotto et al. 2010; Umer et al. 2017). Alcanzar este objetivo requiere auditar las características disponibles en los ficheros CSV generados por CICFlowMeter, identificar qué campos contienen señal estadística legítima y cuáles representan artefactos de extracción o posibles vectores de fuga de información.

Este objetivo implica también definir un contrato de características estable

que tienda un puente entre las publicaciones académicas en formato CSV y las exportaciones brutas de CICFlowMeter. Un contrato estable significa que el conjunto de características utilizado como entrada al modelo no varía entre experimentos, que los campos excluidos por razones de fuga se excluyen de forma consistente, y que cualquier persona que lea el código fuente puede verificar exactamente qué medidas llegan al modelo. Esta estabilidad es un requisito previo para todos los objetivos posteriores, ya que todos los demás componentes del pipeline dependen de un espacio de observación bien definido y auditable.

Criterio de verificación (OBJ-01). El contrato de características es auditable en el código (`src/canonical_schema.py`) y se aplica de forma idéntica a los CSV de CICIDS2017 y a los flujos de laboratorio de la Fase 2; el capítulo de diseño documenta el mapeo, los campos excluidos y su justificación (Algoritmo 1).

2.3.2 Preprocesamiento canónico y esquema de características (OBJ-02)

El segundo objetivo específico es definir y hacer cumplir un esquema canónico estricto de 76 características junto con una máscara de presencia/ausencia de 76 valores, lo que da lugar a un vector de observación estable de 152 dimensiones. Las 76 características canónicas se extraen de estadísticas de estilo CICFlowMeter que cubren la duración del flujo, conteos de paquetes y bytes, distribuciones de tiempos de llegada, conteos de flags TCP, estadísticas de longitud de paquetes, resúmenes de periodos activos e inactivos, y métricas de tasa de flujo. Los campos que podrían exponer información de identidad específica del conjunto de datos, como las direcciones IP de origen y destino, las marcas temporales absolutas, los *Flow ID* y los números de puerto que funcionan como proxies directos del tipo de ataque, se excluyen explícitamente a nivel de esquema, en lugar de filtrarse en pasos de preprocesamiento ad hoc.

La máscara de 76 valores registra si existe una columna fuente mapeada para cada característica canónica. No es una máscara de validez por fila: los valores no finitos de una columna existente se sanean antes del mapeo y conservan indicador 1. Sobre CICIDS2017 nativo la cobertura es completa y la máscara es constante; al pasar a otro extractor puede revelar columnas ausentes. Juntos, valores y máscara constituyen el contrato estructural del que dependen los componentes posteriores.

Criterio de verificación (OBJ-02). El código y las pruebas fijan 152 dimensiones y el capítulo de diseño documenta la semántica exacta de la máscara. Cada ejecución final deberá registrar los mismos nombres y orden de características.

2.3.3 Entorno de aprendizaje por refuerzo (OBJ-03)

El tercer objetivo específico es construir una interfaz conjunto-de-datos-como-entorno compatible con Gymnasium (Farama Foundation 2023) que presente los registros de flujo al agente como una secuencia de observaciones. El entorno envuelve un conjunto de datos etiquetado estático de modo que cada llamada a `step()` devuelve un registro de flujo como observación, junto con la recompensa calculada a partir de la acción anterior del agente y la etiqueta verdadera del último registro. Esta formulación permite que los bucles de entrenamiento de RL estándar, desarrollados para entornos interactivos, se apliquen directamente a conjuntos de datos de flujo offline sin modificar el algoritmo.

El entorno debe implementar una función de recompensa asimétrica y sensible al coste que penalice los falsos negativos más que los falsos positivos, reflejando los perfiles de riesgo en ciberseguridad en los que los ataques no detectados son operacionalmente más perjudiciales que las falsas alarmas (Lee et al. 2002; Cárdenas et al. 2006). Los pesos de recompensa específicos se tratan como parámetros experimentales configurables en lugar de constantes

universales fijas, ya que la relación de costes adecuada depende del modelo de amenaza y del contexto operativo del escenario de despliegue. El entorno debe también admitir reinicios reproducibles, gestión de límites de episodios y particiones del conjunto de datos configurables, de modo que la misma interfaz pueda utilizarse tanto para el entrenamiento como para la evaluación bajo distintos modos de partición.

Criterio de verificación (OBJ-03). El entorno implementado (`src/rl_defender_env.py`; Algoritmo 2) expone la interfaz `Gymnasium` con la matriz de recompensa asimétrica registrada en el artefacto de configuración de cada ejecución (Tabla 4.2) y con evaluación determinista sin barajado.

2.3.4 Agente de decisión binaria basado en QRDQN (OBJ-04)

El cuarto objetivo específico es configurar y entrenar un agente Quantile Regression Deep Q-Network (Dabney et al. 2018), aprovechando su enfoque distribucional para la estimación del valor, para navegar el espacio de acciones binario PERMIT/BLOCK a partir de las observaciones canónicas de 152 dimensiones. QRDQN extiende el DQN estándar estimando la distribución completa de los retornos esperados en lugar de solo la media, utilizando un conjunto de cuantiles aprendidos y una pérdida de regresión cuantílica. Esta representación distribucional resulta de interés en un contexto sensible al coste porque proporciona información más rica sobre la dispersión y la forma de los posibles resultados, no solo el retorno promedio, y en principio permite que la política sea más conservadora ante los peores casos.

La implementación práctica utiliza el módulo QRDQN de la biblioteca `sb3-contrib` (Stable-Baselines3 Contributors 2023), que proporciona una implementación estable y probada del algoritmo con arquitectura de red configurable, buffer de repetición, esquema de exploración e hiperparámetros de

entrenamiento. El objetivo incluye seleccionar una configuración razonable por defecto, validar que el bucle de entrenamiento converge en el benchmark interno, y persistir todos los artefactos del modelo entrenado junto con la configuración y el estado de preprocesamiento utilizados para producirlos. La reproducibilidad exige que un modelo almacenado pueda cargarse y evaluarse de forma idéntica en una fecha posterior sin necesidad de reconstruir el pipeline de entrenamiento completo desde cero.

Criterio de verificación (OBJ-04). El perfil `main-v1` y el contrato de persistencia están completados. El criterio empírico se cerrará cuando la MAIN fresca complete 3 000 000 de pasos y sus artefactos superen validación directa y agregación.

2.3.5 Comparación con línea base supervisada (OBJ-05)

El quinto objetivo específico es implementar una línea base supervisada robusta, utilizando principalmente Random Forest (H. Liu et al. 2019), para ofrecer una evaluación comparativa justa, con el mismo protocolo, del valor empírico del agente QRDQN. Random Forest se elige como línea base principal porque está bien establecido para la clasificación tabular, gestiona interacciones no lineales entre características sin un preprocesamiento extenso, es robusto ante características irrelevantes y tiende a obtener buenos resultados en benchmarks de NIDS basados en flujos (Apruzzese et al. 2022). Utilizar el mismo esquema de características, el mismo protocolo de partición y las mismas métricas de evaluación tanto para el agente de RL como para la línea base es esencial; cualquier discrepancia en el preprocesamiento, la disponibilidad de características o las condiciones de evaluación comprometería la validez de la comparación.

Esta comparación cumple varios propósitos. En primer lugar, determina si la formulación QRDQN ofrece alguna ventaja empírica sobre un clasificador supervisado bien ajustado en condiciones controladas. En segundo lugar,

proporciona una comprobación de cordura sobre la dificultad de la tarea en el benchmark, ya que un Random Forest que alcanza puntuaciones muy altas contextualiza lo que QRDQN logra. En tercer lugar, fundamenta cualquier discusión sobre el diseño de la función de recompensa sensible al coste en evidencia concreta. Si ambos modelos se evalúan utilizando las mismas métricas ponderadas por coste, la comparación revela si la función de recompensa de RL conduce realmente al agente hacia un punto de operación diferente al de un modelo supervisado entrenado con los mismos datos, o si ambos enfoques convergen a un comportamiento similar.

Criterio de verificación (OBJ-05). La implementación y la matriz de RF emparejada están definidas. El objetivo se cerrará con artefactos finales para MAIN, un millón de filas, día completo y los cuatro escenarios retenidos.

2.3.6 Evaluación con control de fugas (OBJ-06)

El sexto objetivo específico es diseñar pipelines de preprocesamiento y protocolos de validación que impidan estrictamente la fuga de información en todos los niveles de evaluación. Esto requiere la exclusión programática de campos identificativos, como las direcciones IP de origen y destino, las marcas temporales absolutas, los Flow ID y los proxies directos de puertos, en la fase de carga de datos antes de que comience cualquier entrenamiento de modelos (Arp et al. 2020; Kaufman et al. 2011). Estos campos se excluyen a nivel de esquema mediante el contrato canónico de características, en lugar de depender de un filtrado ad hoc que podría aplicarse de forma inconsistente en distintas ejecuciones o versiones del conjunto de datos.

Las transformaciones de escalado deben ajustarse exclusivamente sobre la partición de entrenamiento y luego aplicarse a las particiones de validación y prueba utilizando únicamente los parámetros aprendidos de los datos de entrenamiento. Ajustar un escalador sobre el conjunto de datos completo antes de la partición permite que la información distribucional del conjunto de prueba

influya en la normalización de las características de entrenamiento, una forma sutil de fuga de información que puede inflar el rendimiento reportado en conjuntos de datos con una estructura temporal o de escenario marcada. El escalador ajustado se persiste como parte del artefacto de la ejecución para que pueda recargarse y aplicarse de forma idéntica durante la inferencia de la Fase 2 sobre tráfico de laboratorio, garantizando que se utilice la misma normalización para flujos fuera de distribución sin que ninguna información sobre su distribución llegue al estado del escalador.

Criterio de verificación (OBJ-06). La exclusión de identificadores, la caché sin escalar, el ajuste exclusivo sobre entrenamiento y la separación de semillas están implementados. La campaña deberá aportar validación directa, etiquetas barajadas, duplicados, día completo y cuatro holdouts con artefactos actuales.

2.3.7 Análisis de escala de entrenamiento y eficiencia de datos (OBJ-07)

El séptimo objetivo específico es evaluar el rendimiento tanto del agente de RL como de las líneas base supervisadas bajo distintas cantidades de datos de entrenamiento, identificando las características de eficiencia de datos de la formulación propuesta y evaluando con qué rapidez converge QRDQN en comparación con las alternativas supervisadas. Los métodos de RL profundo pueden ser muy demandantes en cuanto a muestras, y su dinámica de aprendizaje difiere sustancialmente de la de los métodos de árboles de decisión en conjunto. Comprender cuánto tráfico etiquetado requiere cada enfoque para alcanzar un rendimiento útil es prácticamente relevante: el tráfico de ataque etiquetado es costoso de recopilar, y los escenarios de despliegue reales pueden no tener acceso a los grandes conjuntos de datos equilibrados disponibles en los benchmarks académicos (Ring et al. 2019).

Los experimentos de escala de entrenamiento utilizan subconjuntos de en-

trenamiento progresivamente más grandes extraídos de la misma partición, midiendo cómo cambian la precisión, el recall, el F1, la tasa de falsos positivos y la tasa de falsos negativos a medida que crece el presupuesto. La comparación entre QRDQN y Random Forest en cada nivel de presupuesto revela si la formulación de RL tiene un requisito mayor de muestras, si converge más lentamente, o si alcanza una meseta de rendimiento final diferente. Estos resultados se interpretan como evidencia interna sobre esta combinación específica de formulación y benchmark, no como afirmaciones generales sobre la eficiencia de datos de RL en todos los contextos de NIDS.

Criterio de verificación (OBJ-07). El protocolo fija seis puntos lógicos entre 100 000 filas y el conjunto completo, con presupuestos proporcionales. El objetivo se considerará alcanzado cuando el agregado final valide todos los puntos; este borrador no reporta cifras de escala.

2.3.8 Validación complementaria con tráfico de laboratorio (OBJ-08)

El octavo objetivo específico es construir un pipeline de inferencia offline robusto capaz de ingerir tráfico capturado por el operador en un entorno de laboratorio doméstico cerrado y no adversarial, y producir predicciones a partir de un modelo entrenado en CICIDS2017. Este nivel de validación ofrece una indicación parcial, de validez externa limitada, de si el esquema canónico de características y el modelo entrenado se comportan de forma razonable sobre flujos extraídos de un entorno de red distinto del benchmark, potencialmente utilizando una versión distinta de CICFlowMeter o una configuración de captura diferente. El cambio de dominio entre la distribución de entrenamiento y la distribución de inferencia es un reto fundamental para los modelos de NIDS y rara vez se evalúa en estudios que solo evalúan sobre un único conjunto de datos público (Apruzzese et al. 2022; Layeghy et al. 2023).

El pipeline de inferencia aplica recorte opcional por percentiles y z-score

para gestionar los valores de características brutas fuera de distribución que caen fuera del rango observado durante el entrenamiento en CICIDS2017. El escalador ajustado de la Fase 1 se carga y aplica sin reajuste, preservando la integridad de la separación entre entrenamiento y prueba. La máscara de presencia/ausencia en el vector de observación codifica explícitamente cualquier característica que no haya podido calcularse a partir de la captura de laboratorio, haciendo visible el caso de datos incompletos en la entrada de predicción en lugar de imputarlo silenciosamente. El resultado esperado de este objetivo es un conjunto de artefactos de predicción y métricas que permitan comparar directamente los resultados del tráfico de laboratorio con los de la validación de la Fase 1, proporcionando evidencia concreta sobre cómo varía el rendimiento bajo un cambio de dominio.

Criterio de verificación (OBJ-08). El pipeline histórico acredita viabilidad técnica. El cierre final exige una inferencia fresca vinculada a la MAIN nueva, con su escalador, percentiles, predicciones, métricas y diagnósticos, manteniendo la validez externa limitada.

La Tabla 2.5 resume qué objetivos ya están cerrados como diseño y cuáles necesitan evidencia final. Esta separación evita marcar como completo un objetivo empírico solo porque su comando o implementación exista.

2.4 Alcance del trabajo

El trabajo está delimitado por los requisitos de una evaluación offline de un pipeline de aprendizaje automático tabular. Comprende la ingesta de registros de flujo en formato CSV tanto de CICIDS2017 como de capturas de laboratorio, la aplicación de escalado estándar y recorte opcional de valores atípicos, la ejecución de bucles de entrenamiento mediante el framework Stable Baselines3 (`sb3-contrib`), y la generación de artefactos de ejecución estructurados y reproducibles que persisten modelos, métricas, escaladores y ficheros de configuración

Objetivo	Capítulos	Evidencia (ejecución o artefacto)	Estado
OBJ-01	4	Contrato de características en código (<code>canonical_schema.py</code>); Algoritmo 1	Completado
OBJ-02	4	Código y pruebas del vector de 152 dimensiones; semántica de máscara documentada	Diseño completado
OBJ-03	4	<code>rl_defender_env.py</code> ; Algoritmo 2; Tablas 4.2 y 4.3	Completado
OBJ-04	4–6	Perfil <code>main-v1</code> ; MAIN histórica como antecedente; MAIN fresca requerida	Diseño completado; evidencia final pendiente
OBJ-05	5–7	Matriz RF emparejada con QRDQN	Evidencia final pendiente
OBJ-06	4–6	Anti-fuga, escalado train-only y controles de campaña	Diseño completado; evidencia final pendiente
OBJ-07	5–7	Escalera de tamaños y estudio de semillas	Evidencia final pendiente
OBJ-08	5–8	Pipeline histórico y contrato de inferencia fresca	Ejecución final pendiente

Tabla 2.5: Matriz de trazabilidad entre objetivos específicos, capítulos, evidencia y estado antes de la campaña final. Fuente: elaboración propia.

junto a cada ejecución de entrenamiento. La experimentación interna se realiza íntegramente sobre el conjunto de datos CICIDS2017, utilizando particiones explícitas entre entrenamiento y prueba y una escalera de validación multietapa para evaluar el comportamiento del modelo bajo condiciones progresivamente más estrictas antes de probarlo con tráfico capturado externamente.

El alcance incluye la MAIN aleatoria, los controles directos, la partición completa por días, la escalera de tamaños, cinco semillas de modelo sobre un millón de filas, cuatro holdouts dirigidos, RF emparejado y la inferencia offline de laboratorio. Los cuatro holdouts no constituyen un leave-one-CSV-out exhaustivo. Cada bloque aporta evidencia diferenciada y forma parte del diseño aprobado.

2.5 No objetivos

Esta tesis excluye explícitamente el desarrollo de monitorización de red en línea o la interceptación de paquetes en tiempo real. El sistema no se integrará con cortafuegos, reglas de `iptables`, controladores de redes definidas por software (SDN, del inglés *Software-Defined Networking*), ni sistemas de prevención de intrusiones (IPS, del inglés *Intrusion Prevention System*) activos. Estas exclusiones existen porque el despliegue en producción requeriría un diseño de sistema sustancialmente diferente, una validación de seguridad y una metodología de pruebas operacionales que quedan fuera del alcance de un estudio académico controlado. La tesis está diseñada como un prototipo experimental, no como un producto operativo destinado a su uso en producción.

La tesis tampoco pretende resolver el problema más amplio de las operaciones cibernéticas adversariales multiagente o la interrupción de ataques con dependencia de secuencias, donde el estado del entorno cambia dinámicamente en respuesta a las acciones del defensor. En la formulación actual, cada registro de flujo se clasifica de forma independiente, y el conjunto de datos no reacciona a las decisiones del agente. Esta es una limitación conocida del enfoque *conjunto-de-datos-como-entorno*, e implica que el sistema no puede modelar la adaptación adversarial, la persistencia del atacante ni las cadenas de ataques multietapa que abarcan muchas decisiones individuales de flujo. Por último, el objetivo no es demostrar que QRDQN represente el estado del arte definitivo para todas las tareas de NIDS, sino evaluar su comportamiento bajo un marco experimental cuidadosamente controlado, libre de fugas y reproducible, y compararlo honestamente con líneas base supervisadas.

2.6 Resultados esperados

El resultado esperado de esta tesis es un pipeline de inferencia offline completamente funcional, documentado y reproducible, respaldado por un análisis empírico detallado. El proyecto producirá artefactos persistentes para todas las ejecuciones de entrenamiento y validación, incluyendo ficheros de modelo entrenado, escaladores ajustados, informes de métricas, salidas de validación y registros de configuración que especifican completamente las condiciones bajo las cuales se produjo cada resultado. Esta disciplina en la gestión de artefactos forma parte de la contribución metodológica: un resultado que no puede reproducirse a partir de su rastro de artefactos proporciona una evidencia más débil que uno que sí puede.

En cuanto al contenido empírico, la campaña está diseñada para producir precisión, *recall*, F1, tasas de falsos positivos y falsos negativos y matrices de confusión para QRDQN y los RF emparejados. La escalera de tamaños, las cinco semillas, el día completo y los cuatro escenarios retenidos caracterizarán dimensiones distintas de robustez. La Fase 2 fresca analizará la transferencia a la captura cerrada de laboratorio. En este borrador estos son resultados esperados del protocolo, no hallazgos; ninguna dirección o magnitud se anticipa.

2.7 Restricciones

El desarrollo del proyecto está condicionado por un conjunto de restricciones que delimitan qué evidencia puede producirse y cómo debe interpretarse. La Tabla 2.6 las cataloga en dos categorías: *factores de dato*, impuestos por las fuentes de datos disponibles, y *factores estratégicos*, decisiones de alcance adoptadas para mantener el proyecto abordable, reproducible y seguro. Ninguna de estas restricciones se oculta en los capítulos posteriores: cada una reaparece,

cuantificada cuando es posible, en los resultados y en las limitaciones.

ID	Restricción	Implicación para el proyecto
<i>Factores de dato</i>		
RES-D1	CICIDS2017 presenta problemas documentados: flujos duplicados, inconsistencias de etiquetado y sensibilidad a la partición.	Los resultados <i>in-distribution</i> se leen como cota optimista; la escalera de validación y el análisis de duplicados son obligatorios.
RES-D2	El mapeo binario colapsa todas las familias de ataque en una única clase BLOCK.	El análisis de errores por familia queda fuera de la ruta principal de RL y exigiría etiquetas de familia preservadas.
RES-D3	El tráfico de la Fase 2 procede de un único laboratorio doméstico cerrado, generado por el operador y no adversarial.	Validez externa limitada: la Fase 2 se interpreta como comprobación de transferencia, no como validación de producción.
RES-D4	No se dispone de tráfico de producción etiquetado de forma independiente.	Las afirmaciones de generalización se restringen al benchmark y al laboratorio.
<i>Factores estratégicos</i>		
RES-E1	Evaluación exclusivamente offline; sin bloqueo en vivo ni integración con cortafuegos, SDN o IPS.	El sistema es un instrumento de medición experimental; un despliegue exigiría diseño y validación adicionales.
RES-E2	Presupuesto acotado en una plataforma GPU experimental.	La matriz se limita a 22 ejecuciones primarias físicas, cinco auxiliares y cuatro holdouts dirigidos.
RES-E3	Cinco semillas solo en el punto de un millón de filas; MAIN completa con una semilla.	La dispersión caracteriza estabilidad local y no varianza global del perfil completo.
RES-E4	Un único algoritmo de RL (QRDQN) y una única línea base supervisada (Random Forest).	Las conclusiones algorítmicas se limitan a esta pareja bajo el protocolo común.

Tabla 2.6: Catálogo de restricciones del proyecto: factores de dato y factores estratégicos. Fuente: elaboración propia.

2.8 Recursos

El proyecto distingue el entorno de desarrollo y construcción documental de la plataforma GPU experimental. La campaña final no presupone proveedor ni hardware: esas características se incorporarán desde los `environment.json` aceptados. La Tabla 2.7 separa requisitos metodológicos, evidencia histórica y

datos pendientes. El inventario ampliado se presenta en el Anexo B.

Recurso	Especificación versión	/	Uso en el proyecto	Procedencia
<i>Hardware</i>				
Plataforma GPU final	Hardware pendiente de observación		Ejecución secuencial de la campaña aprobada	Artefactos finales
Entorno de desarrollo	Equipo local; no caracterizado como plataforma científica final		Código, pruebas, documentación y construcción de la memoria	Entorno de trabajo
<i>Software</i>				
Runtime final	Pendiente de observación		Entrenamiento, evaluación y controles finales	Artefactos finales
Contratos de instalación	<code>requirements.txt</code> , manifiesto GPU y <code>pyproject.toml</code>		Reproducción local, plataforma experimental y pruebas	Manifiestos del repositorio
Runtime histórico	Conservado en el Anexo B		Procedencia de la evidencia previa a campaña	Artefacto histórico
Construcción de la memoria	Windows; TeX Live 2025; <code>latexmk</code> 4.87; <code>biber</code> 2.21		Producción del documento; fuera del runtime del modelo	Tracker de mejora de la memoria

Tabla 2.7: Recursos y procedencia prevista de la información. Los manifiestos expresan contratos; solo los `environment.json` finales podrán acreditar el hardware y runtime de campaña. Fuente: elaboración propia.

El entorno histórico y sus versiones se reportan únicamente como procedencia de la MAIN previa. No se trasladan a la descripción de la campaña final. El entorno local de esta revisión no se utiliza para producir evidencia científica ni para estimar tiempos de ejecución.

2.9 Planificación temporal

La Tabla 2.8 recoge los hitos contractuales y las fases efectivas del proyecto, y la Figura 2.1 las representa como diagrama de Gantt. Las fechas están ancladas a dos fuentes verificables: el calendario contractual del TFG (firma, reunión de inicio y entregas parciales) y la evidencia del propio repositorio (marcas temporales de los artefactos de ejecución y del historial de desarrollo). No se

planifica hacia el futuro: el cronograma documenta lo efectivamente ocurrido.

Hito o fase	Fecha(s)
Firma del contrato	11 de noviembre de 2025
Reunión de inicio	18 de noviembre de 2025
Entregas parciales 1–5	15-dic-2025, 2-feb, 17-mar, 20-may y 15-jun de 2026
Desarrollo e implementación	11 de diciembre de 2025 – 9 de junio de 2026
Redacción de la memoria	12 de diciembre de 2025 – 6 de julio de 2026
Experimentos preliminares A/B/C	12 – 23 de febrero de 2026
Fase 2: captura e inferencia en laboratorio	23 de febrero – 10 de junio de 2026
Entrenamiento del modelo MAIN	9 de junio de 2026
Auditoría histórica de validación y línea base RF	27 – 28 de junio de 2026
Preparación de campaña final y revisión para supervisión	julio – 1 de septiembre de 2026

Tabla 2.8: Hitos contractuales y fases efectivas hasta el cierre de este borrador de supervisión. La ejecución de la campaña final no se representa como completada. Fuente: elaboración propia a partir del calendario contractual y del repositorio.

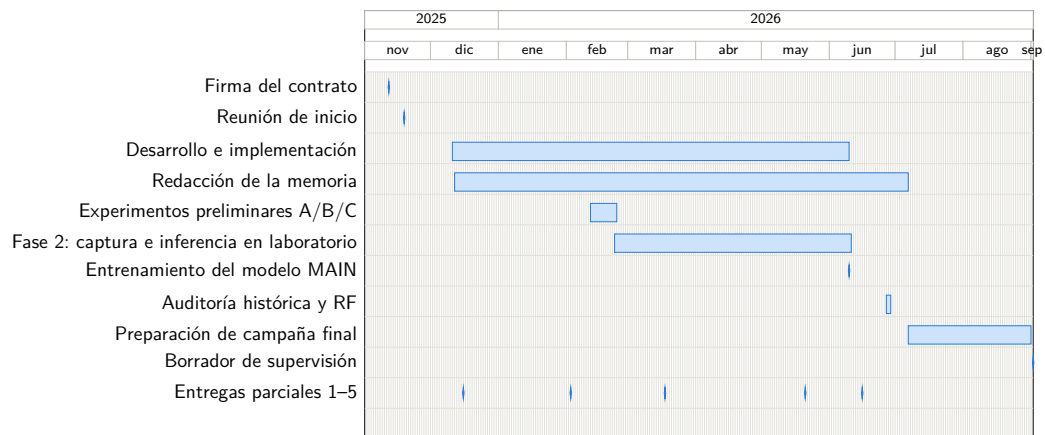


Figura 2.1: Planificación temporal hasta la preparación de la campaña final y el borrador de supervisión. La campaña científica permanece pendiente. Fuente: elaboración propia a partir del calendario contractual y del repositorio.

Estado del arte

Este capítulo revisa el contexto técnico y metodológico que sitúa el prototipo, y delimita el alcance de las afirmaciones que se apoyarán después en los resultados.

3.1 Introducción y alcance del capítulo

Este capítulo mapea el trasfondo técnico de un defensor de ciberseguridad basado en aprendizaje por refuerzo que opera sobre observaciones de red a nivel de flujo. El alcance es deliberadamente más amplio que una comparación de clasificadores binarios. La tesis combina detección de intrusiones en red, representación de características de flujo, conjuntos de datos de referencia públicos, líneas base de aprendizaje supervisado, una formulación de conjunto de datos como entorno al estilo Gymnasium, QRDQN, preprocesamiento con control de fugas, comprobaciones de validación más estrictas, experimentos de eficiencia de datos, y validación planificada o preferida sobre tráfico capturado por el operador en un laboratorio doméstico cerrado y no adversarial.

El capítulo sigue por tanto un embudo. Primero revisa la detección de intrusiones en red y la representación del tráfico, luego sitúa a CICIDS2017 entre

los conjuntos de datos públicos, después discute las líneas base de aprendizaje supervisado y aprendizaje profundo, y finalmente enmarca la formulación de RL tanto dentro de la teoría del aprendizaje por refuerzo como en trabajos previos de RL/DRL para detección de intrusiones y defensa cibernética. Las últimas secciones se centran en los riesgos metodológicos, los protocolos de evaluación, la evidencia sobre escala de entrenamiento y la brecha de investigación que aborda esta tesis.

La afirmación central es cautelosa. El aprendizaje por refuerzo para la detección de intrusiones ya existe, y los benchmarks públicos de NIDS son ampliamente utilizados (W. Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024). Esta tesis no pretende introducir el primer IDS basado en RL, demostrar que QRDQN es el estado del arte para NIDS, ni demostrar una prevención en línea lista para producción. Estudia un pipeline de decisión reproducible, offline, a nivel de flujo, con acciones PERMIT/BLOCK, y evalúa cómo se comporta dicha formulación bajo benchmarks internos controlados, comparación con líneas base y validación progresivamente más estricta.

La revisión también separa tres niveles que con frecuencia se confunden. El primero es la monitorización: recopilar paquetes, flujos, registros o eventos derivados y decidir si son sospechosos. El segundo es la predicción: entrenar un modelo que asigne representaciones de tráfico a etiquetas benignas o maliciosas. El tercero es el control: tomar acciones que modifiquen el estado futuro de la red. Esta tesis trabaja principalmente en los dos primeros niveles. Utiliza el lenguaje de PERMIT y BLOCK porque el entorno expone una decisión de seguridad binaria, pero la implementación actual no es un plano de control en línea. Esa distinción condiciona todas las afirmaciones posteriores.

3.2 Detección de intrusiones y monitorización de seguridad en red

Los sistemas de detección de intrusiones monitorizan la actividad de hosts o redes en busca de evidencias de ataques, usos indebidos o violaciones de políticas (Scarfone et al. 2007). Un sistema de detección de intrusiones en host (HIDS, del inglés *Host Intrusion Detection System*) observa endpoints, eventos del sistema operativo, registros, procesos, cambios en la integridad de archivos o llamadas al sistema. Un sistema de detección de intrusiones en red (NIDS, del inglés *Network Intrusion Detection System*) observa el tráfico que atraviesa enlaces, taps, puertos de mirror de switches, brokers de paquetes, interfaces de red virtual o colectores de flujos. Ambas perspectivas son complementarias: el HIDS puede ver el contexto local de procesos y archivos que los sensores de red no pueden, mientras que el NIDS puede inspeccionar movimientos laterales, escaneos, tráfico de mando y control, y patrones de comunicación agregados sin instalar un agente en cada host.

También es necesario distinguir el IDS de los sistemas de prevención de intrusiones. Un IDS detecta y notifica principalmente actividad sospechosa. Su salida puede ser una alerta, un evento de registro, un registro de flujo enriquecido, un ticket, un evento SIEM o un artefacto forense. Un IPS se posiciona en línea y puede bloquear, descartar, modificar, limitar la tasa, reiniciar o redirigir el tráfico (Scarfone et al. 2007). En la práctica, los productos y arquitecturas frecuentemente difuminan esta frontera porque un motor de detección puede alimentar reglas de firewall, acciones EDR, playbooks SOAR o actualizaciones de política SDN. Para la evaluación, sin embargo, la distinción importa. Un detector puede tolerar un perfil de falsos positivos diferente al de un bloqueador en línea, dado que las alertas falsas principalmente afectan a la carga de trabajo del analista, mientras que los bloqueos falsos pueden interrumpir tráfico legítimo de negocio.

Las taxonomías clásicas de IDS distinguen enfoques basados en firmas, basados en anomalías, basados en especificaciones e híbridos (Scarfone et al. 2007). La detección basada en firmas coteja patrones maliciosos conocidos: secuencias de bytes, predicados de reglas, condiciones de protocolo, cadenas de comandos, rastros de exploits o combinaciones de comportamiento conocido. Su fortaleza es la precisión y la explicabilidad ante amenazas conocidas. Su debilidad es la dependencia del mantenimiento de reglas y la escasa cobertura de ataques nuevos, comportamiento polimórfico, cargas útiles cifradas y variaciones específicas del entorno.

La detección basada en anomalías modela el comportamiento esperado y señala las desviaciones (Sommer et al. 2010; Ring et al. 2019). Esta es la familia más naturalmente asociada con el NIDS basado en ML, pero la palabra anomalía no debe interpretarse como sinónimo automático de aprendizaje no supervisado. Muchos estudios públicos de NIDS son clasificadores supervisados entrenados sobre flujos etiquetados como benignos y de ataque, incluso cuando la motivación más amplia es la detección de anomalías. El reto operativo es que el tráfico benigno es amplio y no estacionario, la prevalencia de ataques es baja, las etiquetas son incompletas, y los cambios en la actividad de negocio pueden parecer anómalos sin ser maliciosos.

La detección basada en especificaciones codifica el comportamiento permitido o esperado, por ejemplo máquinas de estado de protocolo, secuencias de comandos permitidas o flujos de trabajo de control industrial restringidos. Puede ser potente cuando el sistema monitorizado tiene un comportamiento estrecho y estable, pero requiere conocimiento del dominio y mantenimiento. Los sistemas híbridos combinan estas ideas, como reglas de firmas para exploits conocidos, puntuaciones estadísticas de anomalías para comportamiento de flujo inusual, y telemetría de host para confirmación en el endpoint. Los programas modernos de monitorización raramente dependen de un único paradigma de detección.

La generación de alertas y la respuesta activa forman un eje separado.

Un sistema puede limitarse a producir alertas para su posterior clasificación, puede enriquecer las alertas con evidencia contextual, puede recomendar pasos de respuesta o puede desencadenar contención activa. Esta tesis se sitúa deliberadamente en el extremo conservador de ese eje. Estudia un modelo de decisión offline similar a un NIDS sobre filas de flujo extraídas. El modelo emite predicciones PERMIT/BLOCK durante los experimentos, pero no descarta paquetes, no reescribe políticas de firewall, no actualiza reglas SDN, no pone en cuarentena hosts ni opera en un bucle de retroalimentación en vivo. La bibliografía de NIDS es, por tanto, relevante, pero las afirmaciones sobre IPS en producción quedan fuera del alcance soportado.

3.3 Representación del tráfico de red

El tráfico de red puede representarse como paquetes, bytes de carga útil, sesiones, conexiones o flujos. Las representaciones a nivel de paquete preservan la temporización, las cabeceras, los tamaños, las direcciones, los flags y, en ocasiones, parte de la carga útil por paquete. Son adecuadas para el análisis de protocolos a grano fino y el modelado de secuencias, pero resultan costosas de almacenar y procesar en redes de alta velocidad. Las representaciones a nivel de carga útil retienen el contenido de la aplicación y pueden identificar exploits o firmas de malware invisibles en los metadatos. Su utilidad está limitada por el cifrado, las restricciones de privacidad, las restricciones legales y el coste de CPU.

Las representaciones a nivel de sesión o conexión agrupan paquetes pertenecientes al mismo intercambio, habitualmente utilizando la quintupla de IP de origen, IP de destino, puerto de origen, puerto de destino y protocolo. Pueden incluir el estado de la conexión, el comportamiento del handshake, metadatos de la aplicación o atributos derivados de TLS/HTTP. Las representaciones a nivel de flujo son más compactas: agregan paquetes en una ventana temporal

o registro similar a una conexión, y resumen el comportamiento mediante duración, bytes, paquetes, direccionalidad, tiempos entre llegadas, estadísticas de longitud de paquetes, conteos de flags TCP y periodos activos o inactivos (Sperotto et al. 2010; Umer et al. 2017).

Los registros de estilo NetFlow e IPFIX son el punto de referencia operativo para la monitorización de flujos escalable. Routers, switches, sondas y colectores exportan registros de flujo compactos que pueden retenerse durante largos periodos y procesarse a escala. Estos registros son atractivos para el NIDS porque reducen el volumen y evitan la inspección de la carga útil. También tienen limitaciones porque los despliegues exportan a menudo solo un subconjunto de los campos posibles, pueden muestrear el tráfico y pueden omitir estadísticas más ricas utilizadas en conjuntos de datos académicos. Esto crea una brecha entre la telemetría de flujos en producción y los CSV de benchmark con abundantes características.

Las características al estilo CICFlowMeter se sitúan en el lado más rico y orientado a la investigación de esa brecha. CICIDS2017 proporciona PCAPs y CSV de flujos bidireccionales generados, con características como la duración del flujo, conteos de paquetes y bytes en sentido directo e inverso, estadísticas de tiempos entre llegadas, estadísticas de longitud de paquetes, conteos de flags TCP, agregados de longitud de cabecera, tasas de flujo y temporización activa/inactiva (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). Estas características son convenientes para ML tabular porque cada flujo se convierte en una fila de anchura fija, pero dependen de la reconstrucción y el etiquetado offline de PCAPs, así como de las decisiones de extracción de características (Farrukh et al. 2022).

La idoneidad de la representación a nivel de flujo es, por tanto, condicional. Es adecuada para esta tesis porque el defensor implementado consume vectores numéricos de anchura fija, porque CICIDS2017 ya está disponible en formato de flujo, y porque la ruta prevista en Fase 2 también extrae CSV de flujos antes de la inferencia. Está limitada porque se pierden la semántica de la carga útil,

el ordenamiento exacto de paquetes, el contenido de la aplicación y algunas dependencias temporales de múltiples flujos. Los ataques cuya evidencia es principalmente a nivel de contenido, a nivel de host o de comportamiento a largo plazo pueden ser solo indirectamente visibles para un clasificador de flujos (Sperotto et al. 2010; Umer et al. 2017).

La disponibilidad de características es otra limitación. Una característica calculada por CICFlowMeter a partir de PCAPs completos puede no estar disponible en un exportador NetFlow/IPFIX en producción, o puede calcularse de manera diferente. Sarhan et al. abordan este problema mapeando conjuntos heterogéneos de características de NIDS hacia una representación estándar alineada con NetFlow para mejorar la generalizabilidad y la explicabilidad (Sarhan et al. 2021). Esta tesis no adopta el conjunto exacto de características estándar de Sarhan, pero sigue la misma motivación metodológica: definir un contrato de características estable, documentar qué está presente o ausente, y evitar tratar un esquema CSV específico de un benchmark como si fuera universal.

El proyecto actual utiliza un esquema canónico de flujo con 76 valores de características y una máscara de disponibilidad de 76 valores, produciendo observaciones de 152 dimensiones. La máscara registra si existe una columna fuente mapeada; no es una señal de validez por fila, porque los valores no finitos se sanean antes de construirla. Esto importa al pasar de CICIDS2017 a tráfico de laboratorio, donde distintos extractores pueden no producir las mismas columnas. La máscara hace auditable la compatibilidad estructural de la entrada, pero no resuelve por sí sola valores atípicos ni cambio de dominio.

La representación a nivel de flujo también modifica el perfil de privacidad y de fugas. La eliminación de las cargas útiles puede reducir la exposición de privacidad, pero los metadatos aún pueden identificar usuarios, servicios, horarios y escenarios. Las direcciones IP, las marcas temporales absolutas, los Flow ID, la identidad de los endpoints, la estructura del día de captura y los puertos usados como proxies de escenario pueden filtrar etiquetas en lugar de

representar comportamiento de ataque (Arp et al. 2020; Kaufman et al. 2011). La tesis trata por tanto la representación y la selección de características como decisiones de diseño relevantes para la seguridad, no solo como detalles de preprocesamiento.

3.4 Conjuntos de datos públicos para la detección de intrusiones en red

Los conjuntos de datos públicos de NIDS permiten la experimentación compartida, la reproducibilidad y la comparación entre trabajos (Ring et al. 2019; Thakkar et al. 2020; Goldschmidt et al. 2025). También condicionan las conclusiones. Un conjunto de datos refleja su generador de tráfico, su topología, su proceso de captura, su extractor de características, sus etiquetas, sus scripts de ataque y su formato de distribución. El rendimiento sobre un benchmark público es, por tanto, evidencia sobre una tarea controlada, no evidencia directa de preparación para el despliegue (Apruzzese et al. 2022; Layeghy et al. 2023).

KDDCup99 y NSL-KDD son históricamente importantes porque dieron forma a la evaluación temprana de IDS (KDD Cup 1999; Tavallaee et al. 2009). KDDCup99 contiene registros de conexión derivados del tráfico de la era DARPA, con un esquema de 41 características y categorías de ataque como DoS, Probe, R2L y U2R. NSL-KDD redujo algunos problemas de redundancia y facilitó el uso del benchmark, pero mantuvo el mismo origen básico y lógica de características. Estos conjuntos son útiles para comprender la historia de la evaluación de IDS, pero evidencia débil para la defensa moderna basada en flujos: su tráfico, comportamiento de ataques y esquema son antiguos (Tavallaee et al. 2009; Ring et al. 2019; Özgür et al. 2016). En este repositorio, NSL-KDD es contexto histórico, no la ruta final del defensor actual.

UNSW-NB15 es un conjunto de datos de cyber-range más reciente, generado

con tráfico benigno y malicioso contemporáneo, capturas sin procesar y características de ingeniería (Moustafa y Slay 2015; Moustafa y Slay 2016). Sigue siendo útil porque amplió el campo más allá de los benchmarks al estilo KDD e incluye categorías de ataque más ricas. CICIDS2017 y CSE-CIC-IDS2018 son conjuntos de datos de la familia CIC ampliamente utilizados para la investigación de NIDS basada en flujos; ambos proporcionan escenarios de ataque generados en laboratorio y características de flujo derivadas de capturas de paquetes (Sharafaldin et al. 2018; Communications Security Establishment et al. 2018). Bot-IoT y ToN_IoT amplían el panorama hacia IoT e IIoT, donde el comportamiento de los dispositivos, la telemetría y los patrones de ataque difieren de las redes empresariales (Koroniotis et al. 2019; Moustafa, Ahmed et al. 2020; Alsaedi et al. 2020).

La Tabla 3.1 no establece un rango. Muestra que la elección del conjunto de datos debe alinearse con la solidez de las afirmaciones. Los benchmarks históricos sitúan el campo; los conjuntos de datos modernos de laboratorio permiten comparación controlada; los de IoT prueban suposiciones de tráfico diferentes; y el tráfico de laboratorio externo o similar al de producción aporta evidencia de generalización más sólida. Ninguno de estos conjuntos de datos debe tratarse como un sustituto completo de todas las redes reales. Su valor es máximo con preprocesamiento transparente, líneas base sólidas y protocolos que prueben el cambio de distribución, no solo el rendimiento sobre distribuciones aleatorias emparejadas.

3.5 CICIDS2017 como benchmark interno principal

CICIDS2017 es el benchmark interno principal de esta tesis porque proporciona tráfico benigno y de ataque etiquetado, PCAPs sin procesar y CSV de

Conjunto de datos	Papel como benchmark	Representación	Principales fortalezas	Principales advertencias
KDDCup99	Benchmark histórico heredado	Registros de conexión con 41 características artesanales	Muy ampliamente utilizado; línea base sencilla para la bibliografía clásica de IDS (KDD Cup 1999)	Tráfico obsoleto; redundancia; débil como aproximación al NIDS moderno (Ring et al. 2019)
NSL-KDD	Benchmark heredado depurado	Mismo esquema de conexión al estilo KDD	Reduce algunos problemas de registros duplicados; útil para comparación histórica (Tavallaei et al. 2009)	Sigue heredando las suposiciones antiguas de DARPA/KDD y el diseño de características
UNSW-NB15	Conjunto de datos moderno de laboratorio/cyber-range	PCAPs, registros y características tabulares de ingeniería	Taxonomía de ataques más contemporánea y características más ricas (Moustafa y Slay 2015)	Entorno controlado; no constituye evidencia directa de producción
CICIDS2017	Benchmark interno principal de la tesis	PCAPs y CSV de flujos CICFlowMeter	PCAPs públicos; múltiples días de ataque; ricas características de flujo (Sharafaldin et al. 2018)	Problemas conocidos de etiquetado, extracción, duplicación y sensibilidad a la partición (Engelen et al. 2021; Lanvin et al. 2023)
CSE-CIC-IDS2018	Benchmark de mayor tamaño de la familia CIC	PCAPs y flujos al estilo CICFlowMeter	Colaboración más amplia CIC/CSE; comparación moderna útil (Communications Security Establishment et al. 2018)	Advertencias similares sobre generación en laboratorio y artefactos de flujo (L. Liu et al. 2022)
Bot-IoT	Benchmark de IoT/botnets	PCAPs, flujos al estilo Argus, exportaciones CSV	Escenarios de botnet, escaneo, DoS/DDoS y exfiltración orientados a IoT (Koroniotis et al. 2019)	Fuerte desbalanceo y artefactos de topología específicos del conjunto de datos
ToN_IoT	Benchmark de telemetría IoT/IIoT	Flujos de red, registros de host y telemetría de sensores	Perspectiva multimodal de IoT/IIoT (Alsaedi et al. 2020)	Entorno de nicho; los artefactos de identificadores y escenarios siguen siendo posibles

Tabla 3.1: Comparación compacta de los conjuntos de datos públicos de NIDS relevantes para la tesis. Fuente: elaboración propia a partir de las fuentes citadas.

flujos generados con características al estilo CICFlowMeter (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). Fue diseñado para mejorar conjuntos de datos de IDS anteriores mediante perfiles de tráfico contemporáneos, actividad benigna realista y múltiples escenarios de ataque (Sharafaldin et al. 2018; Sharafaldin et al. 2019). Estas propiedades lo hacen adecuado para la experimentación controlada con modelos de NIDS a nivel de flujo.

Su estructura también es atractiva para la tesis. El conjunto abarca múltiples días de captura y familias de ataque, y está disponible tanto como capturas de paquetes sin procesar como en CSV generados (Oyelakin et al. 2023). Los PCAPs están más cerca de la evidencia de medición original. Los CSV facilitan el ML porque CICFlowMeter ya reconstruyó los flujos bidireccionales y calculó características estadísticas. Esa comodidad también es un riesgo: al trabajar solo con CSV resumidos, se puede dejar de comprobar cómo se generaron los flujos, cómo se asignaron las etiquetas y si las filas mantienen la independencia entre las particiones de entrenamiento y prueba.

CICFlowMeter reconstruye flujos de trazas de paquetes mediante claves de endpoint, direccionalidad, lógica de timeout y cálculos de características. Pequeñas diferencias en los parámetros de extracción y en la integridad de los flujos pueden cambiar filas, duraciones, conteos y etiquetas (Pekar et al. 2024). Engelen et al. muestran problemas prácticos en la reconstrucción de flujos y la consistencia de CICIDS2017 (Engelen et al. 2021). Lanvin et al. reportan errores que afectan al rendimiento de detección y que correcciones aparentemente pequeñas pueden producir diferencias métricas significativas (Lanvin et al. 2023; Lanvin et al. 2022). Liu et al. analizan la prevalencia de errores en CIC-IDS-2017 y CSE-CIC-IDS-2018, mientras Rosay et al. ofrecen un análisis más amplio y una discusión orientada a la reconstrucción (L. Liu et al. 2022; Rosay et al. 2022). La crítica de Dube es especialmente relevante para la disciplina en las afirmaciones, porque cuestiona las interpretaciones habituales de puntuaciones elevadas sobre los datos resumidos de CIC-IDS 2017

(Dube 2024).

Estas críticas no hacen inútil a CICIDS2017; hacen más identificable el uso irresponsable. Los patrones comunes incluyen tratar los CSV pregenerados como verdad incuestionable, usar particiones aleatorias por filas de todos los días, conservar identificadores o marcas temporales que exponen el contexto del escenario, informar solo de la exactitud, ignorar el desbalanceo de clases y no documentar si los datos son originales, corregidos, reconstruidos o curados. Las variantes corregidas o reconstruidas pueden ser valiosas, pero no son intercambiables con la distribución original de CSV. Boukhamla y Coronel Gaviro aplican PCA a CICIDS2017 y evalúan esa versión reducida (Boukhamla et al. 2021). Ese caso ilustra por qué una tesis debe nombrar qué variante utiliza, el preprocesamiento aplicado y evitar mezclar resultados de diferentes variantes como si procedieran de un único conjunto de datos.

El repositorio actual responde a estas preocupaciones mediante una copia curada y rastreada, una copia de referencia sin rastrear y limpieza a nivel de cargador en `src/load_cicids2017.py`. El adaptador realiza coerción numérica, tratamiento de valores no válidos, mapeo canónico y exclusión anti-fugas. También ajusta el escalado sobre la partición de entrenamiento y aplica la transformación ajustada a prueba, en vez de ajustar sobre todos los datos. El mapeo canónico en `src/canonical_schema.py` excluye IPs, marcas temporales, Flow ID y puertos específicos de las características canónicas.

El código de evaluación admite particiones aleatorias, por CSV/día y por *holdout* exacto. El rendimiento aleatorio comprueba que modelo e implementación aprenden el benchmark interno. La separación completa por días, el control de etiquetas barajadas, los cuatro holdouts dirigidos y la inferencia offline de laboratorio prueban modos de fallo distintos. Una puntuación alta en el escenario más sencillo no basta para una afirmación de despliegue. CICIDS2017 debe leerse como benchmark interno principal, no como sustituto de validación independiente.

3.6 Aprendizaje supervisado y aprendizaje profundo para NIDS

La mayor parte del NIDS basado en flujos se formula como aprendizaje supervisado: cada registro de tráfico se mapea a una etiqueta benigna, maliciosa o de familia de ataque (H. Liu et al. 2019; Ahmad et al. 2021). Los métodos clásicos incluyen árboles de decisión, Random Forests, *SVMs*, *k-NN*, regresión logística, *Naive Bayes* y conjuntos de árboles con *gradient boosting*. Difieren en sus suposiciones, coste operativo y requisitos de memoria, entrenamiento e inferencia. La regresión logística es simple e interpretable y ofrece una referencia lineal, pero está limitada ante interacciones no lineales. *Naive Bayes* es rápido y sirve como línea base débil, aunque sus supuestos de independencia son irrealistas para estadísticas de flujo correlacionadas. *k-NN* puede capturar vecindades locales, pero escala mal con tablas grandes y es sensible al escalado de características. Las *SVMs* pueden ser potentes en conjuntos pequeños, pero resultan costosas con tablas de flujos grandes y exigen escalado cuidadoso de características y selección del kernel.

Los modelos basados en árboles son especialmente importantes para NIDS tabular. Los árboles de decisión y Random Forests manejan interacciones no lineales, escalas heterogéneas, variables irrelevantes y distribuciones mixtas (H. Liu et al. 2019; Apruzzese et al. 2022). Las familias con *gradient boosting*, como *XGBoost*, *LightGBM* y *CatBoost*, son a menudo aprendices tabulares potentes, pero este capítulo no necesita afirmaciones separadas sobre cada implementación si la tesis no las evalúa. Rodríguez et al. comparan técnicas de ML sobre flujos de CICIDS2017, incluidos flujos obtenidos desde PCAP con Zeek; sus resultados siguen siendo evidencia intra-benchmark, no de despliegue (Rodríguez et al. 2022). El punto metodológico importante es que un clasificador de flujos basado en RL debe compararse con líneas base supervisadas tabulares competentes, no solo con métodos débiles u obsoletos.

Esto es directamente relevante para la tesis. Si cada observación es una fila de flujo etiquetada y la recompensa deriva de la corrección de la clasificación, un clasificador supervisado es la línea base natural. Un agente QRDQN no puede evaluarse únicamente contra su curva de recompensa, pérdida de entrenamiento o una línea base de clase mayoritaria; debe compararse con métodos supervisados sólidos bajo el mismo esquema de características, protocolo de partición, preprocesamiento y métricas. El repositorio ya incluye una línea base de Random Forest alineada con las particiones de QRDQN y artefactos por barrido. La comparación debe limitarse a métricas respaldadas por esos artefactos y al mismo protocolo, en vez de inferir superioridad general.

El aprendizaje profundo amplía el espacio de diseño mediante MLPs, CNNs, RNNs, LSTMs, autoencoders, mecanismos de atención, modelos al estilo transformer y enfoques basados en grafos (Xiao et al. 2021; Asadullah et al. 2023; Sayeeduddin et al. 2022; Nguyen et al. 2022). Los MLPs tratan las características de flujo como vectores numéricos genéricos; las CNNs se utilizan cuando bytes de paquetes, matrices de características o patrones locales se codifican en forma estructurada. Las RNNs y LSTMs se orientan a secuencias temporales de paquetes, flujos o eventos. Los autoencoders apoyan frecuentemente la detección de anomalías al aprender representaciones comprimidas del comportamiento benigno y señalar errores de reconstrucción. Los modelos al estilo transformer y las redes neuronales en grafo son direcciones más recientes para representaciones de tráfico secuenciales y estructurales.

Estas arquitecturas pueden alcanzar puntuaciones de benchmark muy elevadas, sobre todo en conjuntos públicos, pero las revisiones identifican debilidades recurrentes: preprocesamiento poco claro, particiones aleatorias favorables, análisis insuficiente del desbalanceo, validación externa limitada y reproducibilidad insuficiente (H. Liu et al. 2019; Ring et al. 2019; Arp et al. 2020). Un modelo profundo puede sobreajustarse a los artefactos del benchmark igual que un modelo superficial. La familia de modelos por sí sola no resuelve fugas, desbalanceo de clases, disponibilidad de características ni cambio de dominio.

La estandarización de características es otra lección importante del aprendizaje supervisado. Diferentes conjuntos de datos y herramientas exponen conjuntos solapados, pero no idénticos, lo que dificulta la comparación entre conjuntos. Sarhan et al. propusieron un mapeo de características alineado con NetFlow para mejorar la generalizabilidad y la explicabilidad en NIDS (Sarhan et al. 2021). La tesis no adopta ese estándar exacto, pero sí su motivación: un esquema canónico fijo hace auditable la interfaz del modelo y permite comparar las entradas de flujo de CICIDS2017 y de laboratorio a través de un único contrato de preprocesamiento. La máscara de presencia/ausencia extiende esta idea al hacer visibles las características ausentes al modelo y al evaluador.

3.7 Fundamentos del aprendizaje por refuerzo

El aprendizaje por refuerzo estudia cómo un agente selecciona acciones en un entorno para maximizar el retorno (Sutton et al. 2018). Sus conceptos centrales son los estados u observaciones, las acciones, las recompensas, las políticas, las funciones de valor y las transiciones. En un proceso de decisión de Markov, el estado actual resume la información necesaria para la toma de decisiones futura, y las acciones influyen tanto en la recompensa inmediata como en los estados posteriores (Sutton et al. 2018). Esta propiedad secuencial es lo que distingue el RL de la clasificación supervisada ordinaria.

Una política especifica cómo el agente elige acciones a partir de las observaciones. Una función de valor estima el retorno esperado desde un estado, mientras que una función de valor-acción estima el retorno esperado tras tomar una acción particular en un estado. Las recompensas definen el objetivo de la tarea, y el retorno agrega las recompensas futuras, generalmente con descuento. Estas definiciones importan porque cambiar la función de recompensa cambia aquello para lo que el agente está optimizado. En un contexto de seguridad, una recompensa que penaliza más los falsos negativos que los falsos positivos

codifica una preferencia operativa concreta; no es un hecho neutral sobre todas las redes.

La distinción entre tareas episódicas y continuas también es relevante. Las tareas episódicas tienen límites y estados terminales, como un episodio de juego o un escenario simulado de respuesta a incidentes. Las tareas continuas modelan la operación continua, como la monitorización continua de la red. Una formulación estática de conjunto de datos como entorno impone a menudo episodios artificiales: un episodio puede ser un número fijo de filas muestreadas, un recorrido por un conjunto de datos o un archivo. Eso es aceptable para la experimentación, pero es más débil que un entorno donde la acción del defensor modifica el estado siguiente.

El Q-learning clásico es un método de diferencia temporal model-free y off-policy que aprende valores de acción sin requerir un modelo de la dinámica del entorno (Watkins et al. 1992). Los métodos model-free aprenden directamente de la interacción o de transiciones registradas, mientras que los métodos basados en modelo aprenden o utilizan un modelo de transición para la planificación. Los métodos on-policy aprenden sobre la política que está generando el comportamiento actual; los métodos off-policy pueden aprender sobre una política objetivo mientras el comportamiento es generado de manera diferente. DQN y QRDQN heredan el enfoque basado en valores y off-policy. La repetición de experiencias, introducida anteriormente como mecanismo para reutilizar experiencias pasadas, también se volvió central en los sistemas de Q-learning profundo posteriores (Lin 1992).

El RL tabular no es apropiado cuando las observaciones son vectores numéricos de alta dimensión. Una tabla de búsqueda sobre todas las posibles combinaciones de características de flujo es inviable, y pequeños cambios numéricos podrían crear efectivamente estados nuevos. La aproximación de funciones es, por tanto, necesaria. El DRL utiliza redes neuronales para aproximar políticas o funciones de valor sobre espacios de observación grandes. En esta tesis, la observación es un vector de 152 dimensiones: 76 valores canónicos de flujo más

76 valores de máscara de presencia/ausencia. Eso justifica un algoritmo de la familia DQN más que un algoritmo de RL tabular, pero no justifica por sí solo el RL frente al aprendizaje supervisado.

La ciberseguridad puede contener problemas de RL secuenciales genuinos. Un defensor puede aislar un host, parchear un servicio, cambiar el enrutamiento, desplegar una contramedida o elegir dónde inspeccionar a continuación, y esas acciones pueden afectar a las observaciones posteriores y a las oportunidades del atacante. Sin embargo, un conjunto de datos estático de flujos etiquetados es un escenario más débil. Si se muestrea una fila de flujo, se clasifica y va seguida de otra fila que no es causalmente afectada por la acción anterior, el problema se parece más a una clasificación sensible al coste o a un proceso de decisión contextual que a una defensa cibernética autónoma completa (Levine et al. 2020; Fujimoto et al. 2019). La tesis debe mantener esta distinción de forma explícita.

3.8 Q-Learning profundo y aprendizaje por refuerzo distribucional

El Q-Learning Profundo sustituye una función de valor-acción tabular por una red neuronal, haciendo aplicable el RL basado en valores a observaciones de alta dimensión (Mnih et al. 2015). DQN popularizó dos mecanismos estabilizadores: la memoria de repetición y una red objetivo separada (Mnih et al. 2015). La memoria de repetición rompe las correlaciones a corto plazo muestreando transiciones pasadas para el entrenamiento y mejora la reutilización de datos. Una red objetivo estabiliza los objetivos de bootstrapping manteniendo el cálculo del objetivo separado de los pesos de la red online, que cambian rápidamente.

Variantes posteriores de DQN abordaron debilidades conocidas. Double

DQN reduce el sesgo de sobreestimación desacoplando la selección de acciones de la evaluación del objetivo (Hasselt et al. 2016). Dueling DQN separa la estimación del valor del estado y la ventaja de la acción, lo que puede ayudar cuando muchas acciones tienen valores similares en un estado (Wang et al. 2016). Prioritized Experience Replay muestrea transiciones según la señal de aprendizaje en lugar de hacerlo de forma uniforme (Schaul et al. 2016). Rainbow combinó varias mejoras en un único agente de la familia DQN, incluyendo double Q-learning, redes dueling, repetición priorizada, retornos de múltiples pasos, aprendizaje distribucional y exploración con ruido (Hessel et al. 2018).

El aprendizaje por refuerzo distribucional cambia el objeto que se estima. En lugar de estimar únicamente el retorno esperado, aproxima la distribución de los retornos posibles (Bellemare et al. 2017). C51 representa la distribución del retorno sobre un soporte categórico fijo. QRDQN usa regresión cuantílica para aproximar los cuantiles del retorno, evitando la misma suposición de soporte fijo (Dabney et al. 2018). QRDQN no es, por tanto, un paradigma separado de DQN; es un miembro distribucional y basado en valores de la familia DQN.

La motivación de seguridad es intuitiva, pero debe formularse con cuidado. Los falsos positivos y los falsos negativos tienen consecuencias asimétricas, y los errores raros de alto coste pueden importar más de lo que sugiere una puntuación media. Una estimación distribucional del valor puede ser útil para estudiar la estructura del retorno bajo recompensas asimétricas. Puede revelar si una decisión tiene una distribución de retornos estrecha o amplia bajo el modelo aprendido. Sin embargo, el RL distribucional no proporciona automáticamente incertidumbre calibrada, seguridad operativa, comportamiento sensible al riesgo ni superioridad sobre el DQN escalar. Esas propiedades requerirían reglas de decisión explícitas, calibración, objetivos de riesgo en la cola o criterios de evaluación más allá de simplemente entrenar un agente distribucional (Bellemare et al. 2017; Dabney et al. 2018).

En esta tesis, QRDQN es una elección algorítmica exploratoria. Es adecuado para un espacio de acciones binario discreto y observaciones de flujo de

dimensión moderadamente alta, y está disponible a través de la implementación de SB3-Contrib utilizada por el proyecto (Stable-Baselines3 Contributors 2023; Farama Foundation 2023). La elección algorítmica es coherente: dos acciones, observaciones numéricas, aprendizaje de valores off-policy, repetición y un objetivo distribucional. La afirmación evidencial sigue siendo empírica. QRDQN no debe describirse como probadamente superior para NIDS a menos que las comparaciones con el mismo protocolo frente a DQN, Random Forest y otras líneas base apoyen esa afirmación.

3.9 Aprendizaje por refuerzo para la detección de intrusiones y la defensa cibernética

El RL y el DRL para la detección de intrusiones ya forman una bibliografía activa (W. Yang et al. 2026; Kheddar et al. 2025; Gueriani et al. 2024; Adawadkar et al. 2022). Los trabajos previos incluyen clasificadores de intrusiones al estilo DQN, enfoques actor-crítico, formulaciones de entorno adversarial, agentes de selección de características, sistemas orientados a SDN o IoT, y simulaciones de defensa cibernética autónoma. El campo es heterogéneo, por lo que los trabajos no deben tratarse como directamente comparables a menos que su formulación de tarea, conjuntos de datos, acciones, recompensas y protocolos de validación estén alineados.

Una rama está próxima a esta tesis: la detección de intrusiones con conjunto de datos como entorno. Lopez-Martin et al. reformularon la detección supervisada de intrusiones como un problema de RL profundo tratando los registros etiquetados como estados, las predicciones de clase como acciones y la recompensa como función de la corrección (Lopez-Martin, Carro et al. 2020). Su trabajo posterior extendió una formulación relacionada de RL offline a varios conjuntos de datos, incluyendo CICIDS2017 y UNSW-NB15 (Lopez-Martin,

Sanchez-Esguevillas et al. 2021). Ren et al. utilizaron DRL para la selección de características y la clasificación en CSE-CIC-IDS2018 (Ren et al. 2022). Estos trabajos muestran que las formulaciones de RL para IDS tienen precedentes.

Otros trabajos de RL orientados a IDS exploran DQN, Double DQN, actor-crítico, SAC, DDPG y diseños al estilo Rainbow (Caminero et al. 2019; Z. Li et al. 2023; Alavizadeh et al. 2022; Hossain 2025). Los detalles varían ampliamente. Algunos trabajos utilizan clasificación binaria, otros etiquetas de ataque multiclase, selección de características, control de umbral o acciones específicas de SDN. Muchos siguen evaluándose sobre conjuntos de datos públicos estáticos. Eso los convierte en antecedentes relevantes para la tesis, pero también limita la comparabilidad directa. Un trabajo que usa NSL-KDD con particiones aleatorias y recompensas de corrección no es evidencia de que un defensor QRDQN offline generalice de CICIDS2017 a tráfico de laboratorio.

Una segunda rama estudia la defensa cibernética genuinamente secuencial. Los enfoques POMDP y de grafo de ataque modelan acciones defensivas bajo incertidumbre (Hu et al. 2020). CybORG proporciona un entorno al estilo gym para agentes cibernéticos autónomos (Standen et al. 2021). Los trabajos más amplios de defensa cibernética autónoma discuten agentes que seleccionan contramedidas en redes simuladas o emuladas (Palmer et al. 2023; Center for Security and Emerging Technology et al. 2023). Estos escenarios son conceptualmente diferentes de esta tesis porque las acciones del defensor pueden alterar el estado futuro. Representan una forma más fuerte de problema de RL que la clasificación estática por filas.

Tabla 3.2: Síntesis de trabajos de RL/DRL para NIDS y límites de comparabilidad con esta tesis. La relación indicada se infiere de cada fuente y de la formulación QRDQN offline sobre flujos de CICIDS2017 adoptada en este trabajo. Fuente: elaboración propia a partir de las fuentes citadas.

Fuente	Formulación de RL	Datos / ámbito	Límite de evaluación	Relación / diferencia con esta tesis
Malik y Singh Saini (2023)	RL; el algoritmo no es verificable desde el registro primario público.	NIDS genérico; datos no verificables.	El registro no expone protocolo ni métricas comparables.	Antecedente temático, no comparación directa con QRDQN/CICIDS2017.
Malik y Saini (2023)	Técnicas de RL; algoritmo no verificable el registro primario público.	NIDS genérico; datos no verificables.	El registro no expone protocolo ni métricas comparables.	Antecedente genérico, sin comparabilidad empírica directa.
Sanusi (2023)	DRL para IDS binario.	NSL-KDD; tarea multiclase se reduce a binaria.	la Tesis offline sobre benchmark; no evidencia de despliegue.	Antecedente de IDS binario con DRL, pero datos e implementación distintos.
B. Yang et al. (2022)	DQN y gra-diente de política con entrenamiento adversarial.	IDS a nivel de paquete y flujo; CICD-DoS2019.	Evaluación de benchmark; sin evidencia de despliegue operativo.	Diseño paquete/flujo más rico y datos distintos de QRDQN/CICIDS2017.
Cevallos et al. (2023)	M. Revisión/taxonomía de DRL-IDS, no un algoritmo único.	Literatura de IDS en IoT.	Síntesis bibliográfica, sin benchmark propio.	Contexto de decisiones DRL en IoT, no línea base empírica.
Y. Li et al. (2025)	DDPG.	Detección de ataques red; datos verificables.	El registro no expone protocolo ni métricas comparables.	Familia de RL distinta; solo antecedente a nivel de título.
Kanimozhi et al. (2025)	DRL-IDS con LFTS-RNN y PC-JTFOA.	SDN; KDD y WPPD.	NSL-Partitiones de benchmark, y no despliegue SDN vivo.	Tarea, modelo y datos distintos; no equivale a una evaluación QRDQN offline.
Shaikh et al. (2025)	CNN-LSTM con DQN y PPO.	IoMT sanitario; CIoMT2024.	Benchmark especializado; el CI-trabajo deja la detección <i>zero-day</i> como trabajo futuro.	Propuesta IoMT específica, no línea base comparable con CICIDS2017.

La tesis pertenece, por tanto, principalmente a la primera rama: detección offline a nivel de flujo formulada a través de una interfaz de RL. Puede referenciar la defensa cibernética autónoma como contexto más amplio y dirección futura, pero no debe implicar que el sistema actual realice aprendizaje multiagente, adversarial o de control de red en vivo. La formulación más precisa es que la tesis implementa un benchmark de NIDS con conjunto de datos como entorno y una interfaz de acción de seguridad binaria sensible al coste, no un defensor de cyber-range completo.

3.10 Clasificación como RL y decisiones de seguridad binarias

La formulación PERMIT/BLOCK pertenece a la discusión de clasificación como RL. En el entorno actual, cada observación de flujo se presenta al agente, la acción es binaria y la recompensa se calcula a partir de la acción y la etiqueta de verdad. PERMIT corresponde a aceptar el tráfico como benigno, mientras que BLOCK corresponde a identificarlo como ataque. Un falso negativo permite un flujo de ataque; un falso positivo bloquea tráfico benigno.

Este enfoque tiene un beneficio metodológico real: hace explícitas las decisiones de seguridad sensibles al coste. Los errores de IDS no son simétricos en la práctica. Omitir un ataque y bloquear tráfico benigno tienen significados operativos diferentes, y la compensación preferida depende del entorno defendido (Lee et al. 2002; Cárdenas et al. 2006). El diseño de la recompensa puede codificar esta asimetría directamente, lo que es útil para una tesis que estudia reglas de decisión defensivas en lugar de solo la predicción de etiquetas.

La limitación es igualmente importante. Si el conjunto de datos no reacciona a la acción, entonces PERMIT/BLOCK no es una acción de control activa. Es una etiqueta de decisión offline sobre una fila de flujo registrada o extraída.

La fila siguiente no cambia porque el agente haya bloqueado o permitido la fila anterior. La formulación es, por tanto, más próxima a una clasificación con recompensa diseñada que a un firewall, un IPS, un controlador SDN o un agente de defensa cibernética autónoma. Esto no es un defecto si se declara con claridad. Solo se convierte en un defecto si el capítulo reclama en exceso el bloqueo en vivo, la preparación para el despliegue o el control secuencial completo.

La función de recompensa también necesita una interpretación cuidadosa. Una recompensa positiva para un verdadero positivo y una penalización mayor para un falso negativo puede expresar la idea de que los ataques omitidos son costosos. Una penalización por falso positivo puede expresar el coste de interrumpir el tráfico benigno o de sobrecargar a los analistas. Una recompensa de cero o pequeño valor para los verdaderos negativos puede mantener el foco en la detección de ataques. Estas son decisiones de diseño, no economía universal de seguridad. Diferentes organizaciones elegirían distintos costes dependiendo de la continuidad del negocio, el modelo de amenaza, la capacidad del analista y la tolerancia a los ataques omitidos.

Dado que la formulación está próxima a la clasificación sensible al coste, las líneas base supervisadas son obligatorias. Un Random Forest sólido, e idealmente otras líneas base tabulares, ayudan a determinar si QRDQN añade valor empírico más allá de las características canónicas, el preprocesamiento y el diseño de la recompensa. Las mismas métricas deben calcularse tanto a partir de las salidas de RL como de las supervisadas. La recompensa de entrenamiento sola no es suficiente, porque una curva de recompensa puede mejorar mientras la precisión, el recall o el comportamiento de falsos positivos sigue siendo operativamente deficiente. La tesis debe interpretar cualquier ventaja de RL como específica del benchmark y el protocolo, a menos que se disponga de evidencia más amplia.

3.11 Riesgos metodológicos en NIDS basado en ML/DL/RL

La bibliografía de NIDS advierte repetidamente contra sobreinterpretar la exactitud de benchmark. Sommer y Paxson señalan que el ML para detección de intrusiones es difícil de validar en entornos realistas: el tráfico benigno es diverso, los ataques raros, las etiquetas difíciles de obtener y las tasas base operativas difieren de las del benchmark (Sommer et al. 2010). Arp et al. identifican riesgos afines en ML para seguridad, como fuga de información, sesgo de muestreo, métricas inapropiadas y diseño experimental débil (Arp et al. 2020). La evidencia sobre generalización entre conjuntos y sobre la elección de datos de entrenamiento refuerza que un resultado dentro de un benchmark no basta para inferir el comportamiento en otro entorno (Layeghy et al. 2022; Cantone et al. 2024; Iwanowski et al. 2025; Layeghy et al. 2023).

La fuga de información abarca más que poner por error la misma fila en entrenamiento y prueba: incluye cualquier información relacionada con el objetivo que no estaría legítimamente disponible al predecir (Kaufman et al. 2011). En NIDS puede entrar por identificadores de endpoint, marcas temporales absolutas, Flow ID, puertos específicos del escenario, preprocesamiento global ajustado antes de particionar, flujos duplicados o casi duplicados, y particiones que comparten contexto de captura. Un modelo que aprende que una IP, puerto, día o Flow ID concreto corresponde a un ataque puede puntuar bien sin aprender comportamiento malicioso portable.

La fuga temporal y de escenario es especialmente importante. Los conjuntos públicos de NIDS suelen organizarse por día, archivo de captura, ventana de ataque o escenario con script. Una partición aleatoria por filas puede dejar tráfico muy relacionado a ambos lados de la frontera de entrenamiento y prueba. Aunque las filas no sean duplicados exactos, pueden compartir hosts, herramientas de ataque, regímenes de temporización o artefactos de extracción.

Esto importa en CICIDS2017 porque trabajos posteriores reportan errores o artefactos en extracción de flujos, etiquetado y uso de CSV resumidos (Engelen et al. 2021; Lanvin et al. 2023; L. Liu et al. 2022; Rosay et al. 2022; Dube 2024).

El preprocesamiento también puede introducir fugas. El escalado, la imputación, la selección de características, el recorte de valores atípicos o la reducción de dimensionalidad deben ajustarse sobre entrenamiento y aplicarse después a los datos de validación o de prueba. Si un escalador se ajusta sobre todos los datos antes de particionar, la información distribucional de prueba entra en el entrenamiento. Puede parecer menos grave que filtrar etiquetas, pero con estructura fuerte por día o escenario también puede inflar la confianza. Por ello, los objetos `StandardScaler` ajustados sobre entrenamiento en el repositorio forman parte del diseño metodológico, no solo de la codificación.

El desbalanceo de clases y el problema de la tasa base complican la interpretación. En redes operativas, los ataques suelen ser raros frente al tráfico benigno. Un detector puede lograr exactitud elevada mientras omite muchos ataques, o alto recall mientras genera demasiadas alertas falsas para investigarlas. El argumento de la tasa base de Axelsson sigue siendo relevante: incluso tasas bajas de falsos positivos pueden dominar los flujos de alertas cuando la prevalencia real de ataque es baja (Axelsson 2000). Por eso la exactitud no debe ser la métrica principal para NIDS.

El RL introduce riesgos adicionales: el entrenamiento profundo puede ser estocástico y sensible a semillas aleatorias, hiperparámetros, modelado de la recompensa, configuración del buffer de repetición, exploración y detalles del entorno. En un conjunto de datos estático usado como entorno, buenos resultados pueden reflejar las mismas ventajas de características y partición que benefician a modelos supervisados. Una afirmación de superioridad de QRDQN requiere el mismo protocolo de comparación y reconocer la varianza entre ejecuciones cuando no hay semillas repetidas. Las ganancias de una única ejecución deben tratarse como preliminares salvo que las sostengan experimentos

repetidos o artefactos de validación sólidos.

3.12 Protocolos de evaluación, métricas y reproducibilidad

La evaluación debe ajustarse a la solidez de la afirmación. Las particiones aleatorias sobre el mismo conjunto de datos son aceptables para comprobaciones internas de aprendizaje, pero son evidencia débil para el despliegue. Una escalera de validación práctica para esta tesis comienza con particiones aleatorias estratificadas, avanza hacia particiones temporales o por CSV/día, la validación leave-one-scenario o leave-one-CSV-out, las pruebas entre conjuntos de datos con un esquema de características armonizado, y finalmente el tráfico capturado de forma independiente o similar al de producción. La tesis no alcanza este último peldaño; su inferencia sobre tráfico de laboratorio, generado por el operador en un entorno doméstico cerrado y no adversarial, se sitúa por debajo. Cada paso aumenta la separación distribucional entre datos de entrenamiento y prueba, y por tanto permite afirmaciones de generalización más sólidas.

El repositorio actual refleja esta escalera en la campaña aprobada. La MAIN aleatoria verifica aprendizaje intradistribución; el control con etiquetas barajadas comprueba si el rendimiento colapsa al destruir la señal; el experimento completo por días rompe la mezcla temporal; y cuatro holdouts exactos estudian Web Attacks, Infiltration, PortScan y DDoS. No se trata de un leave-one-CSV-out exhaustivo. La inferencia offline de Fase 2 responde a la cuestión separada de si la MAIN fresca puede procesar flujos del laboratorio cerrado con el mismo contrato y preprocesamiento.

Las métricas también deben interpretarse operativamente. La exactitud es útil pero insuficiente para NIDS desequilibrados. La precisión indica cuán fiables son las alertas positivas. El recall indica qué proporción del tráfico de

ataque es detectada. F1 proporciona un equilibrio compacto, pero oculta la compensación real entre falsos positivos y falsos negativos. La tasa de falsos positivos mapea al tráfico benigno bloqueado o alertado incorrectamente. La tasa de falsos negativos mapea al tráfico malicioso omitido por el detector (Axelsson 2000; Fawcett 2006; Saito et al. 2015). Las matrices de confusión, la precisión, el recall, F1, TP, FP, FN, TN, FPR y FNR por clase deben preferirse sobre un único número resumen (Milenkoski et al. 2015).

La evaluación sensible al coste es relevante porque el punto de operación correcto depende del contexto. La guía del NIST y la bibliografía de evaluación de IDS enfatizan ambas las compensaciones entre falsos positivos y falsos negativos (Scarfone et al. 2007; Cárdenas et al. 2006). En esta tesis, el diseño de recompensa asimétrico debe tratarse como una suposición de escenario y evaluarse empíricamente, no como un modelo de coste universal para todas las redes. Si la recompensa penaliza fuertemente los falsos negativos, la política resultante puede preferir bloquear más flujos. Ese comportamiento debe juzgarse mediante precisión, recall, tasa de falsos positivos y las suposiciones de coste específicas declaradas en el experimento.

La reproducibilidad es parte de la evidencia, no un detalle administrativo. Los resultados de ML en ciberseguridad deben preservar la versión del conjunto de datos, la variante exacta del conjunto de datos, el esquema de características, los campos excluidos, el orden del preprocesamiento, la lógica de partición, el estado del escalador, las semillas aleatorias, la configuración del modelo, la configuración de la recompensa, el identificador de ejecución, las métricas y las predicciones donde sea factible. Olszewski et al. muestran que la reproducibilidad sigue siendo un problema importante en la investigación de ML en seguridad (Olszewski et al. 2023). El uso en el repositorio de configuraciones guardadas, métricas, artefactos del modelo, salidas de validación y carpetas de ejecución es, por tanto, una fortaleza metodológica: las conclusiones se limitan a la evidencia efectivamente conservada y validada.

Esta disciplina en los informes también protege contra las afirmaciones

excesivas accidentales. Un resultado vinculado a la ejecución C03 de QRDQN con partición aleatoria completa es un resultado para esa ejecución, con esa partición, configuración de recompensa, ruta de preprocesamiento y estado de artefactos. No debe convertirse silenciosamente en una afirmación sobre todas las configuraciones de QRDQN, todas las variantes de CICIDS2017 o todo el tráfico de red. Cuanto más sólida sea la afirmación, más completo debe ser el rastro de artefactos.

3.13 Eficiencia de datos y evaluación a escala de entrenamiento

La eficiencia de datos pregunta cuánto tráfico etiquetado se necesita para alcanzar un rendimiento útil, y cómo cambia la curva de aprendizaje a medida que crece el conjunto de entrenamiento. Esta pregunta es importante para NIDS porque el tráfico de ataque etiquetado es costoso, las distribuciones de tráfico cambian y la abundancia de benchmarks públicos no implica abundancia de datos de despliegue (Ring et al. 2019; Apruzzese et al. 2022). También es importante para el RL porque los métodos de RL profundo pueden ser exigentes en muestras, especialmente cuando las señales de recompensa son escasas, ruidosas, retardadas o altamente diseñadas (Sutton et al. 2018; Hessel et al. 2018).

La bibliografía existente de RL-IDS con frecuencia entrena sobre conjuntos de datos públicos completos e informa de puntuaciones finales en el benchmark, mientras que las ablaciones controladas a escala de entrenamiento son menos comunes (W. Yang et al. 2026; Kheddar et al. 2025). El trabajo de NIDS supervisado con pocos ejemplos y aprendizaje incremental por clases aborda preguntas relacionadas desde una perspectiva diferente (Di Monda et al. 2024), pero la pregunta de la tesis es más estrecha: bajo un mismo esquema de

características y protocolo de evaluación, ¿cómo escala QRDQN con las filas de entrenamiento disponibles, y cómo se compara eso con las líneas base supervisadas?

La respuesta importa porque la elección del modelo y las necesidades de datos están conectadas. Si un Random Forest alcanza un rendimiento estable con muchas menos filas que QRDQN bajo la misma partición, eso es evidencia importante incluso si QRDQN lo alcanza posteriormente. Si QRDQN es más sensible a los datos reducidos o al desbalanceo de la clase de ataque, la tesis debe reportarlo como una limitación. Si QRDQN es competitivo con presupuestos menores, la evidencia debe seguir vinculada al benchmark y al protocolo de validación, no generalizada a todos los escenarios de NIDS.

La tesis posiciona los experimentos a escala de entrenamiento como evidencia metodológica. Presupuestos como 100k, 250k, 500k, 1M y 2M muestras pueden comprobar si la formulación de RL requiere sustancialmente más datos que una línea base supervisada para alcanzar un rendimiento comparable. Tales experimentos no deben enmarcarse como prueba de aprendizaje con pocos ejemplos ni de preparación para el despliegue. Son evidencia interna sobre los requisitos de datos de esta formulación particular. La salida útil es una comparación de curvas de aprendizaje con preprocesamiento consistente, semillas aleatorias donde sea factible, métricas compartidas y rutas de artefactos explícitas.

3.14 Brecha de investigación y posicionamiento de esta tesis

La bibliografía respalda una brecha de investigación estrecha pero significativa. El NIDS y el ML basado en flujos son áreas maduras; conjuntos de datos públicos como CICIDS2017 son útiles pero frágiles; las líneas base tabulares

supervisadas son sólidas; el RL y el DRL para IDS ya existen; y la defensa cibernética autónoma es un campo más amplio y más secuencial que la implementación actual (Ring et al. 2019; H. Liu et al. 2019; W. Yang et al. 2026; Kheddar et al. 2025). La brecha no es la ausencia de RL para la detección de intrusiones.

La contribución de esta tesis es un estudio experimental reproducible de una formulación offline de decisión de seguridad binaria a nivel de flujo. El sistema mapea CICIDS2017 y entradas posteriores de flujos de laboratorio en un esquema canónico fijo de 76 características, aumenta las observaciones con una máscara de presencia/ausencia para producir entradas de 152 dimensiones, expone las filas a través de un entorno compatible con Gymnasium, y entrena un agente QRDQN para elegir entre PERMIT y BLOCK. Esto hace explícita la interfaz de decisión sensible al coste mientras preserva una obligación de comparación clara contra las líneas base supervisadas.

La tesis también está posicionada metodológicamente. Trata a CICIDS2017 como el benchmark interno principal, no como prueba de preparación para la producción. Separa los resultados con partición aleatoria de la validación más estricta. Utiliza preprocesamiento con control de fugas y comprobaciones de validación para reducir el aprendizaje evidente de artefactos. Trata el tráfico de laboratorio capturado externamente como el siguiente paso evidencial preferido, reconociendo al mismo tiempo que la inferencia actual de Fase 2 es offline y no realiza bloqueo activo.

La brecha final es, por tanto, una brecha de integración: un benchmark QRDQN reproducible, con control de fugas y sensible al coste para el NIDS a nivel de flujo, evaluado contra líneas base supervisadas e interpretado a través de una escalera de validación. La tesis puede contribuir haciendo explícito el contrato de características, guardando artefactos de ejecución, documentando las suposiciones de la recompensa, comparando contra Random Forest bajo particiones emparejadas, y mostrando cómo el rendimiento interno de CICIDS2017 cambia bajo una validación más estricta y restricciones de escala

de entrenamiento.

La afirmación final defendible no es que QRDQN sea universalmente mejor que los modelos de NIDS supervisados, ni que el proyecto implemente defensa cibernética autónoma. La afirmación defendible es que la tesis evalúa un defensor a nivel de flujo basado en QRDQN, sensible al coste, bajo un benchmark reproducible y un pipeline de validación, y utiliza líneas base supervisadas, comprobaciones con control de fugas, análisis de escala de entrenamiento e inferencia sobre tráfico de laboratorio para determinar hasta qué punto puede confiarse en esa formulación. Si experimentos posteriores muestran un rendimiento débil entre archivos o con tráfico de laboratorio, eso no invalida la tesis. Agudiza su contribución al demostrar los límites del RL entrenado sobre benchmark bajo un cambio de distribución más realista.

Diseño del sistema

Este capítulo concreta el diseño del prototipo: contrato de datos, entorno de decisión, agente QRDQN, recompensa y persistencia de artefactos. Describe un flujo de procesamiento experimental fuera de línea, no un mecanismo de bloqueo en producción.

4.1 Visión general metodológica

Este capítulo describe el diseño metodológico empleado para construir y evaluar el defensor de ciberseguridad basado en aprendizaje por refuerzo (RL, del inglés *Reinforcement Learning*) propuesto. El objetivo no es presentar un sistema de prevención de intrusiones (IPS, del inglés *Intrusion Prevention System*) listo para producción, sino definir un marco experimental controlado, reproducible y fuera de línea (*offline*) para decisiones de seguridad binarias sobre registros de flujos de red. Cada flujo se trata como un único punto de decisión: el defensor observa una representación numérica del flujo y decide si permitirlo (PERMIT) como tráfico benigno o bloquearlo (BLOCK) como tráfico malicioso.

La metodología es deliberadamente conservadora. Los benchmarks públi-

cos de detección de intrusiones pueden producir resultados inflados cuando el preprocesamiento, la construcción de las particiones, los controles de fuga de información y las comparaciones con líneas base son débiles (Arp et al. 2020; Engelen et al. 2021; Lanvin et al. 2023). Por ello, el pipeline se ha diseñado en torno a contratos de datos explícitos, artefactos de ejecución reproducibles, normalización ajustada únicamente sobre datos de entrenamiento, y una validación progresivamente más estricta. La misma representación canónica se utiliza tanto para el benchmark interno de CICIDS2017 como para la inferencia offline sobre tráfico de laboratorio en la Fase 2, de modo que la pregunta experimental no queda ligada a una convención de nomenclatura de CSV concreta ni a un extractor de características particular.

A grandes rasgos, la metodología comprende seis etapas encadenadas:

1. ingerir los datos CSV de CICIDS2017 a nivel de flujo y mapear las etiquetas a una tarea binaria de defensa;
2. eliminar los identificadores susceptibles de provocar fuga de información y normalizar los valores numéricos malformados;
3. mapear las columnas específicas de cada extractor a un esquema canónico de características fijo de 76 características;
4. añadir una máscara de presencia/ausencia de 76 dimensiones, produciendo observaciones de 152 dimensiones;
5. conservar la representación canónica sin escalar en una caché validada, construir la partición y ajustar el escalador solo con entrenamiento;
6. entrenar y evaluar un defensor QRDQN compatible con Gymnasium bajo protocolos controlados, compararlo con una línea base supervisada y aplicar la inferencia offline de Fase 2.

Este diseño mantiene la tesis dentro de un marco experimental offline. Las salidas PERMIT y BLOCK son predicciones del modelo sobre registros de flujo

almacenados. No son órdenes de cortafuegos y no modifican ninguna ruta de red activa.

La Figura 4.1 resume esta arquitectura en dos recorridos separados: entrenamiento y evaluación sobre CICIDS2017, e inferencia offline posterior sobre tráfico de laboratorio. La separación visual refuerza una idea metodológica central: la Fase 2 reutiliza el esquema canónico, el escalador y el modelo entrenado, pero no reentrena ni reajusta el pipeline sobre la captura de laboratorio.

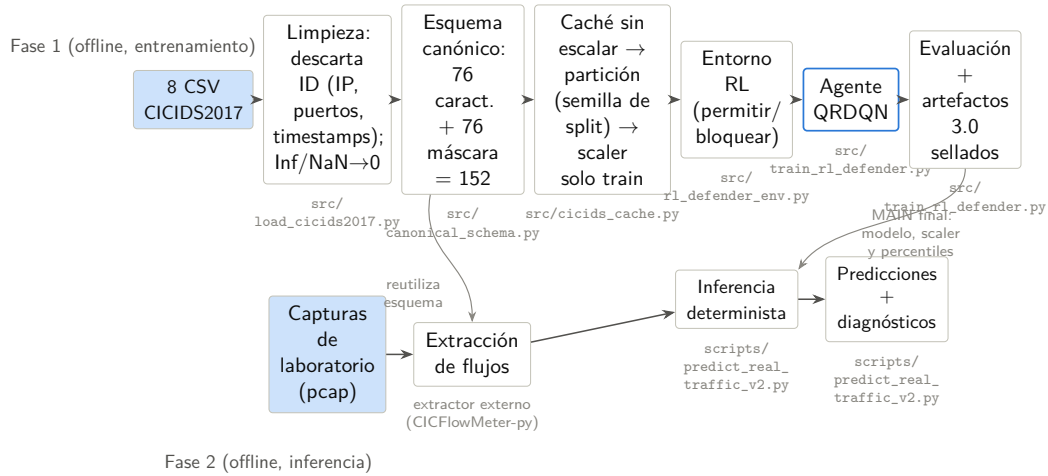


Figura 4.1: Pipeline general del sistema: entrenamiento offline sobre CICIDS2017 e inferencia offline en la Fase 2. Fuente: elaboración propia a partir de `src/load_cicids2017.py`, `src/canonical_schema.py`, `src/train_rl_defender.py` y `scripts/predict_real_traffic_v2.py`.

4.2 Fuente de datos y representación de la entrada

La tesis utiliza una representación tabular a nivel de flujo. Esta representación es un compromiso deliberado entre relevancia operativa, manejabilidad y privacidad. Captura el comportamiento estadístico a lo largo de secuencias de paquetes evitando la inspección de la carga útil.

4.2.1 CSVs de flujos de CICIDS2017

La fuente de datos principal es la publicación de CSVs de flujos de CICIDS2017. Cada fila de un CSV corresponde a un flujo de red bidireccional resumido mediante estadísticas numéricas como duración, conteos de paquetes, totales de bytes, tiempos de llegada entre paquetes, estadísticas de longitud de paquete y conteos de flags TCP. Las representaciones basadas en flujos se utilizan ampliamente en la investigación sobre NIDS porque comprimen el comportamiento de la red en filas numéricas de anchura fija que pueden procesarse mediante métodos estándar de aprendizaje automático (Sperotto et al. 2010; Umer et al. 2017).

El pipeline de carga local espera los ficheros CSV oficiales de CICIDS2017 y admite presupuestos de filas tanto completos como limitados. Los presupuestos limitados permiten pruebas rápidas de humo e iteración ágil; los presupuestos completos son los utilizados en las ejecuciones finales del benchmark. El cargador lee los CSVs grandes en fragmentos, estandariza los nombres de columna eliminando espacios en blanco inconsistentes, identifica la columna de etiqueta de forma robusta y aplica después la limpieza de características antes de cualquier escalado dependiente de la partición.

4.2.2 Mapeo de etiquetas binarias

CICIDS2017 incluye varias etiquetas de ataque. La tesis las colapsa en una decisión binaria del defensor:

- etiqueta 0: **BENIGN**, mapeada a la acción deseada **PERMIT**;
- etiqueta 1: cualquier clase de ataque, mapeada a la acción deseada **BLOCK**.

Este mapeo binario se corresponde con el modelo de decisión de primera línea estudiado aquí. Además, hace que el espacio de acciones de RL sea sencillo y auditable. La limitación es que la información sobre la familia de ataque

no es el objetivo de aprendizaje principal en la ruta de RL actual. Cualquier análisis de errores por familia debe por tanto estar respaldado por etiquetas de familia preservadas o artefactos independientes; no puede inferirse a partir de las etiquetas binarias por sí solas.

4.2.3 Representación tabular a nivel de flujo

El sustrato de aprendizaje es una tabla a nivel de flujo en lugar de paquetes en bruto o bytes de carga útil. Esto reduce la dimensionalidad y hace factible el entrenamiento offline a gran escala. También evita algunas complicaciones legales y técnicas de la inspección profunda de paquetes, especialmente cuando las cargas útiles están cifradas (Sperotto et al. 2010; Umer et al. 2017). Sin embargo, la abstracción de flujo pierde el contenido, el orden preciso de los paquetes y parte del contexto de largo alcance entre múltiples flujos. La metodología estudia por tanto un defensor basado en flujos, no un sistema completo de análisis forense de red.

4.3 Limpieza de datos y preprocesamiento

El preprocesamiento se trata como parte del método experimental, no como un detalle de implementación. En los benchmarks de NIDS, pequeñas decisiones de preprocesamiento pueden cambiar el significado de la tarea al filtrar etiquetas, suprimir flujos malformados o hacer que los datos de prueba influyan en el entrenamiento.

4.3.1 Normalización de columnas y coerción numérica

Las columnas CSV en bruto pueden contener espacios iniciales, capitalización inconsistente y tipos de datos mixtos. El cargador primero normaliza los

nombres de columna y después convierte las columnas de características a valores numéricos. La matriz de características final se representa como `float32`, mientras que las etiquetas se representan como `int64`. Este contrato de tipos mantiene estable la interfaz del entorno, el escalador y el modelo en las etapas posteriores.

No se permite que las filas introduzcan cadenas de texto arbitrarias en el modelo. Las columnas de características no numéricas que sobreviven a la eliminación de identificadores quedan excluidas. Esto evita la introducción accidental de cadenas categóricas cuya codificación sería no controlada o específica de un conjunto de datos.

4.3.2 Valores inválidos, valores ausentes e imputación

La extracción de flujos de red puede producir valores numéricos inválidos. Las características de tasa, por ejemplo, pueden volverse infinitas o indefinidas cuando se calculan sobre una duración nula o casi nula. El pipeline sustituye los infinitos y los valores numéricos ausentes por cero antes del mapeo canónico. Esta elección es pragmática: muchos contadores de flujo, tasas y estadísticas agregadas pueden utilizar de forma segura el cero como marcador de posición neutro cuando se acompaña de una señal explícita de presencia/ausencia.

La imputación por cero se acompaña de una máscara de disponibilidad de columnas. En la implementación actual, la limpieza de valores no finitos ocurre antes de construir el esquema canónico: un valor inválido de una columna existente se convierte en cero, pero no cambia su indicador de máscara. Por tanto, la máscara distingue una columna fuente ausente de una columna disponible; no distingue, fila a fila, un cero medido de un valor no finito saneado. Esta limitación se conserva de forma explícita para no atribuir a la representación una semántica que el código no implementa.

4.3.3 Exclusión de campos susceptibles de fuga

El pipeline de preprocesamiento aplica una política anti-fuga fundamentada en los riesgos de evaluación conocidos en seguridad con ML (Arp et al. 2020; Kaufman et al. 2011). Se excluyen las direcciones IP de origen y destino, los Flow IDs, las marcas temporales absolutas y los identificadores de endpoints. Los campos de puerto directo también se excluyen porque pueden actuar como indicadores del escenario en conjuntos de datos controlados por scripts. Por ejemplo, un día de ataque puede concentrar tráfico malicioso en servicios concretos, lo que permitiría al modelo memorizar un script de captura en lugar de aprender estadísticas de comportamiento de flujo.

Esta exclusión se aplica antes del mapeo canónico. El modelo recibe por tanto únicamente características estadísticas de flujo y los valores de máscara correspondientes. Si en el futuro se introducen nuevos conjuntos de datos o extractores, sus adaptadores deben preservar la misma política: sin identificadores, sin campos de tiempo absoluto y sin puertos utilizados directamente como atajos de etiqueta.

4.4 Esquema canónico de características

El esquema canónico es el contrato de datos central de la tesis. Separa la interfaz del modelo de los nombres de columna, el orden y la disponibilidad de características de un extractor concreto.

4.4.1 Características canónicas de flujo

El esquema define 76 características numéricas de flujo ordenadas. Incluyen duración, conteos de paquetes hacia adelante y hacia atrás, totales de bytes, estadísticas de longitud de paquete, estadísticas de tiempo entre llegadas de

flujo y por dirección, tasas de paquetes y bytes, conteos de flags TCP, longitudes de cabecera, tamaños de ventana, estadísticas de subflujos y características de temporización activa/inactiva. Estas características se han elegido porque son estadísticas de flujo y no identificadores, y porque son ampliamente compatibles con la extracción al estilo CICFlowMeter.

Para CICIDS2017, un adaptador mapea las columnas originales del CSV a estos nombres canónicos. Para la Fase 2, el script de inferencia robusta mapea las columnas del extractor de flujos de laboratorio a los mismos nombres canónicos. El modelo se entrena y evalúa por tanto frente a un contrato de características fijo. Este diseño sigue la motivación metodológica más amplia de la estandarización de características en NIDS: la comparación entre dominios resulta más auditable cuando las definiciones de características son explícitas y estables (Sarhan et al. 2021).

La Tabla 4.1 enumera los grupos lógicos dentro del esquema de 76 características y el número de características aportadas por cada grupo. La agrupación es informativa; el modelo recibe los 76 valores como un vector plano sin señales de agrupación estructural.

La enumeración ordenada de las 76 características, junto con las columnas fuente de CICIDS2017 y de Flowmeter-py que alimentan cada posición, se presenta en el Anexo C. Ese anexo también explicita la correspondencia entre las posiciones 1–76 del vector de valores y las posiciones 77–152 de la máscara.

4.4.2 Máscara de presencia/ausencia

Diferentes extractores pueden no proporcionar todas las características canónicas, e incluso una columna fuente presente puede contener valores inválidos. El pipeline produce por tanto una máscara de presencia/ausencia de 76 dimensiones. Para cada característica canónica x_i , el valor de máscara correspondiente m_i es:

- $m_i = 1$ cuando existe una columna fuente mapeada para la característica;

Grupo de características	Cantidad	Estadísticas representativas
Duración del flujo	1	Duración total del flujo bidireccional
Estadísticas de paquetes hacia adelante	12	Conteo de paquetes, bytes totales, media/desv. típica/mínimo/máximo de longitudes de paquete
Estadísticas de paquetes hacia atrás	12	Las mismas estadísticas para la dirección inversa
Agregados bidireccionales	8	Conteos combinados, bytes totales, promedios de tasa de ráfaga
Estadísticas de tiempo entre llegadas	10	Media, desv. típica, mínimo y máximo del tiempo entre llegadas del flujo, hacia adelante y hacia atrás
Conteos de flags TCP	8	Conteos de FIN, SYN, RST, PSH, ACK, URG, CWE, ECE
Agregados de longitud de cabecera	4	Totales de bytes de cabecera hacia adelante y hacia atrás
Estadísticas de tamaño de ventana	3	Tamaño de ventana inicial y medio por dirección
Características de tasa de flujo	6	Bytes/s, paquetes/s por dirección y combinados
Estadísticas de subflujos	6	Promedios de paquetes y bytes de subflujo por dirección
Temporización activa/inactiva	6	Media, desv. típica y máximo de las duraciones de los periodos activos e inactivos

Tabla 4.1: Agrupación lógica de las 76 características canónicas de flujo. Todos los grupos se fusionan en un único vector plano antes de la entrada al modelo. Fuente: elaboración propia.

- $m_i = 0$ cuando esa columna fuente no está disponible en el extractor.

La máscara desempeña dos funciones metodológicas. En primer lugar, hace visible la compatibilidad estructural entre el extractor y el contrato canónico. En segundo lugar, facilita los diagnósticos de la Fase 2: un extractor de laboratorio que carece de características disponibles en CICIDS2017 produce un patrón de máscara distinto en lugar de ocultar la ausencia dentro de un vector numérico denso.

Conviene precisar la semántica exacta de la máscara en la implementación actual. La máscara se calcula *después* de la limpieza que sustituye los valores no finitos por cero (`src/load_cicids2017.py`: $\pm\infty \rightarrow \text{NaN} \rightarrow 0$), de modo que $m_i = 1$ siempre que la característica canónica tiene una columna fuente mapeada. Como el mapeo $\text{CICIDS2017} \rightarrow \text{canónico}$ cubre las 76 características, sobre los datos nativos de CICIDS2017 la máscara es **constante e igual a 1** en todas las filas: codifica la *presencia de la columna fuente*, no la ausencia de valores fila a fila. La máscara solo se vuelve informativa en la inferencia entre dominios o en la Fase 2 de laboratorio, donde algunas columnas fuente no están disponibles y su valor de máscara permanece en 0; un mapeo histórico como NSL-KDD, por ejemplo, solo cubre 3 de las 76 características canónicas. Así, las 152 dimensiones de la observación se componen de 76 valores de características más 76 indicadores de presencia.

4.4.3 Vector de observación final

La observación final es:

$$o = [x_1, x_2, \dots, x_{76}, m_1, m_2, \dots, m_{76}]$$

Esto produce un vector `float32` de 152 dimensiones. La primera mitad contiene los valores de las características, y la segunda mitad contiene la máscara. El tamaño de la observación es por tanto invariante entre CICIDS2017, las

particiones de validación y la inferencia en la Fase 2. Esta invariancia es importante porque la red de política del QRDQN espera una dimensión de entrada fija.

El Algoritmo 1 resume de forma operativa el proceso descrito en los párrafos anteriores: para cada fila del DataFrame de entrada construye, característica a característica, el valor x_i y el indicador de presencia m_i a partir del mapeo columna-fuente $\rightarrow$ nombre canónico, y concatena ambos vectores en la observación final de 152 dimensiones.

La Figura 4.2 muestra la misma construcción desde el punto de vista de la entrada al modelo: columnas del extractor, mapeo al contrato canónico, 76 valores de características, 76 indicadores de presencia y vector final de 152 dimensiones.

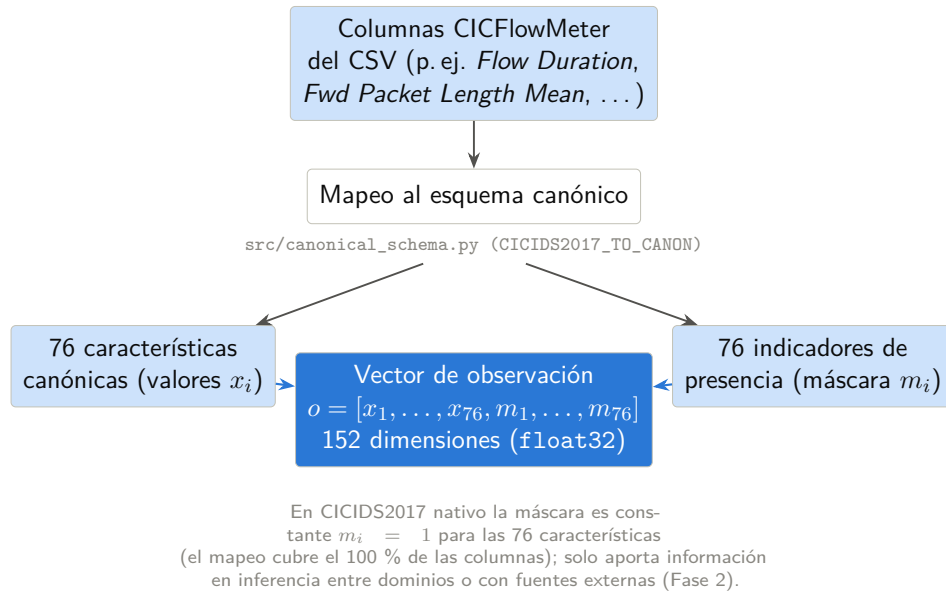


Figura 4.2: Construcción del vector de observación canónico de 152 dimensiones. Fuente: elaboración propia a partir de `src/canonical_schema.py`.

Algoritmo 1 Mapeo al esquema canónico y construcción de la observación.
Fuente: elaboración propia a partir de `src/canonical_schema.py`.

Require: DataFrame D con las columnas producidas por el extractor; mapeo μ : columna_fuente $\rightarrow$ nombre_canónico

Ensure: Matriz de observaciones O , con una fila $o \in \mathbb{R}^{152}$ (`float32`) por cada fila de D

```

1: for cada fila  $d$  de  $D$  do
2:    $x \leftarrow$  vector nulo de dimensión 76
3:    $m \leftarrow$  vector nulo de dimensión 76
4:   for  $i = 1$  hasta 76 do
5:      $c_i \leftarrow$  nombre canónico  $i$ -ésimo de FEATURES_CANON
6:     if existe columna_fuente tal que  $\mu(\text{columna\_fuente}) = c_i$  y columna_fuente  $\in$  columnas( $D$ ) then
7:        $v \leftarrow$  CoerciónNumérica( $d[\text{columna\_fuente}]$ )  $\triangleright$  no numérico  $\rightarrow$  NaN
8:       if  $v$  no es finito (NaN o  $\pm\infty$ ) then
9:          $v \leftarrow 0$ 
10:      end if
11:       $x_i \leftarrow v$ 
12:       $m_i \leftarrow 1$   $\triangleright$  columna fuente mapeada y presente
13:    else
14:       $x_i \leftarrow 0$   $\triangleright$  imputación por ausencia de columna fuente
15:       $m_i \leftarrow 0$ 
16:    end if
17:  end for
18:   $o \leftarrow \text{concat}(x_1, \dots, x_{76}, m_1, \dots, m_{76})$   $\triangleright$  vector float32 de 152 dimensiones
19:  añadir  $o$  a  $O$ 
20: end for
21: return  $O$ 

```

4.5 Formulación del aprendizaje por refuerzo

El defensor se implementa como un entorno compatible con Gymnasium (Farama Foundation 2023). Esto permite que el código estándar de RL basado en valores interactúe con el conjunto de datos de flujos a través de la interfaz habitual `reset()` y `step()`.

4.5.1 Espacio de observación

El espacio de observación es un espacio vectorial continuo de dimensión 152. Cada observación representa un vector de flujo canónico escalado junto con su máscara de presencia/ausencia. Durante el entrenamiento, las observaciones se extraen de la partición de entrenamiento. Durante la evaluación determinista, el entorno itera a través de la partición de prueba sin barajado, lo que permite comparar las predicciones directamente con las etiquetas verdaderas.

4.5.2 Espacio de acciones

El espacio de acciones es discreto con dos acciones:

- **Acción 0 (PERMIT):** clasificar el flujo como benigno y permitirlo en el modelo de decisión experimental;
- **Acción 1 (BLOCK):** clasificar el flujo como malicioso y bloquearlo o generar una alerta en el modelo de decisión experimental.

El valor de la acción está alineado intencionalmente con el valor de la etiqueta binaria: benigno es 0, ataque es 1. Esto simplifica la evaluación directa porque una acción predicha puede compararse con la etiqueta objetivo sin necesidad de una capa de decodificación adicional.

4.5.3 Función de recompensa

La función de recompensa codifica costes de seguridad asimétricos (Lee et al. 2002; Cárdenas et al. 2006). Un falso negativo significa que se permite un ataque. Un falso positivo significa que se bloquea tráfico benigno. En muchos escenarios defensivos, los ataques no detectados son más graves que los bloqueos innecesarios, aunque la relación exacta depende del contexto operativo.

La implementación actual utiliza la siguiente configuración de recompensa por defecto:

$$TP = 1.5, \quad FP = -2.0, \quad FN = -5.0, \quad TN/\text{omisión} = 0.0.$$

La Tabla 4.2 organiza estos valores como una matriz de decisión para los cuatro resultados posibles.

Decisión del agente	Etiqueta real: Ataque	Etiqueta real: Benigno
BLOCK (Acción 1)	+1.5 (Verdadero Positivo)	-2.0 (Falso Positivo)
PERMIT (Acción 0)	-5.0 (Falso Negativo)	0.0 (Verdadero Negativo)

Tabla 4.2: Matriz de recompensa por defecto para el entorno del defensor binario. Los valores son supuestos experimentales registrados con cada ejecución. Fuente: elaboración propia a partir de `src/rl_defender_env.py`.

La Figura 4.3 representa el bucle agente-entorno usado en esta formulación. Aunque se utiliza una interfaz de RL estándar, la recompensa depende únicamente de la acción actual y de la etiqueta real del flujo evaluado; con $\gamma = 0$, el objetivo aprendido se reduce a la recompensa inmediata.

La penalización por falso negativo domina la señal de recompensa. Un agente que emite PERMIT en cada observación acumula un retorno muy negativo en tráfico con alta proporción de ataques, mientras que un agente que emite BLOCK en cada observación acumula penalizaciones por falsos positivos sin obtener crédito por detección de ataques. La estructura de recompensa está por tanto diseñada para alejar al agente de políticas constantes degeneradas y conducirlo

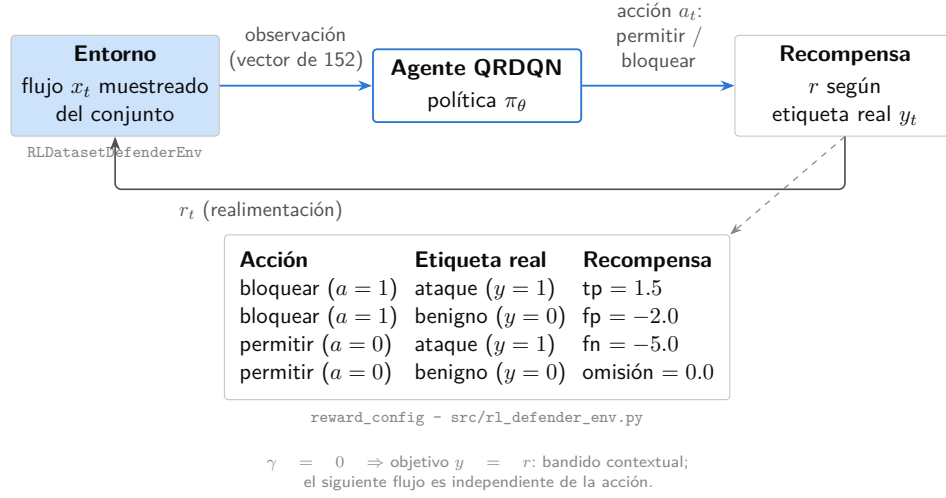


Figura 4.3: Interacción agente-entorno y matriz de recompensa de la decisión PERMIT/BLOCK. Fuente: elaboración propia a partir de src/rl_defender_env.py y del perfil main-v1.

hacia un comportamiento discriminativo. La recompensa cero para el verdadero negativo es deliberada: recompensar explícitamente cada paso benigno correcto sesgaría al agente hacia maximizar la tasa de permiso en lugar de la calidad de detección. Estos valores son supuestos experimentales, no costes empresariales medidos. Se registran con cada ejecución para que las ejecuciones históricas puedan distinguirse de los valores por defecto actuales.

Los valores de la Tabla 4.2 admiten además una lectura como compromiso multiobjetivo implícito entre dos objetivos operativos, seguridad (minimizar ataques admitidos) e impacto (minimizar el bloqueo de tráfico legítimo), que se resume en la Tabla 4.3.

4.5.4 Estructura del episodio

Un episodio es una secuencia de decisiones sobre flujos. Al reiniciar, el entorno inicializa un índice sobre la partición seleccionada. Cuando el barajado está activado, el orden de entrenamiento se aleatoriza. En cada paso, el agente recibe una observación, selecciona PERMIT o BLOCK, recibe una recompensa calculada a partir de la etiqueta real y la acción, y avanza al flujo siguiente. Un

Resultado	Recompensa	Objetivo operativo	Lectura
FN (ataque permitido)	-5.0	Seguridad	Coste dominante: un ataque queda admitido
FP (benigno bloqueado)	-2.0	Impacto	Penaliza el bloqueo de tráfico legítimo
TP (ataque bloqueado)	+1.5	Seguridad	Crédito positivo por detección correcta de un ataque
TN/omisión (benigno permitido)	0.0	Neutralidad deliberada	Evita sesgar al agente hacia maximizar la tasa de permiso

Tabla 4.3: Lectura multiobjetivo de la función de recompensa del entorno del defensor binario: cada resultado se asocia al objetivo operativo de seguridad o de impacto al que responde. Fuente: elaboración propia a partir de `src/rl_defender_env.py`.

episodio termina cuando se agota el conjunto de datos o se alcanza un número máximo de pasos configurado.

Este diseño proporciona a los algoritmos estándar de RL un flujo de transiciones manteniendo el entrenamiento offline y seguro. Ninguna acción afecta al conjunto de datos fuente, al flujo siguiente ni al entorno de red.

4.5.5 Estado del entorno y protocolo de transición

En cada llamada a `reset()`, el entorno selecciona la partición asignada al modo actual (entrenamiento o evaluación), baraja opcionalmente el índice si el modo de barajado está activo, reinicia el contador de pasos interno y devuelve la primera observación junto con un diccionario de metadatos. En cada llamada a `step()`, el entorno registra la acción del agente, recupera la etiqueta de verdad del índice de flujo actual, calcula la recompensa escalar a partir de la matriz de recompensa, avanza el índice y devuelve la siguiente observación, la recompensa, un indicador de terminación, un indicador de truncación y el diccionario de metadatos.

La condición de terminación es el agotamiento del conjunto de datos o un número máximo de pasos configurable. En el modo de entrenamiento, el orden barajado garantiza que las transiciones del minibatch muestreen una distribución amplia de tipos de flujo durante un único episodio. En el modo de evaluación, el entorno es determinista: la misma partición recorrida en el mismo orden produce la misma secuencia de observaciones, recompensas y etiquetas. Este determinismo es esencial para la evaluación reproducible porque implica que las diferencias en métricas entre dos modelos guardados reflejan únicamente el comportamiento del modelo y no el ruido de muestreo en el bucle de evaluación.

La separación entre los modos de entrenamiento y evaluación se aplica a nivel del entorno y no se deja al llamante. Pasar el indicador de modo correcto en el momento de la construcción establece tanto el conjunto de índices como el comportamiento de barajado, lo que evita un error frecuente en el que la evaluación hereda accidentalmente el orden barajado de un entorno de entrenamiento.

El Algoritmo 2 muestra el pseudocódigo del paso de decisión `step(a)` tal y como está implementado, que aplica la matriz de recompensas de la Tabla 4.2 a la etiqueta real del índice actual, hace avanzar el puntero de muestras y evalúa de forma independiente las condiciones de terminación (agotamiento del conjunto de índices) y truncación (límite de pasos por episodio).

4.5.6 Limitaciones de la formulación conjunto-de-datos-como-entorno

Dado que el entorno es un conjunto de datos estático, las acciones del agente no afectan a los estados futuros. Esto no es un problema completo de defensa cibernética autónoma en el que bloquear un flujo podría cambiar el comportamiento posterior del atacante, el estado del host, el enrutamiento o el contexto de alertas. La formulación se asemeja más a un problema de decisión

Algoritmo 2 Paso de decisión del entorno (`env.step`). Fuente: elaboración propia a partir de `src/rl_defender_env.py`.

Require: Índice actual i , contador de pasos t y vector de índices $\mathcal{I}$, inicializados en RESET (con $\mathcal{I}$ barajado si el indicador *shuffle* está activo, según el modo fijado en la construcción del entorno)

```

1: function STEP( $a$ )
2:   if  $a \notin \{0, 1\}$  then
3:     return error (acción inválida)
4:   end if
5:    $idx \leftarrow \mathcal{I}[i]$ 
6:    $y \leftarrow Y[idx]$  ▷ etiqueta real de la muestra actual
7:   if  $y = \text{ataque}$  then
8:     if  $a = 1$  (BLOCK) then
9:        $r \leftarrow tp = 1.5$  ▷ verdadero positivo
10:    else
11:       $r \leftarrow fn = -5.0$  ▷ falso negativo
12:    end if
13:  else ▷  $y = \text{benigno}$ 
14:    if  $a = 0$  (PERMIT) then
15:       $r \leftarrow \text{omission} = 0.0$  ▷ verdadero negativo
16:    else
17:       $r \leftarrow fp = -2.0$  ▷ falso positivo
18:    end if
19:  end if
20:   $i \leftarrow i + 1$ ;  $t \leftarrow t + 1$  ▷ avance del índice y del contador de pasos
21:   $terminated \leftarrow (i \geq N)$  ▷  $N$ : tamaño del conjunto de muestras
22:   $truncated \leftarrow (t \geq T_{\text{máx}})$  ▷  $T_{\text{máx}}$ : máximo de pasos por episodio
23:  if  $terminated \vee truncated$  then
24:     $s' \leftarrow X[idx]$  ▷ se repite la observación de la muestra actual
25:  else
26:     $s' \leftarrow X[\mathcal{I}[i]]$  ▷ observación de la siguiente muestra
27:  end if
28:   $info \leftarrow \{sample\_index : idx, true\_label : y\}$ 
29:  return ( $s', r, terminated, truncated, info$ )
30: end function

```

contextual sensible al coste que a un proceso de decisión de Markov rico (Sutton et al. 2018; Levine et al. 2020).

Esta limitación es explícita en la metodología. El RL se utiliza aquí para estudiar el aprendizaje de decisiones basadas en valor y sensibles al coste sobre registros de flujo, y para establecer un pipeline offline seguro. Las afirmaciones sobre respuesta adversarial en múltiples pasos, adaptación en línea o defensa cibernética activa quedan fuera del alcance implementado.

4.6 Agente QRDQN

El algoritmo de RL principal es Quantile Regression Deep Q-Network (QRDQN) (Dabney et al. 2018). QRDQN pertenece a la familia del RL distribucional, que modela una distribución sobre los retornos en lugar de únicamente un retorno esperado (Bellemare et al. 2017).

4.6.1 Papel algorítmico

El DQN estándar aproxima los valores-acción con una red neuronal y selecciona acciones según el retorno esperado estimado (Mnih et al. 2015). QRDQN extiende esta idea aprendiendo los cuantiles de la distribución del retorno. Esto es relevante para la tesis porque la estructura de recompensa es asimétrica: los falsos negativos conllevan una penalización mucho mayor que la recompensa que generan los permisos benignos correctos. Una visión distribucional puede representar más información sobre la variabilidad del retorno que una única estimación de la media.

La metodología no asume que QRDQN sea inherentemente superior a los clasificadores supervisados para datos de flujo tabulares. En cambio, evalúa QRDQN como el representante principal de RL bajo el mismo preprocesamiento, las mismas particiones y las mismas métricas utilizadas por las líneas base.

La arquitectura de red del perfil **main-v1** se resume en la Figura 4.4: una entrada de 152 dimensiones, tres capas densas ocultas y una salida distribucional de 200 cuantiles por cada una de las dos acciones.

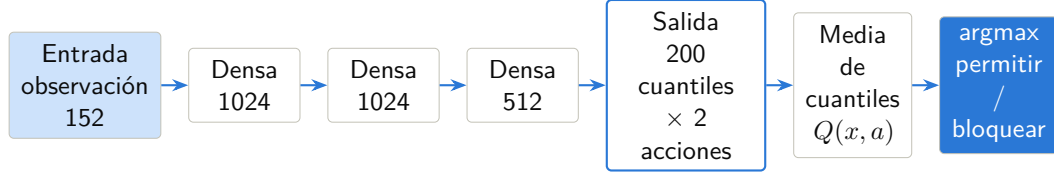


Figura 4.4: Arquitectura QRDQN del perfil congelado **main-v1**: observación canónica, capas densas, cuantiles por acción y decisión final por media de cuantiles. Fuente: elaboración propia a partir del perfil actual y del artefacto histórico coherente con esa arquitectura.

4.6.2 Función de pérdida de regresión cuantílica

QRDQN representa la distribución del retorno de acción mediante N estimaciones de cuantiles aprendidas $\{\theta_i(s, a)\}_{i=1}^N$ en puntos medios de cuantil fijos $\hat{\tau}_i = \frac{2i-1}{2N}$ (Dabney et al. 2018). Durante la selección de acción, estos cuantiles se promedian para producir una estimación del retorno esperado, recuperando la regla codiciosa estándar del DQN bajo expectativa.

Para una única transición (s, a, r, s') , sea $a^* = \arg \max_a \sum_j \theta_j^-(s', a)$ la acción codiciosa bajo la red objetivo, y sea

$$\delta_{ij} = r + \gamma \theta_j^-(s', a^*) - \theta_i(s, a)$$

el error de diferencia temporal entre el cuantil fuente i y el cuantil objetivo j calculado mediante bootstrapping. QRDQN minimiza la función de pérdida de regresión cuantílica de Huber sumada sobre todos los N^2 pares de cuantiles:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \left| \hat{\tau}_i - \mathbf{1}[\delta_{ij} < 0] \right| \cdot L_{\kappa}(\delta_{ij}),$$

donde L_κ es la pérdida de Huber elemento a elemento con umbral κ :

$$L_\kappa(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{si } |\delta| \leq \kappa, \\ \kappa\left(|\delta| - \frac{1}{2}\kappa\right) & \text{en caso contrario.} \end{cases}$$

A diferencia de C51, que representa los retornos sobre un soporte categórico fijo, QRDQN no impone ninguna restricción sobre el soporte de la distribución del retorno y se adapta automáticamente a la escala de las recompensas (Bellemare et al. 2017). En el entorno de recompensa asimétrica de esta tesis, la distribución del retorno puede desarrollar colas negativas pronunciadas cuando los eventos de falso negativo son frecuentes, lo que hace que una visión distribucional sea más informativa que una estimación escalar de la media para comprender el comportamiento del agente en torno a la frontera de decisión PERMIT/BLOCK.

4.6.3 Factor de descuento $\gamma = 0$: formulación de un solo paso

En todas las ejecuciones de esta tesis, incluida la ejecución oficial, el factor de descuento se fija en $\gamma = 0$ (valor registrado en el artefacto de configuración de la ejecución). La elección es deliberada y coherente con la formulación conjunto-de-datos-como-entorno: dado que el entorno es un conjunto de datos estático y las acciones del agente no afectan a los estados futuros, no existe dependencia temporal entre flujos que un factor de descuento positivo pudiera explotar.

Con $\gamma = 0$, el error de diferencia temporal definido más arriba se reduce a

$$\delta_{ij} = r - \theta_i(s, a),$$

es decir, el término de bootstrapping $\gamma \theta_j^-(s', a^*)$ se anula y la red objetivo deja de contribuir. Cada cuantil aprendido $\theta_i(s, a)$ regresa por tanto hacia la

recompensa inmediata de la decisión, y QRDQN actúa como un estimador distribucional de un solo paso: formalmente, un *bandit contextual sensible al coste*, y no un proceso de decisión de Markov secuencial.

Esta reducción no vacía de contenido la elección de QRDQN. La cabeza distribucional sigue modelando la distribución completa de la recompensa asimétrica en torno a la frontera PERMIT/BLOCK —y no únicamente su media—, lo que aporta información sobre la incertidumbre del valor bajo coste asimétrico. La diferencia frente a un clasificador supervisado no reside en la presencia de dinámica secuencial, inexistente aquí por diseño, sino en que el aprendizaje se realiza mediante valores sobre una matriz de coste explícita (con la penalización de falso negativo muy superior a la de falso positivo) en lugar de mediante entropía cruzada sobre etiquetas. La tesis no reivindica autonomía secuencial ni respuesta adversarial en múltiples pasos: tal y como se recoge en las limitaciones de la formulación, esas capacidades quedan fuera del alcance implementado.

4.6.4 Estrategia de exploración

Durante el entrenamiento, el agente emplea una política ε -greedy. Con probabilidad ε selecciona una acción uniformemente aleatoria de {PERMIT, BLOCK}; con probabilidad $1 - \varepsilon$ selecciona la acción cuya estimación media de cuantiles es más alta (Mnih et al. 2015). ε decae linealmente desde un valor inicial cercano a uno hasta un valor final pequeño a lo largo de una fracción fija del total de pasos de entrenamiento, y luego permanece constante. En el entorno de acción binaria, el presupuesto de exploración es relativamente barato porque solo existen dos alternativas. La fracción de exploración y el valor final de ε se almacenan en el artefacto de configuración de la ejecución para que las ejecuciones con diferentes programas no se mezclen o comparen en silencio sin tener en cuenta sus diferencias de exploración.

4.6.5 Configuración del entrenamiento

La implementación utiliza el módulo QRDQN de `sb3-contrib` (Stable-Baselines3 Contributors 2023). El modelo emplea una política de perceptrón multicapa sobre las observaciones de 152 dimensiones. El entrenamiento utiliza un buffer de repetición, actualizaciones por minibatch, una red objetivo y predicción determinista durante la evaluación. El script de entrenamiento mantenido admite preajustes rápidos y completos, modos de partición aleatoria y por día, semillas explícitas, límites de filas configurables y pasos temporales configurables.

Los valores por defecto del preajuste completo actual utilizan lotes más grandes y más pasos de gradiente que el preajuste rápido. Esta separación permite al proyecto ejecutar comprobaciones rápidas de humo sin confundirlas con las ejecuciones finales del benchmark. Cualquier resultado reportado debe incluir el identificador exacto de la ejecución y la configuración.

El Algoritmo 3 formaliza este pipeline de entrenamiento de extremo a extremo, desde la carga de los datos brutos de CICIDS2017 hasta la persistencia de los artefactos finales, con independencia del preajuste concreto o de los valores de hiperparámetros seleccionados en cada ejecución. La secuencia de pasos que ejecuta el script mantenido es única; solo los valores de las variables de configuración, documentados aparte, distinguen una ejecución de otra.

4.6.6 Procedencia de los hiperparámetros

Los hiperparámetros ($\gamma = 0$, arquitectura `[1024, 1024, 512]`, 200 cuantiles, tasa de aprendizaje 5×10^{-5} , 20 pasos de gradiente y 3 000 000 de pasos para el perfil completo) constituyen el perfil fijo `main-v1`, establecido manualmente y congelado antes de la campaña final. No proceden de una búsqueda automática. El repositorio conserva una sonda Optuna exploratoria cuyo espacio no contiene esta combinación; por tanto, el perfil no debe presentarse como un óptimo

Algoritmo 3 Entrenamiento del agente defensor QRDQN sobre CICIDS2017: carga y limpieza de datos, mapeo canónico, partición train/test, ajuste del escalador exclusivamente sobre train, bucle de entrenamiento QRDQN con $\gamma = 0$ y persistencia de artefactos. Fuente: elaboración propia a partir de `src/train_rl_defender.py`.

Require: CSV de CICIDS2017; semilla; preajuste (rápido/completo); modo de partición (aleatorio/día); hiperparámetros QRDQN $\eta, B, G, \tau, \varepsilon_0, \varepsilon_T, f_\varepsilon$

Ensure: Modelo QRDQN, escalador, percentiles de entrenamiento, configuración y métricas de test persistidos

- 1: Fijar la semilla en `numpy/random/torch`; cargar y limpiar los CSV según el preajuste $\triangleright$ sin escalar
 - 2: Mapear a esquema canónico (76 características + máscara de 76) $\rightarrow$ observaciones de 152 dimensiones; particionar en $(X_{\text{train}}, y_{\text{train}}), (X_{\text{test}}, y_{\text{test}})$
 - 3: Calcular percentiles $p_{0.5}, p_{99.5}$ de X_{train} sin escalar; ajustar $\Sigma = \text{StandardScaler}$ solo sobre X_{train} $\triangleright X_{\text{test}}$ no interviene en el ajuste
 - 4: $X_{\text{train}} \leftarrow \Sigma(X_{\text{train}})$; $X_{\text{test}} \leftarrow \Sigma(X_{\text{test}})$; persistir Σ , percentiles, nombres de características y metadatos del entorno
 - 5: Construir el entorno Gymnasium sobre $(X_{\text{train}}, y_{\text{train}})$ con la matriz de recompensa fijada
 - 6: Inicializar red de cuantiles θ , red objetivo $\theta^- \leftarrow \theta$, buffer $\mathcal{D} \leftarrow \emptyset$, $\varepsilon \leftarrow \varepsilon_0$, $t \leftarrow 0$
 - 7: **while** $t < \text{total de pasos temporales}$ **do**
 - 8: Observar s ; con probabilidad ε , $a \leftarrow$ acción aleatoria en $\{\text{PERMIT}, \text{BLOCK}\}$; en caso contrario $a \leftarrow \arg \max_{a'} \frac{1}{N} \sum_i \theta_i(s, a')$
 - 9: Ejecutar a ; observar r, s' , terminado; almacenar $(s, a, r, s', \text{terminado})$ en $\mathcal{D}$; $t \leftarrow t + 1$
 - 10: Actualizar ε por decaimiento lineal de ε_0 a ε_T sobre la fracción f_ε del total de pasos
 - 11: **if** $t \bmod \text{intervalo de red objetivo} = 0$ **then**
 - 12: $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$ $\triangleright \tau = 1$: copia dura
 - 13: **end if**
 - 14: **if** $t \bmod \text{train_freq} = 0$ y $t \geq \text{learning_starts}$ **then**
 - 15: **for** $g = 1, \dots, G$ **do**
 - 16: Muestrear de $\mathcal{D}$ un minibatch $\{(s_k, a_k, r_k, s'_k, \text{terminado}_k)\}_{k=1}^B$
 - 17: Para cada k y cada cuantil objetivo $j = 1, \dots, N$: $y_j \leftarrow r_k$ $\triangleright \gamma = 0$: sin bootstrap ni θ^- ; regresión hacia r_k
 - 18: Pérdida $\mathcal{L} \leftarrow$ Huber cuantílica entre $\theta_i(s_k, a_k)$ y $\{y_j\}$, sumada sobre todos los pares (i, j)
 - 19: Actualizar θ por descenso de gradiente sobre $\mathcal{L}$ (Adam, tasa η); recortar norma del gradiente si corresponde
 - 20: **end for**
 - 21: **end if**
 - 22: **end while**
 - 23: Guardar el modelo; evaluar de forma determinista sobre $(X_{\text{test}}, y_{\text{test}})$; calcular matriz de confusión y métricas
 - 24: Persistir configuración final, métricas de test y manifiesto de artefactos
-

seleccionado por esa sonda.

El listado completo de valores congelados se recoge en la Tabla 4.4. La MAIN histórica comparte estos valores relevantes, pero no sustituye la ejecución fresca exigida por la campaña.

4.6.7 Persistencia del modelo y el preprocesamiento

El entrenamiento produce un directorio de ejecución conforme al esquema de artefactos 3.0. Este conserva configuración, métricas, entorno, monitorización, tiempos, resumen, modelo, escalador, percentiles de entrenamiento, nombres de características, manifiesto y sumas SHA-256. Persistir el preprocesamiento es esencial porque la Fase 2 debe reutilizar exactamente el estado de MAIN. Reajustar un escalador sobre tráfico de laboratorio filtraría información de la distribución de inferencia y rompería la trazabilidad.

4.7 Marco de implementación

El pipeline experimental está implementado en Python y se apoya en un conjunto de bibliotecas de código abierto bien establecidas. El componente de RL profundo principal utiliza la extensión `sb3-contrib` de Stable Baselines3, que proporciona una implementación contrastada de QRDQN con arquitectura de red configurable, buffer de repetición, programa de exploración y actualizaciones de la red objetivo (Stable-Baselines3 Contributors 2023). El envoltorio del entorno sigue la API de Gymnasium (Farama Foundation 2023), haciendo que el conjunto de datos de flujos sea compatible con cualquier bucle de entrenamiento que cumpla con Gymnasium sin cambios en la implementación del algoritmo.

Hiperparámetro	Valor	Observación
Arquitectura de la red (net_arch)	[1024, 1024, 512]	MLP de tres capas ocultas
Número de cuantiles (n_quantiles)	200	distribución de retorno QRDQN
Tasa de aprendizaje	5×10^{-5}	
Tamaño del buffer de repetición	1 000 000	transiciones
Pasos de calentamiento (learning_starts)	50 000	
Tamaño de lote (batch_size)	2048	
Factor de descuento (γ)	0.0	formulación de un solo paso con $\gamma = 0$ la red objetivo no contribuye al error de diferencia temporal
Tasa de actualización objetivo (τ)	1.0	pasos
Intervalo de actualización objetivo	10 000	
Frecuencia de entrenamiento (train_freq)	100	pasos
Pasos de gradiente por actualización	20	
ϵ inicial de exploración	1.0	decaimiento lineal
ϵ final de exploración	0.02	decaimiento lineal
Fracción de exploración	0.1	decaimiento lineal
Norma máxima de gradiente	10.0	
Pasos totales de entrenamiento	3 000 000	
Semilla de partición	42	split_seed ; fija la división de datos
Semilla del modelo	42	model_seed ; controla aprendizaje e inicialización

Tabla 4.4: Hiperparámetros congelados del perfil **main-v1**. Perfil fijado a mano, no procedente de la sonda Optuna. Fuente: especificación de campaña y perfil mantenido en el repositorio.

4.7.1 Pila de procesamiento de datos y aprendizaje supervisado

La carga y el preprocesamiento de datos se apoyan en `pandas` para la ingesta de CSV, la estandarización de columnas y la lectura fragmentada de ficheros grandes. Las matrices de características se convierten a arrays de `numpy` y se castean a `float32` antes de entrar en el entorno de Gymnasium o en la línea base supervisada. La estandarización utiliza `StandardScaler` de `scikit-learn`, elegido por su compatibilidad con el protocolo de serialización de `sklearn`, que permite persistir y recargar los escaladores ajustados sin reimplementación. La línea base de Random Forest también utiliza `scikit-learn`, consumiendo los mismos arrays de entrada que entran en el entorno de RL. Esta ruta de datos compartida garantiza que cualquier diferencia de rendimiento entre los dos modelos refleje el comportamiento algorítmico y no una discrepancia en el manejo de los datos.

4.7.2 Gestión de artefactos

Cada ejecución produce un `RUN_ID` único y autocontenido. Los trabajos de validación e inferencia generan artefactos independientes que enlazan la ejecución fuente y la especificación. Al cerrar la campaña, el exportador copia cada ejecución física sellada y genera un paquete comprimido con suma de comprobación; los puntos declarados como alias reutilizan la misma procedencia sin duplicar binarios. Git conserva la especificación, índices y evidencia ligera revisable, mientras que Git LFS se reserva para binarios rastreados que lo requieran.

4.7.3 Controles de reproducibilidad

`split_seed` gobierna la selección y partición de filas; `model_seed` se propaga a `numpy`, `random`, `torch`, el entorno y el modelo. Cada punto conserva ambos valores. El estudio de semillas mantiene fijo `split_seed=42` y cambia `model_seed=42-46`, con un directorio por ejecución. Estos controles reducen ambigüedad, aunque no eliminan todo el indeterminismo de kernels GPU; el entorno real y sus versiones se capturan en `environment.json`.

Protocolo experimental

El protocolo distingue entre el diseño aprobado y la evidencia ya ejecutada. La especificación autoritativa de la campaña final es `experiments/final_experiment_campaign.json`; los artefactos históricos se conservan para contexto, pero no satisfacen una celda final por coincidencia aproximada. El perfil científico `main-v1` está congelado y la campaña no modifica arquitectura, recompensa, contrato canónico ni hiperparámetros MAIN.

5.1 Preguntas experimentales

La campaña responde a siete preguntas ordenadas:

- P1.** ¿reproduce una MAIN fresca la capacidad de aprendizaje intradistribución del pipeline actual?;
- P2.** ¿coinciden la evaluación persistida y una validación directa e independiente del modelo?;
- P3.** ¿qué parte del resultado aleatorio puede asociarse a duplicados y cuánto cambia al separar días?;

- P4.** ¿cómo varía el comportamiento al aumentar el tamaño de entrenamiento y su presupuesto proporcional?;
- P5.** ¿cuánto cambia al variar solo la semilla del modelo sobre una partición fija?;
- P6.** ¿qué sensibilidad aparece al retener cuatro escenarios dirigidos?;
- P7.** ¿los patrones observados son específicos de QRDQN o también aparecen en Random Forest, y cómo se transfieren a la captura de laboratorio?

La Figura 5.1 muestra las dependencias entre bloques. MAIN es a la vez experimento primario y fuente de los controles auxiliares; los alias reutilizan una ejecución física solo cuando la especificación declara identidad exacta.

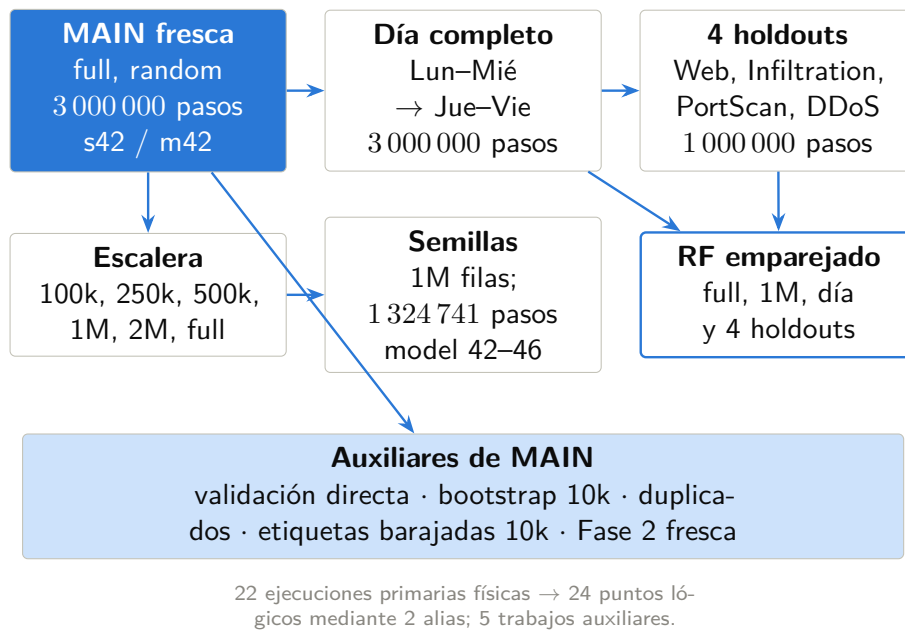


Figura 5.1: Relación entre la MAIN fresca, la partición completa por días, la escalera de tamaños, la sensibilidad a semilla, los cuatro escenarios retenidos, las comparaciones RF y las validaciones auxiliares. Las flechas indican reutilización de artefactos o comparación emparejada, no resultados. Fuente: elaboración propia a partir de `experiments/final_experiment_campaign.json`.

5.2 Datos, unidad de análisis y fases

La unidad es una fila de flujo. La Fase 1 emplea los ocho CSV de CICIDS2017 después de limpieza, mapeo binario y transformación al esquema canónico. La Fase 2 aplica el modelo final de forma offline a una captura etiquetada del laboratorio del autor. En ningún caso se actúa sobre una red ni se retroalimenta el entorno con la decisión.

La composición por día de CICIDS2017 se mantiene como contexto del protocolo porque explica por qué una separación temporal modifica simultáneamente prevalencia y repertorio de ataques. La Figura 5.2 procede de conteos comprometidos, no de la futura campaña.

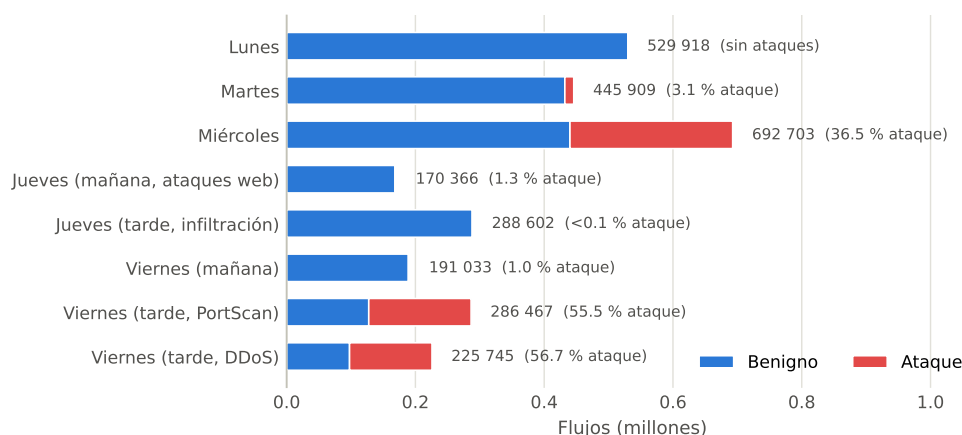


Figura 5.2: Filas y proporción de tráfico benigno/ataque por CSV de CICIDS2017. Fuente: conteos deterministas conservados en `memoria/figuras/data_composicion_dia.json`.

5.3 Preprocesamiento y control de fuga

El pipeline construye primero una representación canónica *sin escalar*. La caché aprobada almacena esta representación y metadatos de procedencia para evitar repetir la carga y coerción de los CSV. No almacena valores normalizados:

el escalador depende de la partición y debe ajustarse dentro de cada ejecución.

Para una ejecución dada, el orden es:

1. cargar o validar la caché canónica sin escalar;
2. seleccionar filas y construir la partición usando `split_seed`;
3. ajustar `StandardScaler` exclusivamente sobre entrenamiento;
4. transformar entrenamiento y prueba con ese estado;
5. entrenar con `model_seed`;
6. persistir modelo, escalador, percentiles, métricas y procedencia.

La separación entre semillas evita que un estudio de estabilidad del modelo cambie inadvertidamente el test. Los identificadores de flujo, IP, tiempo y puertos específicos excluidos por el contrato canónico no se reintroducen desde la caché. Los controles de manifiesto y firma impiden reutilizar una caché creada con otro conjunto de fuentes o versión de esquema.

5.4 Perfil congelado main-v1

El perfil MAIN fija la representación de 152 dimensiones, la arquitectura [1024,1024,512], 200 cuantiles, tasa de aprendizaje 5×10^{-5} , $\gamma = 0$, 20 pasos de gradiente por ciclo y la recompensa `tp=1.5`, `fp=-2.0`, `fn=-5.0`, `omission=0.0`. La ejecución completa usa 3 000 000 de pasos. Estos valores proceden del perfil congelado, cuya firma se registra en la especificación; no se vuelven a ajustar durante la campaña.

La MAIN final `qrdqn_main_random_full_s42_m42` usa el conjunto completo, partición aleatoria estratificada, `split_seed=42`, `model_seed=42` y 3 000 000 de pasos. Es una ejecución fresca: la MAIN histórica de 2026 no se promueve ni se copia para ocupar esta celda.

5.5 Matriz primaria

5.5.1 Partición aleatoria y día completo

La MAIN usa una división aleatoria estratificada 80/20. Este diseño conserva la prevalencia aproximada y ofrece el punto intradistribución común para QRDQN y RF. Su objetivo es verificar capacidad de aprendizaje, no generalización externa.

El contraste temporal `qrdqn_day_full_s42_m42` entrena con lunes, martes y miércoles y prueba con jueves y viernes. Usa el perfil completo y 3 000 000 de pasos. Sustituye como evidencia temporal principal a la Comprobación C histórica, que era una red proxy de 30 000 pasos.

5.5.2 Escalera de tamaños

La escalera mantiene la partición aleatoria y fija ambas semillas en 42. Incluye 100 000, 250 000, 500 000, 1 000 000, 2 000 000 de filas de entrenamiento y el conjunto completo. Los presupuestos son, respectivamente, 132 474, 331 185, 662 370, 1 324 741, 2 649 482 y 3 000 000 de pasos. El punto completo es un alias de MAIN y no requiere una segunda ejecución física.

La curva permitirá describir eficiencia dentro de este protocolo. Como tamaño y pasos cambian juntos, no se interpretará como efecto causal puro del número de filas.

5.5.3 Sensibilidad a la semilla del modelo

El estudio fija 1 000 000 de filas, 1 324 741 pasos y `split_seed=42`. Solo cambia `model_seed` entre 42, 43, 44, 45 y 46. El punto 42 es alias del peldaño de un millón. La salida final reportará la distribución y los puntos individuales, no solo una media.

5.5.4 Cuatro escenarios retenidos dirigidos

El estudio retiene exactamente cuatro CSV: Web Attacks, Infiltration, PortScan y DDoS. Cada QRDQN se entrena durante 1 000 000 de pasos sobre los restantes CSV, con semillas de partición y modelo fijadas en 42. El propósito es cubrir escenarios de naturaleza distinta con un coste experimental controlado. No se presenta como leave-one-CSV-out exhaustivo de los ocho ficheros.

5.6 Random Forest emparejado

Random Forest utiliza la misma representación canónica, filas de entrenamiento, conjunto de prueba y métricas de cada contraste. La campaña incluye RF para la MAIN aleatoria completa, el punto aleatorio de un millón, la partición completa por días y los cuatro escenarios retenidos. No recibe presupuesto de pasos, pero sí un contrato de partición y semillas explícito.

Esta línea base permite separar una dificultad del dominio de un efecto específico de QRDQN. Las comparaciones se realizarán por pares; no se mezclan con puntuaciones publicadas bajo otros filtros o particiones.

5.7 Controles auxiliares

La MAIN fresca alimenta cinco trabajos auxiliares:

- **validación directa:** recarga modelo y preprocesamiento, reproduce la partición y verifica predicciones y conteos;
- **bootstrap:** 10 000 remuestreos estratificados con semilla 12345 sobre el test fijo;

- **duplicados:** cuantifica duplicados internos y coincidencias exactas entre entrenamiento y prueba;
- **etiquetas barajadas:** control breve de 10 000 pasos que debe perder la señal predictiva cuando se destruye la relación con la etiqueta;
- **Fase 2 fresca:** aplica MAIN, escalador y percentiles finales a la captura de laboratorio.

El *bootstrap* no reentrena el modelo y no mide varianza entre semillas. El análisis de duplicados tampoco atribuye por sí mismo causalidad al rendimiento. Ambos se interpretan junto con los contrastes que cambian la distribución.

5.8 Jerarquía de evaluación

La Figura 5.3 organiza las afirmaciones posibles. El nivel no aumenta porque una métrica sea más alta, sino porque la separación entre entrenamiento y prueba es más exigente. La Fase 2 añade una fuente distinta, aunque su captura siga siendo cerrada y generada por el autor.

5.9 Métricas y reglas de lectura

Se reportan matriz de confusión, exactitud, exactitud balanceada, MCC, precisión, *recall* y F1 por clase, además de tasas de falsos positivos y falsos negativos. La clase ATTACK es positiva. Para seguridad, *recall* de ataque y tasa de falsos negativos reflejan ataques omitidos; para impacto, la tasa de falsos positivos refleja benignos bloqueados. La exactitud nunca se interpreta aislada.

Las curvas de entrenamiento describen optimización, no evaluación. Los intervalos se presentan con su unidad de incertidumbre. Las comparaciones por

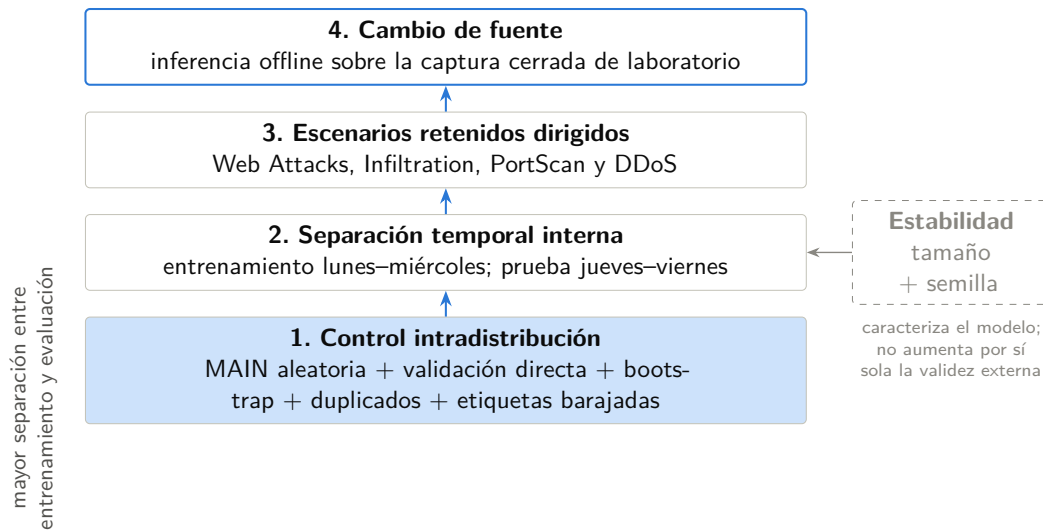


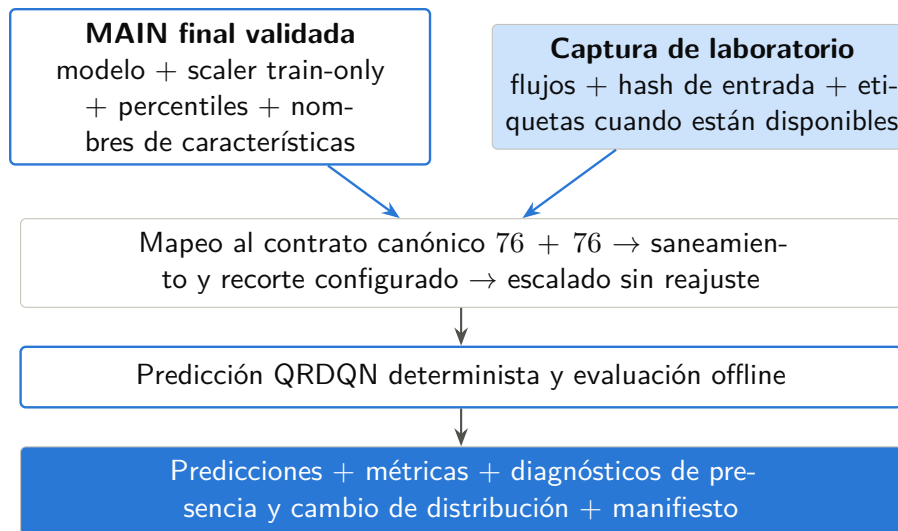
Figura 5.3: Jerarquía de evidencia: control intradistribución, separación por día, escenarios retenidos y transferencia a laboratorio. La sensibilidad a tamaño y semilla caracteriza estabilidad, pero no constituye por sí misma un nivel de validez externa. Fuente: elaboración propia.

escenario incluyen tamaño y prevalencia del test para evitar que una media oculte distribuciones distintas.

5.10 Fase 2: inferencia offline y diagnósticos

La inferencia fresca carga el modelo, escalador, percentiles y nombres de características de MAIN. El mapeo construye la observación canónica, registra qué columnas fuente están disponibles, sanea valores no finitos de acuerdo con el contrato y aplica el escalador persistido. No vuelve a ajustar ningún estado con la captura de laboratorio.

Los diagnósticos incluyen presencia de características, valores no finitos, valores estandarizados extremos, recorte respecto a percentiles de entrenamiento y tasa de bloqueo. Si existen etiquetas válidas se calculan las mismas métricas binarias. La Figura 5.4 refleja que la fuente del modelo será la MAIN final y que las salidas mantienen procedencia.



BLOCK es una etiqueta escrita en artefactos; no se ejecuta ninguna acción sobre la red.

Figura 5.4: La MAIN final proporciona modelo, escalador, percentiles y contrato; la captura de laboratorio proporciona flujos y, cuando procede, etiquetas. La salida incluye predicciones, métricas y diagnósticos, sin actuar sobre la red. Fuente: elaboración propia a partir del pipeline actual de Fase 2.

5.11 Contrato de artefactos y cierre de campaña

Cada ejecución primaria del esquema 3.0 persiste, como mínimo, `config.json`, `metrics.json`, `environment.json`, `run_summary.json`, `timings.json`, monitorización, modelo, escalador, percentiles, nombres de características, manifiesto y sumas SHA-256. Los trabajos auxiliares usan contratos equivalentes adaptados a su salida. El `RUN_ID` identifica el directorio y enlaza configuración, evidencia y agregado.

La campaña ejecuta 22 trabajos primarios físicos que representan 24 puntos lógicos mediante dos alias, más cinco trabajos auxiliares. El cierre exige que todos estén completos, sean coherentes con `main-v1` y pasen la agregación. Los alias no duplican modelos ni exportaciones.

Los artefactos sellados se copian al destino de evidencia junto con un archivo comprimido y su suma. Git conserva especificación, índices, métricas y

manifiestos adecuados para revisión; Git LFS se reserva para binarios rastreados que exceden el flujo normal. Los datos originales, cachés grandes, directorios de ejecución activos y exportaciones voluminosas no se incorporan por defecto al historial.

5.12 Plataforma experimental y preflight

La metodología final utiliza lenguaje neutral: *plataforma GPU experimental*. Antes de ejecutar se comprueban commit, limpieza del árbol, disponibilidad de datos y LFS, firma del perfil, caché, espacio, memoria, CUDA, importaciones, capacidad de crear artefactos y un ensayo corto no científico. Esta verificación determina si la plataforma puede ejecutar el protocolo; no produce resultados de tesis.

El hardware, sistema operativo, controlador, CUDA, GPU, CPU, RAM y paquetes finales solo se describirán desde los `environment.json` de las ejecuciones aceptadas. No se anticipa un proveedor ni una GPU concreta. Los detalles de la plataforma histórica permanecen únicamente en el Anexo B como procedencia de evidencia previa.

5.13 Estado del protocolo

El diseño, la implementación y los contratos de artefactos están preparados. La ejecución científica permanece pendiente. Esta separación permite redactar el método como texto casi final y, al mismo tiempo, impide que una orden preparada, un *dry-run* o un artefacto histórico se conviertan en evidencia final.

Evidencia final pendiente. La campaña no se ha ejecutado durante esta revisión de la memoria. El Capítulo de Resultados solo podrá poblarse después de validar el conjunto completo de artefactos contra la especificación congelada.

Resultados

Este capítulo separa el estado de la evidencia de su interpretación. En la fecha de este borrador, la campaña final definida en `experiments/final_experiment_campaign.json` todavía no se ha ejecutado. En consecuencia, primero se fija la estructura en la que se incorporarán sus artefactos y, después, se conserva una síntesis de resultados históricos útiles para supervisión. Estos últimos no constituyen la evaluación final del sistema y serán sustituidos o relegados cuando exista una campaña completa y validada.

6.1 Condiciones generales y trazabilidad

Una cifra se considera evidencia final únicamente si puede trazarse desde una ejecución de la campaña aprobada hasta su `config.json`, `metrics.json` o fichero de validación, `environment.json`, resumen, manifiesto y sumas de comprobación. La Figura 6.1 resume esa cadena. Esta regla evita mezclar tres objetos distintos: el perfil experimental congelado, los artefactos históricos ya existentes y los artefactos finales aún pendientes.

La Tabla 6.1 fija el orden definitivo. No se crean tablas vacías para simular resultados: cada bloque se incorporará cuando el agregador haya comprobado

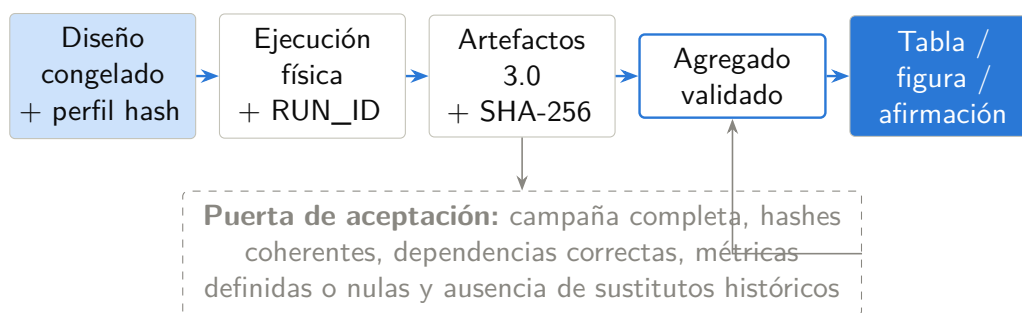


Figura 6.1: Cadena de trazabilidad exigida para incorporar una cifra final: especificación congelada, ejecución identificada, artefactos sellados, agregación validada y elemento de tesis. Una ausencia o inconsistencia detiene la incorporación. Fuente: elaboración propia a partir del esquema de artefactos de campaña.

que están presentes todas las ejecuciones físicas y alias previstos.

Evidencia final pendiente. Los siete bloques de la Tabla 6.1 permanecen sin valores finales. La actualización posterior deberá producirse desde el agregado validado de campaña, nunca mediante transcripción estimada o reutilización silenciosa de cifras históricas.

6.2 Evidencia histórica previa a la campaña final

Los siguientes resultados proceden de artefactos comprometidos antes de congelar la campaña actual. Siguen siendo útiles para comprobar el funcionamiento del pipeline, identificar amenazas y justificar el diseño de la campaña; no deben leerse como una ejecución anticipada de esta.

6.2.1 MAIN histórica: capacidad de aprendizaje intra-distribución

La ejecución histórica `MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655` entrenó QRDQN durante 3 000 000 de pasos sobre una parti-

Bloque final	Pregunta que resuelve	Evidencia requerida
MAIN y controles directos	¿Aprende el perfil congelado en la partición aleatoria y coinciden evaluación y validación independiente?	MAIN nueva, validación directa, <i>bootstrap</i> , duplicados y etiquetas barajadas
Partición completa por días	¿Persiste el comportamiento al romper la mezcla aleatoria entre días?	QRDQN y RF con lunes–miércoles para entrenamiento y jueves–viernes para prueba
Escalera de tamaños	¿Cómo cambia el resultado con la cantidad de datos y el presupuesto proporcional de pasos?	Seis puntos lógicos entre 100 000 filas y el conjunto completo
Sensibilidad a semilla	¿Cuánto varía el modelo al mantener fija la partición?	Cinco semillas de modelo, 42–46, sobre 1 000 000 de filas
Escenarios retenidos	¿Qué ataques o capturas inducen una degradación específica?	Cuatro <i>holdouts</i> dirigidos y sus RF equivalentes
Comparación de modelos	¿Qué parte del comportamiento depende de QRDQN y cuál del protocolo tabular?	Comparaciones emparejadas QRDQN–RF
Fase 2	¿Puede el MAIN final procesar la captura de laboratorio y con qué cambio de dominio?	Inferencia fresca, métricas y diagnósticos de distribución

Tabla 6.1: Orden de incorporación de la campaña final y contrato mínimo de evidencia para cada bloque. La tabla describe decisiones analíticas, no resultados ejecutados. Fuente: elaboración propia a partir de `experiments/final_experiment_campaign.json`.

ción aleatoria estratificada con semilla 42. Sobre 566 149 filas de prueba registró una exactitud de 0.99381, *recall* de ataque de 0.99536 y F1 de ataque de 0.98445. La matriz de confusión fue $TN = 451\,631$, $FP = 2\,989$, $FN = 518$ y $TP = 111\,011$.

Métrica	MAIN histórica
Exactitud	0.99381
Precisión de ataque	0.97378
Recall de ataque	0.99536
F1 de ataque	0.98445
Tasa de falsos positivos	0.00658
Tasa de falsos negativos	0.00465

Tabla 6.2: Métricas de la ejecución MAIN histórica sobre su partición aleatoria fija. No son resultados de la campaña final. Fuente: `metrics.json` y `bootstrap_ci_seed42.json` históricos.

Las curvas de la Figura 6.2 documentan que la ejecución completó un proceso de aprendizaje coherente; no permiten inferir por sí mismas generalización ni estabilidad entre semillas.

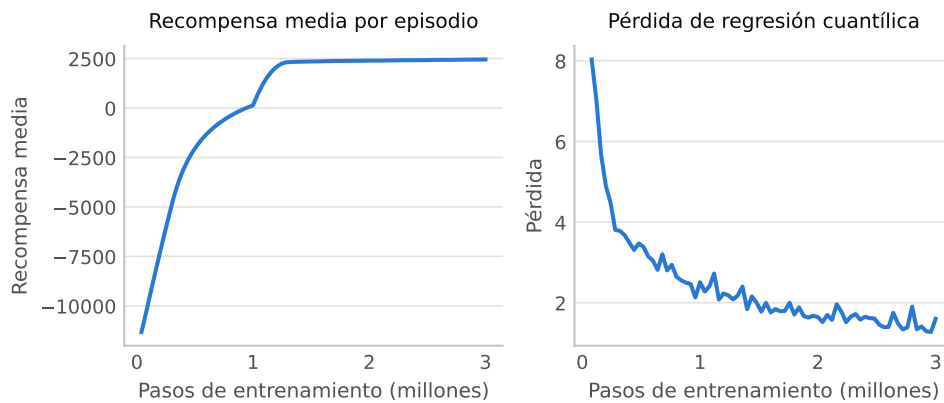


Figura 6.2: Curvas de entrenamiento de la ejecución MAIN histórica. Fuente: escalares TensorBoard exportados del artefacto previo a campaña.

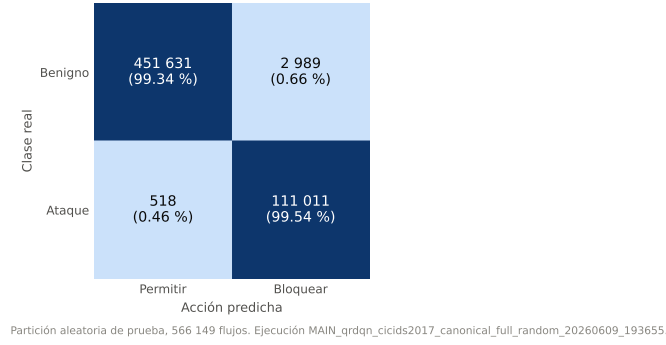


Figura 6.3: Matriz de confusión de la MAIN histórica sobre la partición aleatoria fija. Fuente: `bootstrap_ci_seed42.json`; no es la matriz de la MAIN final.

6.2.2 Precisión de muestreo y duplicados históricos

El *bootstrap* histórico de 10 000 remuestreos estratificados sitúa la F1 de ataque en $[0.98394, 0.98496]$, el *recall* de ataque en $[0.99495, 0.99575]$, la tasa de falsos positivos en $[0.00634, 0.00681]$ y la tasa de falsos negativos en $[0.00425, 0.00505]$. Estos intervalos cuantifican la precisión sobre un conjunto de prueba fijo y un único modelo; no estiman varianza de entrenamiento.

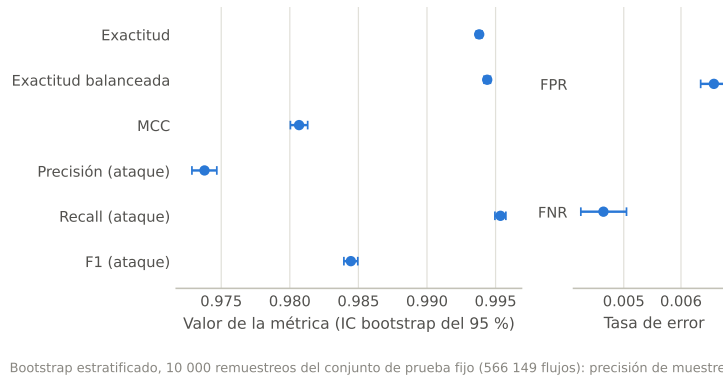


Figura 6.4: Intervalos al 95 % de la MAIN histórica sobre su test fijo. Fuente: `runs/validation/bootstrap_ci_seed42.json`. La campaña final regenerará este análisis.

El análisis histórico de duplicados encontró que el 22.30 % de las filas completas eran duplicados exactos y que el 24.86 % del test aleatorio coincidía con alguna fila de entrenamiento. Entre los ataques de prueba, la coincidencia

ascendía al 40.12 %. No demuestra por sí solo una fuga de etiquetas en la implementación, pero sí que la mezcla aleatoria ofrece una evaluación optimista sobre un benchmark con observaciones repetidas.

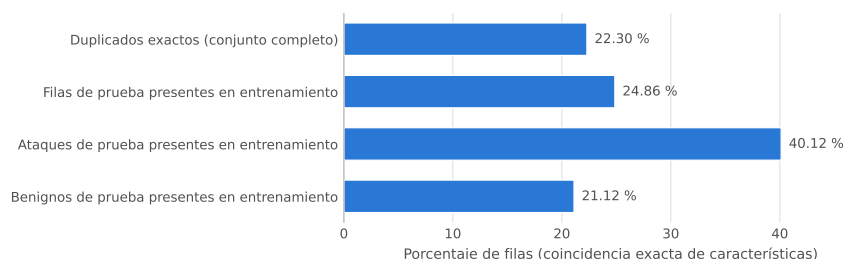


Figura 6.5: Duplicados exactos y coincidencias entre entrenamiento y prueba de la partición histórica con semilla 42. Fuente: `runs/validation/duplicate_analysis_seed42.json`.

6.2.3 Controles A, B y proxy temporal histórico

La Comprobación A reprodujo la predicción directa del modelo sobre 10 000 muestras y confirmó que las métricas no dependían del bucle del entorno. La Comprobación B reentrenó brevemente con etiquetas barajadas y obtuvo 0.4773 de exactitud frente a 0.5227 de la clase mayoritaria, señal compatible con la ausencia de una vía trivial de fuga en ese control.

La Comprobación C entrenó una red proxy [512,256] durante 30 000 pasos con lunes–miércoles y evaluó jueves–viernes. Registró 0.52954 de *recall* y 0.62578 de F1 de ataque. Esta ejecución es una señal histórica de dificultad temporal, pero no sustituye el experimento final: usa otra arquitectura, otro presupuesto y un contrato de artefactos anterior.

6.2.4 Random Forest histórico como referencia interpretativa

El barrido histórico de *Random Forest* empleó la misma representación canónica y particiones equivalentes. En la partición aleatoria superó a QRDQN

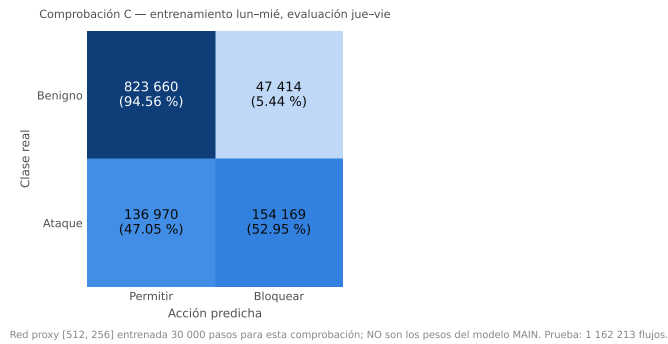


Figura 6.6: Matriz de confusión de la Comprobación C histórica. Fuente: `VAL_checks_C_20260213_004847/validation_results.json`. Se reemplazará como evidencia temporal principal por el experimento completo de 3 000 000 de pasos.

histórica en F1 de ataque (0.99676 frente a 0.98445). En la partición por día, su *recall* de ataque descendió a 0.08135 y su F1 a 0.15005, frente a 0.52954 y 0.62578 del proxy QRDQN. Esta comparación motiva una evaluación emparejada actual, pero no demuestra que QRDQN generalice mejor de forma global: el QRDQN temporal histórico no era MAIN y faltan escenarios repetidos.

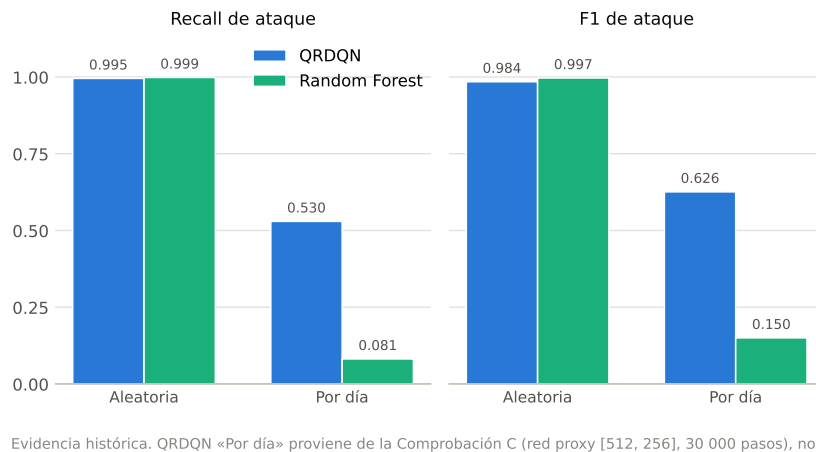


Figura 6.7: Recall y F1 de ataque en las particiones histórica aleatoria y por día. La fila temporal de QRDQN corresponde a un proxy; no existe aquí una comparación final de campaña. Fuente: artefactos históricos MAIN, Check C y RF.

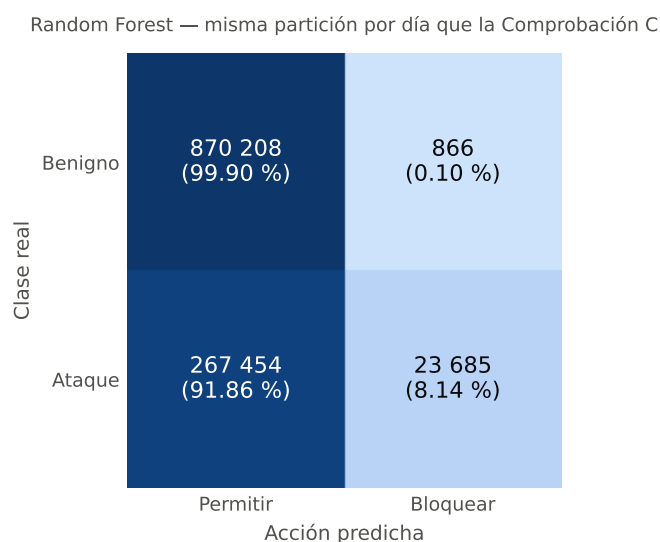


Figura 6.8: Matriz de confusión de Random Forest en la partición histórica por día. Fuente: `rf_cicids2017_canonical_20260628_024735__day_split/metrics.json`.

6.2.5 Inferencia histórica de Fase 2

La ejecución histórica `P2v2_pred_20260610_161231_MAIN` aplicó la MAIN histórica a 2 000 000 de flujos etiquetados del laboratorio del autor. Registró 0.991862 de exactitud, 0.988452 de *recall* de ataque, 0.98380 de F1 de ataque y una tasa de bloqueo de 0.252364. La captura era cerrada, generada por el operador y no adversarial; por ello el resultado solo acredita que ese pipeline procesó esa captura concreta. La inferencia fresca de campaña volverá a ejecutar el análisis con la MAIN final y sus artefactos de preprocesamiento.

6.3 Inserción de la campaña final

La actualización final de este capítulo seguirá la Tabla 6.1. La MAIN fresca se presentará junto con su validación directa, *bootstrap*, duplicados y control de etiquetas; después se analizarán la partición completa por días, la curva de tamaño, la distribución entre semillas y los cuatro escenarios retenidos.

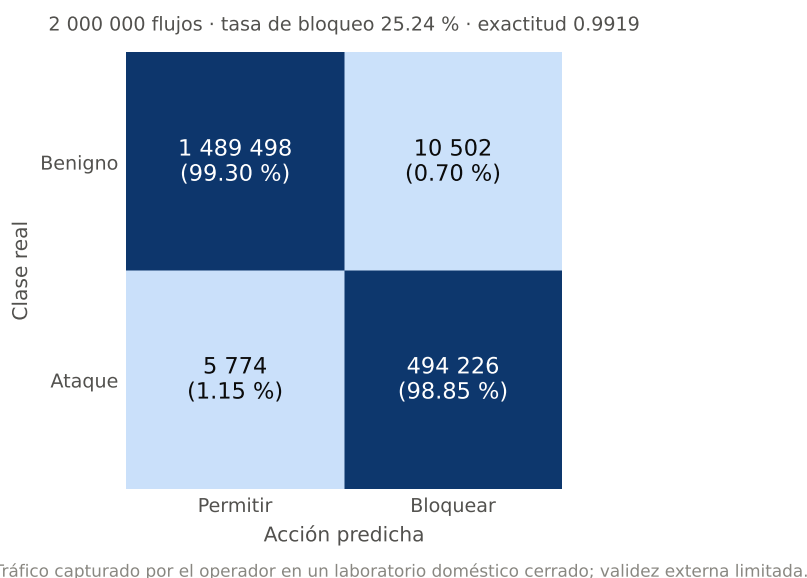


Figura 6.9: Matriz y tasa de bloqueo de la inferencia histórica sobre tráfico de laboratorio. Fuente: `runs/phase2/P2v2_pred_20260610_161231_MAIN/metrics.json`. Su validez externa es limitada.

Solo entonces se realizará la comparación emparejada con Random Forest y se cerrará con la inferencia fresca de Fase 2. Este orden separa capacidad intradistribución, robustez y transferencia, y evita repetir la misma métrica en varias secciones.

Evidencia final pendiente. No existen todavía valores para la MAIN fresca, el día completo, la escalera de tamaños, las semillas 42–46, los cuatro holdouts, los RF nuevos, la validación directa, el bootstrap, los duplicados, el control barajado ni la Fase 2 fresca. Las figuras cuantitativas finales se generarán únicamente cuando el agregador de campaña declare el conjunto completo.

6.4 Síntesis provisional

La evidencia histórica permite afirmar que la implementación aprende la tarea definida y que el protocolo aleatorio, aun siendo útil como comprobación

interna, no basta para sostener generalización. También justifica mantener Random Forest como línea base fuerte y tratar la transferencia al laboratorio como una comprobación offline de alcance limitado. No permite determinar todavía la estabilidad entre semillas, la eficiencia respecto al tamaño, la sensibilidad de los cuatro escenarios, el comportamiento del perfil completo por días ni el balance final entre QRDQN y RF. Esas son, exactamente, las preguntas reservadas a la campaña final.

Discusión

La discusión no repite el inventario del Capítulo 6.1; relaciona cada bloque de evidencia con la afirmación científica que puede sostener. Mientras la campaña final permanezca pendiente, los razonamientos que dependen de ella se expresan como criterios de decisión y no como resultados anticipados.

7.1 Rendimiento intradistribución frente a generalización

Una partición aleatoria estratificada responde a una pregunta necesaria pero limitada: si entrenamiento y prueba se extraen de la misma mezcla de CICIDS2017, ¿puede el pipeline aprender la regla binaria definida? La MAIN histórica responde afirmativamente para una ejecución anterior. Sin embargo, esta pregunta no equivale a saber si el modelo reconocerá ataques de otro día, captura o distribución. Los patrones de tráfico, herramientas, prevalencias y familias de ataque varían entre los CSV del benchmark, y una mezcla aleatoria diluye esa separación.

La campaña final trata la partición aleatoria como control intradistribución y

reserva la afirmación de generalización al contraste con el experimento completo por días y los escenarios retenidos. Si MAIN mantiene métricas altas pero estas caen al separar días o CSV, la interpretación correcta será que el modelo explota bien la distribución mezclada, no que ofrece una defensa robusta. Si el comportamiento persiste, la afirmación podrá reforzarse únicamente para esos desplazamientos observados; seguirá sin equivaler a producción independiente.

7.2 Duplicados y continuidad entre particiones

El análisis histórico encontró una proporción relevante de coincidencias exactas entre entrenamiento y prueba. Esto no prueba una fuga de etiqueta dentro del código ni invalida automáticamente toda evaluación aleatoria: demuestra que el conjunto de prueba contiene menos novedad de la que su tamaño sugiere. En consecuencia, los intervalos estrechos obtenidos por *bootstrap* pueden coexistir con una validez externa débil. El primero mide precisión sobre un test fijo; la segunda depende de la separación que ese test proporciona.

La regeneración final del análisis de duplicados debe interpretarse junto con el día completo. Si la tasa de coincidencias vuelve a ser elevada y el rendimiento temporal disminuye, ambas señales ofrecerán una explicación coherente del optimismo aleatorio. Si el rendimiento temporal se conserva, los duplicados seguirán siendo una amenaza del benchmark, pero no bastarán para atribuirles el resultado del modelo.

7.3 Lectura sensible al coste

La recompensa distingue cuatro consecuencias: permitir tráfico benigno, bloquear un ataque, bloquear tráfico benigno y permitir un ataque. La penalización más severa del falso negativo expresa una preferencia defensiva, pero

no elimina el coste del falso positivo. Por eso la exactitud agregada no basta: deben leerse conjuntamente *recall* de ataque, tasa de falsos negativos, precisión de ataque, tasa de falsos positivos y matriz de confusión.

Esta lectura es especialmente importante fuera de la partición aleatoria. Un modelo puede conservar una precisión alta de ataque porque bloquea muy pocos flujos y, a la vez, omitir la mayoría de las intrusiones. También puede mejorar *recall* a costa de interrumpir tráfico legítimo. La campaña no fijará un umbral operativo ni ejecutará bloqueos reales; comparará el compromiso producido por las políticas entrenadas bajo una función de recompensa explícita.

7.4 QRDQN frente a Random Forest

Random Forest no se incluye como adversario débil, sino como prueba de necesidad. Con $\gamma = 0$ y filas independientes, QRDQN resuelve una tarea próxima a clasificación tabular sensible al coste. Un clasificador supervisado sólido bajo el mismo esquema, partición y métrica permite preguntar qué aporta realmente la formulación distribucional, en lugar de atribuir a RL cualquier rendimiento que provenga de las características o del benchmark.

La evidencia histórica muestra que RF puede dominar la partición aleatoria y fallar ante una separación temporal concreta. Ese contraste es indicativo, pero está desbalanceado porque el QRDQN temporal histórico era un proxy. La campaña final corrige esta asimetría con pares equivalentes en MAIN, un millón de filas, día completo y cuatro escenarios retenidos. Si RF y QRDQN se degradan de forma parecida, la limitación principal apuntará a datos y dominio. Si difieren de forma consistente, podrá discutirse una sensibilidad específica de la familia de modelo. Un resultado favorable aislado no justificará superioridad general.

7.5 Conjunto de datos como entorno y $\gamma = 0$

El uso de un conjunto tabular como entorno aporta una interfaz explícita de acciones y recompensas, compatibilidad con instrumentación de RL y una base para estudiar políticas sensibles al coste. A la vez, con $\gamma = 0$ la devolución no incorpora consecuencias futuras: cada decisión depende de la observación actual y de su recompensa inmediata. La distribución cuantílica modelada por QRDQN describe la devolución de esa decisión, no una secuencia causal de estados de red.

Esta formulación enseña una lección metodológica: llamar «entorno» a un iterador sobre filas no convierte automáticamente la detección en un problema secuencial. La comparación supervisada es, por tanto, parte constitutiva de la evaluación. Una extensión genuinamente secuencial exigiría definir estado temporal, consecuencias de las acciones, episodios con dinámica de red y riesgos de intervención; queda fuera de este trabajo.

7.6 Eficiencia de datos y sensibilidad a la semilla

La escalera de tamaños distinguirá dos explicaciones que una única MAIN no puede separar: que el comportamiento dependa de casi todo CICIDS2017 o que se alcance con menos datos. Los presupuestos de pasos crecen proporcionalmente hasta el perfil completo, por lo que la curva debe leerse como respuesta conjunta a volumen y presupuesto de optimización. No medirá una ley universal de escalado.

El estudio de semillas responde a otra incertidumbre. Mantiene 1 000 000 de filas y la semilla de partición 42, y cambia solo la semilla del modelo entre 42 y 46. Así estima sensibilidad de inicialización y aprendizaje sobre una partición

fija; no incluye variación del conjunto de prueba. Media, dispersión, rango y casos atípicos deberán reportarse juntos. Una diferencia pequeña entre cinco ejecuciones reforzará la estabilidad local de ese punto, pero no sustituirá más particiones ni el MAIN completo repetido.

7.7 Sensibilidad por escenario

Los cuatro *holdouts* —Web Attacks, Infiltration, PortScan y DDoS— fueron elegidos como escenarios dirigidos, no como barrido exhaustivo de los ocho CSV. La comparación debe mostrar el soporte y prevalencia de cada test junto con las métricas: un F1 no es comparable si la composición o la clase positiva cambian sustancialmente. El patrón entre escenarios será más informativo que una media única, porque permitirá identificar qué distribución retenida provoca cada error.

Esta selección limita la inferencia. Incluso si los cuatro escenarios fueran favorables, no quedaría demostrada invariancia frente a cualquier CSV, ataque o día. Si alguno falla, tampoco se deduce que la arquitectura sea inútil; sí se identifica una frontera concreta que requiere análisis de características, cobertura y posible cambio de dominio.

7.8 Fase 2 y cambio de dominio

La Fase 2 comprueba que el modelo, el escalador, los percentiles y el contrato canónico pueden aplicarse a flujos externos al cargador de CICIDS2017. Sus diagnósticos —presencia de características, valores no finitos, recorte y puntuaciones estandarizadas— ayudan a distinguir un error del pipeline de un desplazamiento de entrada. Cuando existan etiquetas, la matriz de confusión añade una señal de comportamiento.

No obstante, la captura pertenece a un único laboratorio cerrado, fue generada por el operador y no representa adversarios ni operación independiente. Un resultado alto demostraría compatibilidad y comportamiento sobre esa captura; uno bajo señalaría una limitación de transferencia o del mapeo. Ninguno de los dos, por sí solo, probaría preparación para despliegue.

7.9 Amenazas a la validez y evidencia más fuerte

La validez interna depende del contrato de partición, escalado solo con entrenamiento, semillas explícitas, control de etiquetas barajadas, validación directa y artefactos sellados. La validez de constructo está limitada por el mapeo binario y por tratar decisiones independientes como bandido contextual. La validez externa está limitada por CICIDS2017, cuatro escenarios seleccionados y una captura de laboratorio. La validez de conclusión estará condicionada por cinco semillas en un único tamaño y por la ausencia de repetición completa del MAIN.

Evidencia más fuerte requeriría capturas etiquetadas e independientes de varios entornos, repetición de los principales contrastes, comparación con más familias tabulares bajo el mismo contrato y evaluación previa a cualquier acción real. Esta tesis no pretende cubrir ese último nivel. Su objetivo es dejar una cadena reproducible y honesta que permita saber qué afirmación respalda cada experimento y cuál sigue fuera de alcance.

Limitaciones

Las limitaciones se agrupan aquí para definir el alcance de las conclusiones, no para compensar una lectura optimista de los resultados. Algunas pertenecen al diseño y permanecerán tras la campaña; otras son incertidumbres que la campaña final está diseñada para reducir.

8.1 Benchmark y particiones

CICIDS2017 es un banco de pruebas reproducible, pero no representa por sí solo redes actuales ni operación independiente. Sus días combinan prevalencias y familias de ataque distintas, y la evidencia histórica muestra duplicados exactos entre entrenamiento y prueba aleatorios. Por ello la partición aleatoria sirve como evaluación intradistribución, no como estimación de despliegue. La partición por días y los cuatro escenarios retenidos mejoran la separación, pero siguen perteneciendo al mismo benchmark.

El análisis final de duplicados cuantificará la continuidad de la MAIN fresca. No elimina patrones casi duplicados ni sesgos de captura que una comparación exacta no detecta. Una evaluación externa más fuerte requeriría conjuntos o capturas independientes y un protocolo de armonización de características.

8.2 Formulación binaria y contextual

El mapeo BENIGN/ATTACK reduce familias heterogéneas a una decisión binaria y no evalúa atribución, severidad ni ataques desconocidos. La recompensa expresa una preferencia fija entre falsos negativos y falsos positivos; no ha sido calibrada con costes de una organización real.

Con $\gamma = 0$, las decisiones son contextuales y no incorporan consecuencias futuras. El entorno no modela colas, sesiones, respuesta del adversario, degradación del servicio ni cambios producidos por bloquear. Por tanto, el estudio no demuestra capacidad de defensa secuencial ni control autónomo de red.

8.3 Variabilidad experimental

El estudio aprobado cambia la semilla del modelo entre 42 y 46 sobre una única partición de 1 000 000 de filas. Cinco ejecuciones permiten describir sensibilidad local, pero no estimar toda la distribución de entrenamiento. La MAIN completa no se repite con varias semillas, y la semilla de partición permanece fija. Los intervalos *bootstrap* cubren precisión de muestreo sobre un test, no esta variabilidad.

La escalera de tamaños varía conjuntamente cantidad de filas y pasos de entrenamiento proporcionales. Es adecuada para comparar presupuestos del pipeline, pero no separa de forma causal el efecto de más datos del efecto de más actualizaciones.

8.4 Cobertura de modelos y escenarios

QRDQN es el único algoritmo RL evaluado y Random Forest la única familia supervisada de campaña. La comparación será más fuerte que contrastar cifras publicadas con protocolos distintos, pero no permite generalizar a todos los clasificadores tabulares ni a todo RL profundo.

Los cuatro *holdouts* son dirigidos: Web Attacks, Infiltration, PortScan y DDoS. No constituyen un leave-one-CSV-out exhaustivo de los ocho ficheros. Cada conclusión deberá referirse al escenario observado y a su soporte, evitando promedios que oculten fallos específicos.

8.5 Fase 2 y validez externa

La captura de Fase 2 procede de un laboratorio cerrado, fue generada por el operador y no contiene un adversario independiente. Sus etiquetas describen la intención del generador y no equivalen a anotación operacional. Los diagnósticos de distribución y la inferencia fresca pueden verificar compatibilidad del contrato y revelar cambio de dominio, pero no acreditan robustez en producción.

8.6 Evidencia todavía pendiente

En este borrador no están disponibles la MAIN fresca, la validación directa, el *bootstrap*, los duplicados, el control barajado, el día completo, la escalera de tamaños, las cinco semillas, los cuatro *holdouts*, los RF nuevos ni la Fase 2 fresca. Por ello no se cierran conclusiones cuantitativas sobre esos bloques. El registro operativo de `docs/thesis/FINALIZATION_REGISTER.md` enumera el trabajo restante sin duplicar el plan de campaña.

Evidencia final pendiente. La campaña final reducirá incertidumbre sobre reproducibilidad local, tamaño, escenario y cambio temporal. No resolverá la antigüedad de CICIDS2017, la validez de una captura independiente, la ausencia de dinámica secuencial ni la seguridad de un despliegue activo.

8.7 Alcance operacional

Toda la evaluación es offline. BLOCK es una etiqueta experimental y nunca una orden aplicada a la red. No se han analizado integración con controles reales, latencia en línea, disponibilidad, evasión adaptativa, respuesta a incidentes, revisión humana ni impacto empresarial. Cualquier transición operativa exigiría una evaluación de seguridad nueva y queda fuera del TFG.

Consideraciones éticas y legales

Las herramientas de detección de intrusiones operan sobre tráfico de red que puede contener información sensible, y un sistema que decide entre permitir y bloquear conlleva consecuencias operativas directas. Aunque esta tesis se limita a un prototipo experimental de inferencia offline, las decisiones de diseño que la hacen reproducible y segura tienen también una dimensión ética y legal que conviene hacer explícita.

9.1 Uso dual y alcance estrictamente defensivo

La investigación en ciberseguridad tiene una naturaleza intrínsecamente dual: las mismas técnicas que permiten detectar tráfico malicioso pueden, en principio, informar su evasión. El trabajo aquí descrito es defensivo por construcción y, además, deliberadamente inerte respecto a cualquier red real. Las salidas **PERMIT** y **BLOCK** son clasificaciones experimentales sobre registros de flujo estáticos: el sistema no realiza bloqueo en vivo, descarte de paquetes ni manipulación de **iptables**, y no se integra con cortafuegos, controladores de redes definidas por software (SDN) ni sistemas de prevención de intrusiones (IPS), tal como se delimita en los no objetivos del Capítulo 2. Esta ausencia de

actuación en línea elimina el riesgo de que el artefacto, por sí mismo, provoque un bloqueo indebido de tráfico legítimo en una red en producción. El sistema es un instrumento de medición experimental, no un mecanismo de defensa desplegable.

Esta decisión de alcance también limita el riesgo de uso dual práctico. El repositorio contiene código de entrenamiento, validación e inferencia offline, pero no instrucciones para operar el modelo como herramienta de interrupción activa de tráfico ni para integrarlo en una infraestructura ajena. Incluso en la Fase 2, el resultado es un fichero de predicciones y métricas, no una acción ejecutada sobre la red. La utilidad defensiva del trabajo reside en estudiar el comportamiento de un detector sensible al coste y en documentar sus límites, no en proporcionar una capacidad operativa inmediata.

9.2 Privacidad y tratamiento de los datos

El sistema opera exclusivamente sobre características estadísticas a nivel de flujo y no sobre el contenido (*payload*) de los paquetes, evitando la inspección profunda de paquetes y buena parte de sus implicaciones legales y de privacidad. El contrato canónico de características refuerza esta posición al excluir a nivel de esquema los identificadores potencialmente sensibles o reidentificadores —direcciones IP de origen y destino, marcas temporales absolutas, Flow ID y números de puerto—, que se descartan antes de cualquier entrenamiento. El benchmark CICIDS2017 es un conjunto de datos público publicado por el Canadian Institute for Cybersecurity (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017). El tráfico empleado en la Fase 2 fue generado por el propio operador en un laboratorio doméstico cerrado y no adversarial: se trata de tráfico propio, no contiene datos personales de terceros y no fue capturado de redes ajenas, por lo que su uso no plantea problemas de consentimiento ni de exposición de información personal.

La Fase 2 se diseñó además con separación entre resultados reproducibles y metadatos sensibles. Las salidas que se pueden conservar como artefacto de investigación contienen métricas, diagnósticos y predicciones agregadas o depuradas; cualquier exportación con información operacional más sensible queda tratada como local y no forma parte de la evidencia pública de la memoria. Esta separación es importante porque incluso los metadatos de flujo, sin carga útil, pueden revelar patrones de actividad, horarios o servicios. La tesis evita apoyarse en esos identificadores y centra la evaluación en la representación canónica no identificativa.

9.3 Riesgo operativo y asimetría de errores

Las consecuencias de los dos tipos de error son profundamente asimétricas. Un falso negativo puede permitir que un ataque avance; un falso positivo puede interrumpir tráfico legítimo y consumir capacidad de análisis. La recompensa codifica esta asimetría penalizando más el falso negativo, pero la memoria reporta ambas tasas. La evidencia histórica mostró un contraste fuerte entre una partición aleatoria y un proxy por días; la campaña final está diseñada para repetir esa pregunta con el perfil completo. Este límite es una razón central para presentar el sistema como prototipo de investigación: un uso responsable exigiría tráfico etiquetado independiente, varios entornos y un análisis de seguridad antes de delegar decisiones con efectos reales (Apruzzese et al. 2022; Sommer et al. 2010).

El hecho de que la recompensa penalice más el falso negativo no autoriza a ignorar el falso positivo. En una red real, bloquear tráfico legítimo puede cortar sesiones de trabajo, provocar pérdida de disponibilidad y desplazar carga a operadores humanos. Por eso el prototipo no se conecta a un mecanismo de bloqueo activo y por eso las conclusiones se formulan en términos de inferencia offline. Antes de convertir una etiqueta BLOCK en una actuación, harían falta

umbrales de decisión revisables, registro de auditoría, posibilidad de intervención humana y evaluación específica del impacto operativo.

9.4 Reproducibilidad como práctica responsable

Por último, la disciplina de reproducibilidad descrita en la metodología es también una práctica ética: la investigación en seguridad con aprendizaje automático adolece con frecuencia de código ausente y evaluaciones irreproducibles, lo que dificulta verificar o refutar las afirmaciones publicadas (Olszewski et al. 2023; Arp et al. 2020). La persistencia de artefactos por ejecución, la partición de prueba fija con semilla 42 y su manifiesto de referencia permiten que cualquier tercero audite exactamente bajo qué condiciones se obtuvo cada cifra reportada en esta tesis. Hacer las limitaciones verificables, en lugar de presentarlas como una declaración de buenas intenciones, es la forma de honestidad metodológica que este trabajo persigue.

Esta práctica también protege contra una forma habitual de sobrepromesa: presentar como resultado lo que solo está diseñado. En esta memoria, la MAIN fresca, el día completo, la escalera de tamaños, las semillas 42–46, los cuatro holdouts, los RF nuevos y la Fase 2 fresca permanecen pendientes mientras no exista evidencia validada. Esa frontera es una obligación ética básica en un trabajo aplicado a seguridad, donde una afirmación exagerada podría inducir a confiar en un sistema fuera de su alcance real.

Conclusiones

Este capítulo ofrece una conclusión provisional pero utilizable: cierra lo que ya está sustentado por el diseño, la implementación y los artefactos históricos, y deja condicionadas las afirmaciones que dependen de la campaña final. No se presenta el protocolo aprobado como si fuese un resultado ejecutado.

10.1 Problema abordado y sistema construido

El trabajo aborda una pregunta concreta: cómo formular y evaluar, de manera auditable, un defensor offline que decida flujo a flujo entre **PERMIT** y **BLOCK** cuando los errores tienen costes distintos. La dificultad no reside solo en alcanzar una métrica alta. Un NIDS experimental debe evitar fugas, distinguir rendimiento intradistribución de generalización, conservar el preprocesamiento exacto y limitar su alcance frente a un benchmark conocido por su heterogeneidad y duplicados.

La respuesta construida comprende un cargador de CICIDS2017, un contrato canónico de 76 características y 76 indicadores de presencia, exclusión de identificadores susceptibles de fuga, escalado ajustado solo con entrenamiento, un entorno Gymnasium, una recompensa asimétrica, un agente QRDQN y

una línea base de Random Forest. El modelo y sus evaluaciones producen artefactos trazables, y la Fase 2 reutiliza el mismo contrato para inferencia offline sobre flujos de laboratorio. No se ha construido un cortafuegos autónomo ni un sistema de producción.

10.2 Contribución metodológica

La contribución más sólida es el diseño experimental reproducible. El perfil `main-v1` congela la configuración científica; `split_seed` y `model_seed` separan la partición de la aleatoriedad del aprendizaje; la caché conserva datos canónicos sin escalar; cada ejecución ajusta su propio escalador solo con entrenamiento; y el esquema de artefactos registra configuración, métricas, entorno, tiempos, monitorización, modelo, preprocesamiento, manifiesto y sumas de comprobación. La campaña final organiza estas piezas en contrastes que responden a preguntas distintas, no en una colección de ejecuciones aisladas.

También es una contribución declarar con precisión la naturaleza del problema. Con $\gamma = 0$, QRDQN actúa como un bandido contextual sensible al coste. Esta elección permite estudiar una política de decisión y su distribución de retornos, pero no atribuir al agente planificación temporal ni consecuencias activas sobre una red. La honestidad de esta interpretación mejora la comparabilidad con modelos supervisados y evita exagerar el alcance de RL.

10.3 Qué respalda la evidencia disponible

Los artefactos históricos respaldan tres conclusiones provisionales. Primero, el pipeline puede aprender la tarea binaria bajo una partición aleatoria de CICIDS2017 y conservar todo lo necesario para auditar esa evaluación. Segundo, los controles históricos con etiquetas barajadas, validación directa y análisis de

duplicados justifican que el rendimiento aleatorio se interprete como comprobación interna y no como prueba suficiente de generalización. Tercero, Random Forest es una referencia fuerte: puede superar a QRDQN en distribución y, por tanto, impide defender que RL sea preferible por definición.

La inferencia histórica de Fase 2 respalda una conclusión de ingeniería limitada: el modelo y el preprocesamiento persistidos pudieron aplicarse a una captura concreta externa al CSV de entrenamiento. No respalda robustez frente a producción, adversarios independientes ni otros entornos.

10.4 Qué no respalda todavía

La evidencia disponible no permite afirmar estabilidad entre semillas, eficiencia de datos, robustez del perfil completo ante la separación por días, comportamiento consistente en los cuatro escenarios retenidos, superioridad de QRDQN frente a RF ni transferencia final de la MAIN fresca. Tampoco permite elegir un umbral operativo, estimar impacto real de bloqueos o declarar preparación para despliegue. Los intervalos *bootstrap* históricos miden precisión sobre un test fijo, no varianza de entrenamiento.

Estas ausencias no son meras tareas editoriales. Definen el límite actual de la conclusión científica. Por ese motivo el resumen no contiene una cifra final y los capítulos de resultados y discusión conservan estructuras de inserción, no valores provisionales.

10.5 Por qué la partición aleatoria no basta

Una partición aleatoria mezcla filas procedentes de días y escenarios comunes y puede situar duplicados exactos a ambos lados de la frontera. Sirve para comprobar que el pipeline aprende y para comparar modelos bajo una

distribución controlada. No exige enfrentarse a una captura distinta. La partición completa por días rompe parte de esa continuidad; los *holdouts* aíslan escenarios específicos; la sensibilidad a semilla separa un resultado puntual de una tendencia local; y la escalera de tamaños muestra si el comportamiento depende del volumen y del presupuesto de entrenamiento. Solo la lectura conjunta de estos contrastes puede sostener una conclusión de generalización proporcionada a la evidencia.

10.6 Papel de Random Forest

Random Forest cumple una función interpretativa central. En una tarea tabular de un paso, obliga a demostrar que QRDQN aporta algo más que un clasificador complejo aplicado a buenas características. La comparación final será emparejada por partición, tamaño y escenario. Si ambos modelos siguen el mismo patrón, la explicación más probable residirá en el conjunto de datos o en el protocolo. Si difieren de forma reproducible, podrá discutirse una propiedad específica del aprendizaje o de la política sensible al coste. En ningún caso una sola partición justificará una afirmación universal.

10.7 Aprendizajes sobre RL con un conjunto de datos como entorno

El trabajo muestra que un conjunto etiquetado puede envolverse como entorno RL de forma reproducible, pero que esta transformación no crea por sí misma dinámica secuencial. La recompensa obliga a expresar preferencias entre falsos positivos y falsos negativos; el agente permite estudiar una política distribucional; y los artefactos de RL hacen observable el entrenamiento. Aun

así, el valor científico depende de la calidad de la partición, la ausencia de fugas y la comparación supervisada, igual que en clasificación. El principal aprendizaje es que la formulación RL debe evaluarse con más controles, no con menos.

10.8 Cuestiones reservadas a la campaña final

La campaña está diseñada para resolver cinco cuestiones pendientes: si una MAIN fresca reproduce el comportamiento intradistribución; si este persiste con separación completa por días; cómo cambia con el tamaño de entrenamiento; cuánto varía entre semillas de modelo; y qué escenarios retenidos provocan degradación. Los RF emparejados determinarán si los patrones son específicos de QRDQN, mientras que los controles directos, *bootstrap*, duplicados y etiquetas barajadas reforzarán la validez interna. La Fase 2 fresca cerrará la cadena con el modelo final.

Evidencia final pendiente. La versión de entrega deberá sustituir esta nota por una síntesis breve, derivada de los artefactos finales, que responda a las cinco cuestiones anteriores y mantenga explícitas las amenazas que no queden resueltas.

10.9 Cierre provisional

Antes de disponer de la campaña final, la conclusión defendible no es que QRDQN haya resuelto la detección de intrusiones ni que supere a un clasificador supervisado. Es que el proyecto ha convertido una propuesta de defensor basado en flujos en un método verificable: define qué observa, qué decide, qué coste optimiza, cómo evita fugas, contra qué se compara y qué evidencia necesita para hablar de generalización. Esa estructura permite que los resultados finales, sean

favorables o adversos, puedan incorporarse sin cambiar la pregunta científica ni ocultar sus límites.

Como trabajo futuro posterior a la campaña quedan la validación con capturas independientes de varios entornos, más repeticiones y familias de modelo, tratamiento de tráfico desconocido y una evaluación de seguridad específica antes de considerar acciones reales. Ninguna de estas extensiones puede deducirse automáticamente de CICIDS2017 ni de un laboratorio cerrado.

Bibliografía

- [1] Amrin Maria Khan Adawadkar y Nilima Kulkarni. «Cyber-Security and Reinforcement Learning: A Brief Survey». En: *Engineering Applications of Artificial Intelligence* 114 (2022), pág. 105116. DOI: [10.1016/j.engappai.2022.105116](https://doi.org/10.1016/j.engappai.2022.105116) (vid. pág. 47).
- [2] Zeeshan Ahmad, Adnan Shahid Khan, Cheah Wai Shiang, Johari Abdullah y Farhan Ahmad. «Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches». En: *Transactions on Emerging Telecommunications Technologies* 32.1 (2021), e4150. DOI: [10.1002/ett.4150](https://doi.org/10.1002/ett.4150) (vid. pág. 41).
- [3] Hooman Alavizadeh, Hootan Alavizadeh y Julian Jang-Jaccard. «Deep Q-Learning Based Reinforcement Learning Approach for Network Intrusion Detection». En: *Computers* 11.3 (2022), pág. 41. DOI: [10.3390/computers11030041](https://doi.org/10.3390/computers11030041). URL: <https://doi.org/10.3390/computers11030041> (vid. pág. 48).
- [4] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood y Adnan Anwar. «TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems». En: *IEEE Access* 8 (2020), págs. 165130-165150. DOI: [10.1109/ACCESS.2020.3022862](https://doi.org/10.1109/ACCESS.2020.3022862) (vid. págs. 37, 38).

- [5] Giovanni Apruzzese, Luca Pajola y Mauro Conti. «The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems». En: *IEEE Transactions on Network and Service Management* 19.4 (2022), págs. 5152-5169. DOI: [10.1109/TNSM.2022.3157344](https://doi.org/10.1109/TNSM.2022.3157344) (vid. págs. 4, 17, 20, 36, 41, 56, 121).
- [6] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro y Konrad Rieck. «Dos and Don'ts of Machine Learning in Computer Security». En: (2020). arXiv: [2010.09470 \[cs.CR\]](https://arxiv.org/abs/2010.09470) (vid. págs. 4, 18, 36, 42, 52, 61, 66, 122).
- [7] Momand Asadullah, Jan Sana Ullah y Naeem Ramzan. «A Systematic and Comprehensive Survey of Recent Advances in Intrusion Detection Systems Using Machine Learning: Deep Learning, Datasets, and Attack Taxonomy». En: *Expert Systems* (2023). DOI: [10.1155/2023/6048087](https://doi.org/10.1155/2023/6048087) (vid. pág. 42).
- [8] Stefan Axelsson. «The Base-Rate Fallacy and the Difficulty of Intrusion Detection». En: *ACM Transactions on Information and System Security* 3.3 (2000), págs. 186-205. DOI: [10.1145/357802.357804](https://doi.org/10.1145/357802.357804) (vid. págs. 2, 53, 55).
- [9] Marc G. Bellemare, Will Dabney y Rémi Munos. «A Distributional Perspective on Reinforcement Learning». En: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. 2017, págs. 449-458. URL: <https://proceedings.mlr.press/v70/bellemare17a.html> (vid. págs. 46, 78, 80).
- [10] Akram Boukhamla y Javier Coronel Gavero. «CICIDS2017 Dataset: Performance Improvements and Validation as a Robust Intrusion Detection System Testbed». En: *International Journal of Information and Computer Security* 16.1/2 (2021), págs. 20-32. DOI: [10.1504/IJICS.2021.117392](https://doi.org/10.1504/IJICS.2021.117392). URL: <https://www.inderscience.com/info/inarticle.php?artid=117392> (vid. pág. 40).

- [11] Guillermo Caminero, Manuel Lopez-Martin y Belen Carro. «Adversarial Environment Reinforcement Learning Algorithm for Intrusion Detection». En: *2019 International Joint Conference on Neural Networks (IJCNN)*. 2019, págs. 1-8. DOI: [10.1109/IJCNN.2019.8852380](https://doi.org/10.1109/IJCNN.2019.8852380) (vid. pág. 48).
- [12] Canadian Institute for Cybersecurity. *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. Dataset documentation page. 2017. URL: <https://www.unb.ca/cic/datasets/ids-2017.html> (vid. págs. 6, 34, 39, 120).
- [13] Marco Cantone, Claudio Marrocco y Alessandro Bria. «Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study». En: *IEEE Access* 12 (2024), págs. 144489-144508. DOI: [10.1109/ACCESS.2024.3472907](https://doi.org/10.1109/ACCESS.2024.3472907). URL: <https://doi.org/10.1109/ACCESS.2024.3472907> (vid. pág. 52).
- [14] Alvaro A. Cárdenas, John S. Baras y Karl Seamon. «A Framework for the Evaluation of Intrusion Detection Systems». En: *2006 IEEE Symposium on Security and Privacy (S&P)*. 2006, págs. 63-77. DOI: [10.1109/SP.2006.2](https://doi.org/10.1109/SP.2006.2) (vid. págs. 6, 15, 50, 55, 73).
- [15] Center for Security and Emerging Technology y The Alan Turing Institute. *Autonomous Cyber Defense*. <https://cset.georgetown.edu/publication/autonomous-cyber-defense/>. 2023 (vid. pág. 48).
- [16] Jesús F. Cevallos M., Alessandra Rizzardi, Sabrina Sicari y Alberto Coen Porisini. «Deep Reinforcement Learning for Intrusion Detection in Internet of Things: Best Practices, Lessons Learnt, and Open Challenges». En: *Computer Networks* 236 (2023), pág. 110016. DOI: [10.1016/j.comnet.2023.110016](https://doi.org/10.1016/j.comnet.2023.110016). URL: <https://doi.org/10.1016/j.comnet.2023.110016> (vid. pág. 49).
- [17] Communications Security Establishment y Canadian Institute for Cybersecurity. *A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018)*. 2018.

- URL: <https://registry.opendata.aws/cse-cic-ids2018/> (vid. págs. 37, 38).
- [18] Will Dabney, Mark Rowland, Marc G. Bellemare y Rémi Munos. «Distributional Reinforcement Learning with Quantile Regression». En: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, págs. 2892-2901. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/11791> (vid. págs. 3, 6, 16, 46, 78, 79).
- [19] Davide Di Monda, Antonio Montieri, Valerio Persico, Pasquale Voria, Matteo De Ieso y Antonio Pescapè. «Few-Shot Class-Incremental Learning for Network Intrusion Detection Systems». En: *IEEE Open Journal of the Communications Society* 5 (2024), págs. 6736-6757. DOI: [10.1109/OJCOMS.2024.3481895](https://doi.org/10.1109/OJCOMS.2024.3481895). URL: <https://doi.org/10.1109/OJCOMS.2024.3481895> (vid. pág. 56).
- [20] Rohit Dube. «Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research». En: *Journal of Computer Virology and Hacking Techniques* 20.1 (2024), págs. 203-211. DOI: [10.1007/s11416-023-00509-7](https://doi.org/10.1007/s11416-023-00509-7) (vid. págs. 40, 53).
- [21] Gints Engelen, Vera Rimmer y Wouter Joosen. «Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study». En: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, págs. 7-12. DOI: [10.1109/SPW53761.2021.00009](https://doi.org/10.1109/SPW53761.2021.00009) (vid. págs. 3, 7, 38, 39, 53, 61).
- [22] Farama Foundation. *Gymnasium Documentation*. Software documentation; access date 2026. 2023. URL: <https://gymnasium.farama.org/> (vid. págs. 6, 15, 47, 72, 84).
- [23] Yasir Ali Farrukh, Irfan Khan, Syed Wali, David Bierbrauer, Nathaniel D. Bastian y John Pavlik. «Payload-Byte: A Tool for Extracting and Labeling Packet Capture Files of Modern Network Intrusion Detection Datasets». En: *TechRxiv* (sep. de 2022). Preprint. DOI: [10.36227/](https://doi.org/10.36227/)

- [techrxiv.20714221.v1](https://doi.org/10.36227/techrxiv.20714221.v1). URL: <https://doi.org/10.36227/techrxiv.20714221.v1> (vid. pág. 34).
- [24] Tom Fawcett. «An Introduction to ROC Analysis». En: *Pattern Recognition Letters* 27.8 (2006), págs. 861-874. DOI: [10.1016/j.patrec.2005.10.010](https://doi.org/10.1016/j.patrec.2005.10.010) (vid. pág. 55).
- [25] Scott Fujimoto, David Meger y Doina Precup. «Off-Policy Deep Reinforcement Learning without Exploration». En: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. 2019, págs. 2052-2062. URL: <https://proceedings.mlr.press/v97/fujimoto19a.html> (vid. pág. 45).
- [26] Patrik Goldschmidt y Daniela Chudá. «Network Intrusion Datasets: A Survey, Limitations, and Recommendations». En: *Computers & Security* 156 (2025), pág. 104510. DOI: [10.1016/j.cose.2025.104510](https://doi.org/10.1016/j.cose.2025.104510). URL: <https://doi.org/10.1016/j.cose.2025.104510> (vid. pág. 36).
- [27] Afrah Gueriani, Hamza Kheddar y Ahmed Cherif Mazari. «Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey». En: (2024). Preprint; published version in IC2EM proceedings 2023. arXiv: [2405.20038](https://arxiv.org/abs/2405.20038) [cs.CR] (vid. págs. 30, 47).
- [28] Arash Habibi Lashkari, Gerard Draper Gil, Mohammad Saiful Islam Mamun y Ali A. Ghorbani. «Characterization of Tor Traffic Using Time Based Features». En: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*. 2017, págs. 253-262. DOI: [10.5220/0006105602530262](https://doi.org/10.5220/0006105602530262) (vid. págs. 34, 39).
- [29] Hado van Hasselt, Arthur Guez y David Silver. «Deep Reinforcement Learning with Double Q-Learning». En: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. 2016, págs. 2094-2100. arXiv: [1509.06461](https://arxiv.org/abs/1509.06461) (vid. pág. 46).

- [30] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar y David Silver. «Rainbow: Combining Improvements in Deep Reinforcement Learning». En: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018, págs. 3215-3222. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/11796> (vid. págs. 46, 56).
- [31] Md. Alamgir Hossain. «Deep Q-Learning Intrusion Detection System (DQ-IDS): A Novel Reinforcement Learning Approach for Adaptive and Self-Learning Cybersecurity». En: *ICT Express* 11.5 (2025), págs. 875-880. DOI: [10.1016/j.icte.2025.05.007](https://doi.org/10.1016/j.icte.2025.05.007). URL: <https://doi.org/10.1016/j.icte.2025.05.007> (vid. pág. 48).
- [32] Zhisheng Hu, Minghui Zhu y Peng Liu. «Adaptive Cyber Defense Against Multi-Stage Attacks Using Learning-Based POMDP». En: *ACM Transactions on Privacy and Security* 24.1 (2020), págs. 1-31. DOI: [10.1145/3418897](https://doi.org/10.1145/3418897) (vid. pág. 48).
- [33] Marcin Iwanowski, Dominik Olszewski, Waldemar Graniszewski, Jacek Krupski y Franciszek Pelc. «The Choice of Training Data and the Generalizability of Machine Learning Models for Network Intrusion Detection Systems». En: *Applied Sciences* 15.15 (2025), pág. 8466. DOI: [10.3390/app15158466](https://doi.org/10.3390/app15158466). URL: <https://www.mdpi.com/2076-3417/15/15/8466> (vid. pág. 52).
- [34] R. Kanimozhi y P. S. Ramesh. «Deep Reinforcement Learning-Based Intrusion Detection Scheme for Software-Defined Networking». En: *Scientific Reports* 15.1 (2025), pág. 38827. DOI: [10.1038/s41598-025-24869-w](https://doi.org/10.1038/s41598-025-24869-w). URL: <https://www.nature.com/articles/s41598-025-24869-w> (vid. pág. 49).
- [35] Shachar Kaufman, Saharon Rosset y Claudia Perlich. «Leakage in Data Mining: Formulation, Detection, and Avoidance». En: *Proceedings of the*

- 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2011, págs. 556-563. DOI: [10.1145/2020408.2020496](https://doi.org/10.1145/2020408.2020496) (vid. págs. 4, 18, 36, 52, 66).
- [36] KDD Cup. *KDD Cup 1999 Data*. Dataset page. 1999. URL: <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html> (vid. págs. 36, 38).
- [37] Hamza Kheddar, Diana W. Dawoud, Ali Ismail Awad, Yassine Himeur y Muhammad Khurram Khan. «Reinforcement-Learning-Based Intrusion Detection in Communication Networks: A Review». En: *IEEE Communications Surveys & Tutorials* 27.4 (2025), págs. 2420-2469. DOI: [10.1109/COMST.2024.3484491](https://doi.org/10.1109/COMST.2024.3484491) (vid. págs. 30, 47, 56, 58).
- [38] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova y Benjamin Turnbull. «Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset». En: *Future Generation Computer Systems* 100 (2019), págs. 779-796. DOI: [10.1016/j.future.2019.05.041](https://doi.org/10.1016/j.future.2019.05.041) (vid. págs. 37, 38).
- [39] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé y Éric Totel. «Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes». En: *Risks and Security of Internet and Systems*. Vol. 13857. Lecture Notes in Computer Science. Springer, 2023, págs. 19-34. DOI: [10.1007/978-3-031-31108-6_2](https://doi.org/10.1007/978-3-031-31108-6_2) (vid. págs. 3, 38, 39, 53, 61).
- [40] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé y Éric Totel. «Faulty Use of the CIC-IDS 2017 Dataset in Information Security Research». En: *Risks and Security of Internet and Systems*. Vol. 13561. Lecture Notes in Computer Science. Earlier presentation of dataset-critique work; see also Lanvin2023Errors. Springer, 2022 (vid. pág. 39).

- [41] Siamak Layeghy y Marius Portmann. «Explainable Cross-Domain Evaluation of ML-Based Network Intrusion Detection Systems». En: *Computers and Electrical Engineering* 108 (2023), pág. 108692. DOI: [10.1016/j.compeleceng.2023.108692](https://doi.org/10.1016/j.compeleceng.2023.108692) (vid. págs. 20, 36, 52).
- [42] Siamak Layeghy y Marius Portmann. «On Generalisability of Machine Learning-Based Network Intrusion Detection Systems». En: (2022). DOI: [10.48550/arXiv.2205.04112](https://doi.org/10.48550/arXiv.2205.04112). arXiv: [2205.04112 \[cs.NI\]](https://arxiv.org/abs/2205.04112). URL: <https://arxiv.org/abs/2205.04112> (vid. pág. 52).
- [43] Wenke Lee, Wei Fan, Matthew Miller, Salvatore J. Stolfo y Erez Zadok. «Toward Cost-Sensitive Modeling for Intrusion Detection and Response». En: *Journal of Computer Security* 10.1–2 (2002), págs. 5-22. DOI: [10.3233/JCS-2002-101-202](https://doi.org/10.3233/JCS-2002-101-202) (vid. págs. 3, 15, 50, 73).
- [44] Sergey Levine, Aviral Kumar, George Tucker y Justin Fu. «Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems». En: *arXiv preprint arXiv:2005.01643* (2020). Preprint. arXiv: [2005.01643 \[cs.LG\]](https://arxiv.org/abs/2005.01643) (vid. págs. 5, 45, 78).
- [45] Yantao Li y Wenhui Ding. «Optimized Network Security Attack Detection Algorithm Based on Deep Deterministic Policy Gradient (DDPG)». En: *2025 International Conference on Artificial Intelligence and Digital Ethics (ICAIDE)*. 2025, págs. 214-217. DOI: [10.1109/ICAIDE65466.2025.11189334](https://doi.org/10.1109/ICAIDE65466.2025.11189334). URL: <https://doi.org/10.1109/ICAIDE65466.2025.11189334> (vid. pág. 49).
- [46] Zhengfa Li, Chuanhe Huang, Shuhua Deng, Wanyu Qiu y Xieping Gao. «A Soft Actor-Critic Reinforcement Learning Algorithm for Network Intrusion Detection». En: *Computers & Security* 135 (2023), pág. 103502. DOI: [10.1016/j.cose.2023.103502](https://doi.org/10.1016/j.cose.2023.103502). URL: <https://doi.org/10.1016/j.cose.2023.103502> (vid. pág. 48).

- [47] Long-Ji Lin. «Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching». En: *Machine Learning* 8.3–4 (1992), págs. 293–321. DOI: [10.1007/BF00992699](https://doi.org/10.1007/BF00992699) (vid. pág. 44).
- [48] Hongyu Liu y Bo Lang. «Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey». En: *Applied Sciences* 9.20 (2019), pág. 4396. DOI: [10.3390/app9204396](https://doi.org/10.3390/app9204396) (vid. págs. 4, 17, 41, 42, 58).
- [49] Lisa Liu, Gints Engelen, Timothy M. Lynar, Daryl Essam y Wouter Joosen. «Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018». En: *2022 IEEE Conference on Communications and Network Security (CNS)*. 2022, págs. 254–262. DOI: [10.1109/CNS56114.2022.9947235](https://doi.org/10.1109/CNS56114.2022.9947235) (vid. págs. 38, 39, 53).
- [50] Manuel Lopez-Martin, Belén Carro y Antonio Sanchez-Esguevillas. «Application of Deep Reinforcement Learning to Intrusion Detection for Supervised Problems». En: *Expert Systems with Applications* 141 (2020), pág. 112963. DOI: [10.1016/j.eswa.2019.112963](https://doi.org/10.1016/j.eswa.2019.112963) (vid. pág. 47).
- [51] Manuel Lopez-Martin, Antonio Sanchez-Esguevillas, Juan I. Arribas y Belén Carro. «Network Intrusion Detection Based on Extended RBF Neural Network With Offline Reinforcement Learning». En: *IEEE Access* 9 (2021), págs. 153153–153170. DOI: [10.1109/ACCESS.2021.3127689](https://doi.org/10.1109/ACCESS.2021.3127689) (vid. pág. 47).
- [52] Malika Malik y Kamaljit Singh Saini. «Network Intrusion Detection System Using Reinforcement Learning Techniques». En: *2023 International Conference on Circuit Power and Computing Technologies (ICCPCT)*. 2023, págs. 1642–1649. DOI: [10.1109/ICCPCT58313.2023.10245608](https://doi.org/10.1109/ICCPCT58313.2023.10245608). URL: <https://doi.org/10.1109/ICCPCT58313.2023.10245608> (vid. pág. 49).

- [53] Malika Malik y Kamaljit Singh Saini. «Network Intrusion Detection System using Reinforcement learning». En: *2023 4th International Conference for Emerging Technology (INCET)*. 2023, págs. 1-4. DOI: [10.1109/INCET57972.2023.10170630](https://doi.org/10.1109/INCET57972.2023.10170630). URL: <https://doi.org/10.1109/INCET57972.2023.10170630> (vid. pág. 49).
- [54] Aleksandar Milenkoski, Marco Vieira, Samuel Kounev, Alberto Avritzer y Bryan D. Payne. «Evaluating Computer Intrusion Detection Systems: A Survey of Common Practices». En: *ACM Computing Surveys* 48.1 (2015), 12:1-12:41. DOI: [10.1145/2808691](https://doi.org/10.1145/2808691) (vid. pág. 55).
- [55] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg y Demis Hassabis. «Human-Level Control through Deep Reinforcement Learning». En: *Nature* 518.7540 (2015), págs. 529-533. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236) (vid. págs. 45, 78, 81).
- [56] Nour Moustafa, Mohiuddin Ahmed y Sherif Ahmed. «Data Analytics-Enabled Intrusion Detection: Evaluations of ToN_IoT Linux Datasets». En: *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 2020, págs. 727-735. DOI: [10.1109/TrustCom50675.2020.00080](https://doi.org/10.1109/TrustCom50675.2020.00080) (vid. pág. 37).
- [57] Nour Moustafa y Jill Slay. «The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set». En: *Information Security Journal: A Global Perspective* 25.1-3 (2016), págs. 18-31. DOI: [10.1080/19393555.2015.1125974](https://doi.org/10.1080/19393555.2015.1125974) (vid. pág. 37).
- [58] Nour Moustafa y Jill Slay. «UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)». En: *2015 Military Communications and Information Systems Conference*

- (*MilCIS*). 2015, págs. 1-6. DOI: [10.1109/MilCIS.2015.7348942](https://doi.org/10.1109/MilCIS.2015.7348942) (vid. págs. 37, 38).
- [59] Loc Gia Nguyen y Kohei Watabe. «Flow-Based Network Intrusion Detection Based on BERT Masked Language Model». En: *CoNEXT 2022 Student Workshop*. 2022. DOI: [10.1145/3565477.3569152](https://doi.org/10.1145/3565477.3569152). arXiv: [2306.04920](https://arxiv.org/abs/2306.04920) (vid. pág. 42).
- [60] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler y Patrick Traynor. «“Get in Researchers; We’re Measuring Reproducibility”: A Reproducibility Study of Machine Learning Papers in Tier 1 Security Conferences». En: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, págs. 3433-3459. DOI: [10.1145/3576915.3623130](https://doi.org/10.1145/3576915.3623130) (vid. págs. 55, 122).
- [61] Akinyemi Oyelakin, A. O. Ameen, T. S. Ogundele, T. Salau-Ibrahim, U. T. Abdulrauf, H. I. Olufadi, I. K. Ajiboye, S. Muhammad-Thani e I. A. Adeniji. «Overview and Exploratory Analyses of CICIDS 2017 Intrusion Detection Dataset». En: *Journal of Systems Engineering and Information Technology (JOSEIT)* 2.2 (2023), págs. 45-52. DOI: [10.29207/joseit.v2i2.5411](https://doi.org/10.29207/joseit.v2i2.5411). URL: <https://jurnal.iaii.or.id/index.php/JOSEIT/article/view/5411> (vid. pág. 39).
- [62] Atilla Özgür y Hamit Erdem. «A Review of KDD99 Dataset Usage in Intrusion Detection and Machine Learning Between 2010 and 2015». En: *PeerJ Preprints* 4 (2016). Preprint, e1954v1. DOI: [10.7287/peerj.preprints.1954v1](https://doi.org/10.7287/peerj.preprints.1954v1). URL: <https://peerj.com/preprints/1954/> (vid. pág. 36).
- [63] Gregory Palmer, Chris Parry, Daniel J. B. Harrold y Chris Willis. «Deep Reinforcement Learning for Autonomous Cyber Defence: A Survey». En: (2023). arXiv: [2310.07745](https://arxiv.org/abs/2310.07745) [cs.CR] (vid. pág. 48).

- [64] Adrian Pekar y Richard Jozsa. «Evaluating ML-Based Anomaly Detection Across Datasets of Varied Integrity: A Case Study». En: *Computer Networks* 251 (2024), pág. 110617. DOI: [10.1016/j.comnet.2024.110617](https://doi.org/10.1016/j.comnet.2024.110617). URL: <https://doi.org/10.1016/j.comnet.2024.110617> (vid. pág. 39).
- [65] Kezhou Ren, Jinshu Zhu, Tao Yuan, Sheng Wen y Frank Jiang. «ID-RDRL: A Deep Reinforcement Learning-Based Feature Selection Intrusion Detection Model». En: *Scientific Reports* 12 (2022), pág. 15370. DOI: [10.1038/s41598-022-19366-3](https://doi.org/10.1038/s41598-022-19366-3) (vid. pág. 48).
- [66] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes y Andreas Hotho. «A Survey of Network-Based Intrusion Detection Data Sets». En: *Computers & Security* 86 (2019), págs. 147-167. DOI: [10.1016/j.cose.2019.06.005](https://doi.org/10.1016/j.cose.2019.06.005). arXiv: [1903.02460](https://arxiv.org/abs/1903.02460) (vid. págs. 3, 19, 32, 36, 38, 42, 56, 58).
- [67] María Rodríguez, Álvaro Alesanco, Lorena Mehavilla y José García. «Evaluation of Machine Learning Techniques for Traffic Flow-Based Intrusion Detection». En: *Sensors* 22.23 (2022), pág. 9326. DOI: [10.3390/s22239326](https://doi.org/10.3390/s22239326). URL: <https://www.mdpi.com/1424-8220/22/23/9326> (vid. pág. 41).
- [68] Arnaud Rosay, Eloïse Cheval, Florent Carlier y Pascal Leroux. «Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017». En: *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP)*. 2022, págs. 25-36. DOI: [10.5220/0010774000003120](https://doi.org/10.5220/0010774000003120) (vid. págs. 39, 53).
- [69] Takaya Saito y Marc Rehmsmeier. «The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets». En: *PLOS ONE* 10.3 (2015), e0118432. DOI: [10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432) (vid. pág. 55).

- [70] Hamed T. Sanusi. «Network Intrusion Detection Using Deep Reinforcement Learning». Tesis de maestría. Statesboro, Georgia, USA: Georgia Southern University, 2023. URL: <https://digitalcommons.georgiasouthern.edu/etd/2676/> (vid. pág. 49).
- [71] Mohanad Sarhan, Siamak Layeghy y Marius Portmann. «Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection». En: (2021). Proposes standardised NetFlow-aligned feature mapping across CIC and UNSW-NB15 datasets. arXiv: [2104.07183 \[cs.CR\]](https://arxiv.org/abs/2104.07183) (vid. págs. 35, 43, 67).
- [72] Habeeb Mohammed Sayeeduddin y T. Ranga Babu. «Network Intrusion Detection System: A Survey on Artificial Intelligence-Based Techniques». En: *Expert Systems* (2022). DOI: [10.1111/exsy.13066](https://doi.org/10.1111/exsy.13066) (vid. pág. 42).
- [73] Karen A. Scarfone y Peter M. Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Inf. téc. Special Publication 800-94. National Institute of Standards y Technology, 2007. URL: <https://csrc.nist.gov/pubs/sp/800/94/final> (vid. págs. 1, 31, 32, 55).
- [74] Tom Schaul, John Quan, Ioannis Antonoglou y David Silver. «Prioritized Experience Replay». En: *International Conference on Learning Representations*. 2016. arXiv: [1511.05952](https://arxiv.org/abs/1511.05952) (vid. pág. 46).
- [75] Jamshed Ali Shaikh, Chengliang Wang, Muhammad Wajeeh Us Sima, Muhammad Arshad, Muhammad Owais, Dina S. M. Hassan, Reem Alkanhel y Mohammed Saleh Ali Muthanna. «A Deep Reinforcement Learning-Based Robust Intrusion Detection System for Securing IoMT Healthcare Networks». En: *Frontiers in Medicine* 12 (2025), pág. 1524286. DOI: [10.3389/fmed.2025.1524286](https://doi.org/10.3389/fmed.2025.1524286). URL: <https://www.frontiersin.org/journals/medicine/articles/10.3389/fmed.2025.1524286/full> (vid. pág. 49).

- [76] Iman Sharafaldin, Arash Habibi Lashkari y Ali A. Ghorbani. «A Detailed Analysis of the CICIDS2017 Data Set». En: *Information Systems Security and Privacy*. Vol. 977. Communications in Computer and Information Science. Springer, 2019, págs. 172-188. DOI: [10.1007/978-3-030-25109-3_9](https://doi.org/10.1007/978-3-030-25109-3_9) (vid. pág. 39).
- [77] Iman Sharafaldin, Arash Habibi Lashkari y Ali A. Ghorbani. «Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization». En: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, págs. 108-116. DOI: [10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116). URL: <https://www.unb.ca/cic/datasets/ids-2017.html> (vid. págs. 6, 34, 37-39, 120).
- [78] Robin Sommer y Vern Paxson. «Outside the Closed World: On Using Machine Learning for Network Intrusion Detection». En: *2010 IEEE Symposium on Security and Privacy (S&P)*. 2010, págs. 305-316. DOI: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25) (vid. págs. 32, 52, 121).
- [79] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras y Burkhard Stiller. «An Overview of IP Flow-Based Intrusion Detection». En: *IEEE Communications Surveys & Tutorials* 12.3 (2010), págs. 343-356. DOI: [10.1109/SURV.2010.032210.00054](https://doi.org/10.1109/SURV.2010.032210.00054) (vid. págs. 2, 13, 34, 35, 63, 64).
- [80] Stable-Baselines3 Contributors. *QR-DQN: Stable Baselines3 Contrib Documentation*. Software documentation; access date 2026. 2023. URL: <https://sb3-contrib.readthedocs.io/en/master/modules/qrdqn.html> (vid. págs. 6, 16, 47, 82, 84).
- [81] Maxwell Standen, Martin Lucas, David Bowman, Toby J. Richer, Junae Kim y Damian Marriott. «CybORG: A Gym for the Development of Autonomous Cyber Agents». En: *arXiv preprint arXiv:2108.09118* (2021). Preprint; also cited as an IJCAI-21 Adaptive Cyber Defense workshop paper. arXiv: [2108.09118](https://arxiv.org/abs/2108.09118) [cs.CR] (vid. pág. 48).

- [82] Richard S. Sutton y Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2.^a ed. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html> (vid. págs. 3, 43, 56, 78).
- [83] Mahbod Tavallaei, Ebrahim Bagheri, Wei Lu y Ali A. Ghorbani. «A Detailed Analysis of the KDD CUP 99 Data Set». En: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. 2009, págs. 1-6. DOI: [10.1109/CISDA.2009.5356528](https://doi.org/10.1109/CISDA.2009.5356528) (vid. págs. 36, 38).
- [84] Ashish Thakkar y Rachna Lohiya. «A Review of the Advancement in Intrusion Detection Datasets». En: *Procedia Computer Science* 167 (2020), págs. 636-645. DOI: [10.1016/j.procs.2020.03.270](https://doi.org/10.1016/j.procs.2020.03.270) (vid. pág. 36).
- [85] Muhammad Fahad Umer, Muhammad Sher y Yaxin Bi. «Flow-Based Intrusion Detection: Techniques and Challenges». En: *Computers & Security* 70 (2017), págs. 238-254. DOI: [10.1016/j.cose.2017.05.009](https://doi.org/10.1016/j.cose.2017.05.009) (vid. págs. 2, 13, 34, 35, 63, 64).
- [86] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot y Nando de Freitas. «Dueling Network Architectures for Deep Reinforcement Learning». En: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. Vol. 48. 2016, págs. 1995-2003. URL: <https://proceedings.mlr.press/v48/wangf16.html> (vid. pág. 46).
- [87] Christopher J. C. H. Watkins y Peter Dayan. «Q-Learning». En: *Machine Learning* 8.3-4 (1992), págs. 279-292. DOI: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698) (vid. pág. 44).
- [88] Jianping Xiao, Chun Long, Jing Zhao, Jinxia Wei, Anlei Hu y Guanyao Du. «A Survey on Network Intrusion Detection Based on Deep Learning». En: *Frontiers of Data and Computing* 3.3 (2021), págs. 59-74. DOI: [10.11871/jfdc.issn.2096-742X.2021.03.006](https://doi.org/10.11871/jfdc.issn.2096-742X.2021.03.006) (vid. pág. 42).

- [89] Bin Yang, Muhammad Haseeb Arshad y Qing Zhao. «Packet-Level and Flow-Level Network Intrusion Detection Based on Reinforcement Learning and Adversarial Training». En: *Algorithms* 15.12 (2022), pág. 453. DOI: [10.3390/a15120453](https://doi.org/10.3390/a15120453). URL: <https://www.mdpi.com/1999-4893/15/12/453> (vid. pág. 49).
- [90] Wanrong Yang, Alberto Acuto, Yihang Zhou y Dominik Wojtczak. «A Survey for Deep Reinforcement Learning Based Network Intrusion Detection». En: *Applied AI Letters* 7.2 (2026), e70026. DOI: [10.1002/ai12.70026](https://doi.org/10.1002/ai12.70026). arXiv: [2410.07612](https://arxiv.org/abs/2410.07612) [cs.CR] (vid. págs. 30, 47, 56, 58).

Manual de reproducibilidad

Este anexo resume el recorrido científico reproducible. La guía operativa mantenida es `docs/gpu_experimental_environment.md`; aquí se conservan únicamente las decisiones y órdenes necesarias para comprender cómo se obtendrá la evidencia. Ninguna orden de campaña se ejecutó durante la revisión de esta memoria.

A.1 Código, revisión y datos

La reproducción comienza desde un commit identificado del repositorio y una especificación de campaña sin modificar. Git LFS debe materializar los ocho CSV curados de CICIDS2017. Los hashes publicados en el README permiten comprobar que las fuentes coinciden con las esperadas; la captura de Fase 2 se valida mediante su ruta, tamaño y SHA-256 sin publicar metadatos sensibles.

Repositorio `git clone <URL> y git lfs pull.`

Especificación

`experiments/final_experiment_campaign.json.`

Perfil	<code>main-v1</code> , incluido su hash congelado.
Datos	ocho CSV de CICIDS2017 y entrada etiquetada de laboratorio, todos verificados antes de ejecutar.

Los datos originales, credenciales, cachés generadas y exportaciones voluminosas no se incorporan al historial normal de Git. La disponibilidad de un fichero local no lo convierte en evidencia: su hash debe quedar enlazado al preflight y a la ejecución que lo consume.

A.2 Entorno de software

La plataforma final utiliza Linux, Python 3.12 y un runtime CUDA compatible con el manifiesto GPU mantenido. Las dependencias directas de campaña se instalan desde `requirements-gpu-cu130.txt`; `pyproject.toml` y `uv.lock` definen desarrollo y pruebas. El fichero histórico de compatibilidad con nombre de proveedor no forma parte de la descripción científica final.

Las comprobaciones ligeras incluyen importaciones, pruebas unitarias, firma del perfil y dimensiones del esquema:

```
python -m pytest -q
```

Comprobación programática: `len(FEATURES_CANON) == 76`.

Estas comprobaciones no sustituyen la captura de hardware, controlador, CUDA, cuDNN y versiones reales en `environment.json`.

A.3 Caché canónica sin escalar

La caché se construye por CSV y conserva observaciones canónicas, etiquetas y metadatos antes del escalado. Nunca contiene un `StandardScaler`, una

asignación train/test, predicciones ni estado de modelo. La equivalencia entre la ruta con y sin caché se comprueba a nivel de arrays y etiquetas.

```
python scripts/build_cicids_cache.py build
--dataset-root <DATASET_ROOT>
--cache-root <CACHE_ROOT>

python scripts/build_cicids_cache.py validate
--dataset-root <DATASET_ROOT>
--cache-root <CACHE_ROOT>
```

La política oficial es `cache_policy=require`. Un fragmento ausente o obsoleto detiene la campaña; sustituirlo exige una reconstrucción explícita. Cada ejecución ensambla los fragmentos, selecciona su partición, ajusta un escalador nuevo solo con entrenamiento y persiste ese estado.

A.4 Preflight de la plataforma experimental

El preflight se ejecuta en el mismo entorno y con las mismas rutas que la campaña:

```
python scripts/preflight_gpu_environment.py
--dataset-root <DATASET_ROOT>
--cache-root <CACHE_ROOT>
--artifact-root runs/final_campaign
--snapshot-root <SNAPSHOT_ROOT>
--campaign-spec    experiments/final_experiment_campaign.
json
--output runs/final_campaign/preflight_report.json
```

La comprobación cubre CPU, RAM, GPU/VRAM, espacio, CUDA, datos y LFS, caché, entrada de Fase 2, escritura de artefactos y un smoke sintético no científico. Un informe ausente, fallido, obsoleto o incompatible con la caché

impide una ejecución real. El smoke solo valida el recorrido y el contrato de artefactos; no se incorpora a Resultados.

A.5 Campaña seca y ejecución controlada

Antes de entrenar se revisa una resolución seca de toda la matriz. La ejecución real usa exactamente el mismo identificador, rutas e informe y añade `-resume`. La campaña permanece secuencial aunque haya recursos libres.

```
python scripts/run_campaign.py \  
    experiments/final_experiment_campaign.json \  
    --campaign-id <CAMPAIGN_ID>  
    --artifact-root runs/final_campaign  
    --cache-root <CACHE_ROOT>  
    --snapshot-root <SNAPSHOT_ROOT>  
    --preflight-report <PREFLIGHT_JSON>  
    --dry-run
```

Después de revisar la salida, la misma orden se ejecuta sin `-dry-run` y con `-resume`. Las opciones `-stage` y `-run` permiten recuperar una parte concreta, pero no ejecutan dependencias ausentes de forma implícita. Una ejecución validada no se repite para reparar una exportación fallida.

A.6 Semillas y subconjuntos

Todos los puntos registran `split_seed` y `model_seed`; se usa `null` solo cuando la selección por nombre de fichero hace que una semilla no sea aplicable. La MAIN usa 42/42. La escalera mantiene 42/42 y subconjuntos anidados. El estudio de estabilidad fija `split_seed=42`, un millón de filas y 1 324 741 pasos,

y cambia `model_seed=42-46`. Los cuatro holdouts se seleccionan por nombre exacto y no forman un barrido exhaustivo.

Cada configuración conserva hashes de filas, etiquetas, fuentes, caché y perfil. Así se puede verificar que cambiar `model_seed` no alteró la partición y que un alias reutiliza exactamente la ejecución física declarada.

A.7 Artefactos por ejecución

Una ejecución primaria aceptable conserva como mínimo:

- configuración y resultados: `config.json` y `metrics.json`;
- entorno y tiempos: `environment.json`, `timings.json` y `run_summary.json`;
- monitorización y escalares TensorBoard exportados;
- modelo, escalador, percentiles y nombres de características;
- predicciones o evidencia suficiente para validación directa;
- `artifact_manifest.json` y sumas SHA-256.

Los trabajos auxiliares enlazan la MAIN física y los hashes exactos de los artefactos que consumen. El control de etiquetas barajadas permanece en un grupo separado y nunca se agrega como rendimiento MAIN. Los valores de una métrica no definida se conservan como nulos; no se sustituyen por cero.

A.8 Exportación y recuperación

Después de cada ejecución física validada, el runner copia el directorio completo al destino externo y produce un `tar.gz` verificado con su suma SHA-256. La exportación incluye ficheros ignorados o LFS presentes dentro del

directorio de ejecución. Los alias no generan copias nuevas y las exportaciones nunca mueven, eliminan ni mutan la fuente canónica.

Un snapshot o bundle de campaña es una comodidad adicional, no sustituto de la exportación por ejecución ni prueba de durabilidad independiente. El estado de campaña se escribe atómicamente para permitir reanudación tras interrupciones.

A.9 Agregación y generación de figuras

Solo cuando todas las ejecuciones, auxiliares y alias validan se construye el agregado:

```
python scripts/aggregate_campaign.py
  --campaign-dir runs/final_campaign/<CAMPAIGN_ID>
  --output-dir <AGGREGATE_DIR>

python scripts/generate_campaign_figures.py
  --aggregate-dir <AGGREGATE_DIR>
  --output-dir <FIGURE_STAGING_DIR>
```

El agregador rechaza campañas incompletas, corruptas, incompatibles o sustituidas con evidencia histórica. Los generadores de tesis se ejecutan primero en una ruta de staging fuera de `memoria/`; tras revisión de procedencia y diseño, los productos seleccionados se incorporan al documento. No existe un modo de generar resultados finales con datos de ejemplo.

A.10 Reproducción de la memoria

La construcción documental es independiente del runtime GPU:

```
cd memoria
latexmk -C
latexmk -pdf memoria.tex
```

La aceptación requiere además revisar advertencias, referencias, índices, tablas y páginas renderizadas. Una compilación con código de salida cero no acredita por sí sola la calidad visual.

Entorno hardware y software

Este anexo distingue tres objetos: el contrato de la plataforma GPU experimental, las observaciones que producirán los artefactos finales y el entorno histórico de evidencia previa. Los requisitos de lanzamiento no se presentan como hardware medido.

B.1 Contrato de la plataforma final

La campaña se ejecutará en una plataforma Linux con GPU CUDA y recursos suficientes para la matriz aprobada. El preflight usa umbrales de CPU, RAM, VRAM y espacio como puertas de lanzamiento configurables. Estos umbrales expresan capacidad requerida; no describen una máquina concreta.

El entorno final se instalará desde `requirements-gpu-cu130.txt`. La especificación científica conserva el perfil, semillas, datos, caché y artefactos independientemente del proveedor, hostname, cuenta, método de acceso o rutas de montaje. La ejecución permanece secuencial para reducir competencia de recursos y simplificar recuperación.

B.2 Evidencia ambiental pendiente

Cada ejecución aceptada generará `environment.json`. La tabla ambiental final de la tesis se poblará desde esos ficheros y deberá incluir, al menos:

Campo	Fuente final
Sistema y arquitectura	<code>environment.json</code> : plataforma, kernel y arquitectura
CPU y memoria	inventario observado durante la ejecución
GPU y VRAM	dispositivo CUDA visible y memoria registrada
Controlador, CUDA y cuDNN	runtime observado, no versión supuesta por el manifiesto
Python y paquetes principales	versiones importadas durante la ejecución
Commit y estado del árbol	metadatos de procedencia de la ejecución
Tiempos y monitorización	<code>timings.json</code> , monitorización y <code>run_summary.json</code>

Tabla B.1: Campos que se incorporarán desde los artefactos reales de campaña. En este borrador no se asignan valores. Fuente: contrato de artefactos 3.0.

Evidencia final pendiente. No se ha medido todavía el hardware ni el runtime de la campaña final. La versión de entrega reemplazará esta nota por una tabla consolidada y señalará cualquier diferencia relevante entre ejecuciones.

B.3 Entorno histórico de la MAIN previa

La MAIN de 9 de junio de 2026 conserva un `environment.json` propio. Se reporta solo para establecer la procedencia de las métricas históricas del Capítulo 6.2; no impone ni verifica la futura plataforma.

Los datos de CPU y RAM comunicados fuera del artefacto no se usan para caracterizar el método final. Esta decisión evita mezclar confirmaciones operativas con observación persistida.

Componente	Valor observado en la MAIN histórica
Sistema	Linux 6.8.0-110-generic, x86_64; glibc 2.39
Python	3.12.11
GPU visible	NVIDIA GeForce RTX 3090 Ti
CUDA / cuDNN	13.0 / 9.20.0
PyTorch	2.12.1+cu130
NumPy / pandas	2.4.6 / 3.0.3
scikit-learn / joblib	1.9.0 / 1.5.3
Gymnasium / SB3 / sb3-contrib	1.2.3 / 2.8.0 / 2.8.0

Tabla B.2: Versiones observadas durante la MAIN histórica. No describen la campaña final. Fuente: `environment.json` de `MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655`.

B.4 Contratos de dependencias

Contrato	Función
<code>requirements-gpu-cu130.txt</code>	Dependencias directas de la plataforma experimental GPU
<code>requirements.txt</code>	Ruta compatible de instalación general del proyecto
<code>pyproject.toml</code> y <code>uv.lock</code>	Desarrollo, pruebas y resolución bloqueada
<code>environment.json</code>	Observación real por ejecución; prevalece sobre una expectativa del manifiesto

Tabla B.3: Separación entre manifiestos de instalación y entorno observado. Fuente: repositorio actual.

Un manifiesto puede declarar una versión compatible sin demostrar qué importó una ejecución. Por eso las versiones finales se toman del artefacto y se contrastan con el contrato antes de aceptar el run.

B.5 Entorno de construcción de la memoria

La memoria se construye en Windows con TeX Live 2025, `latexmk` 4.87, Biber 2.21 y Poppler para renderizado de control. Este entorno solo produce el

PDF: no se utiliza para generar métricas científicas.

Componente	Uso
pdfLaTeX y Biber	compilación, referencias y bibliografía
latexmk	construcción limpia y repetible
Poppler	metadatos, extracción de texto y rasterizado para QA visual
Python/matplotlib	figuras cuantitativas deterministas desde artefactos
TikZ/PGF	diagramas conceptuales vectoriales

Tabla B.4: Pila utilizada para construir y revisar la memoria, independiente de la plataforma de entrenamiento. Fuente: entorno de esta revisión.

Esquema canónico completo

Este anexo fija la enumeración completa del contrato de entrada descrito de forma agrupada en el Capítulo 4. El orden procede de `FEATURES_CANON`, definido en `src/canonical_schema.py`. Para una característica canónica de índice i , con $1 \leq i \leq 76$, su valor x_i ocupa la posición i del vector de observación y su indicador m_i ocupa la posición $76 + i$. Por tanto, las posiciones 1–76 contienen los valores canónicos y las posiciones 77–152 contienen, en el mismo orden, los indicadores cuyos nombres persistidos siguen el patrón `m_<identificador_canónico>`.

En CICIDS2017, la máscara registra la presencia de la columna fuente tras la limpieza previa al mapeo: los valores no finitos ya se han sustituido. Por tanto, 1 indica que la columna mapeada existe en ese punto del pipeline, no que preserve ausencias fila a fila. Como las 76 columnas tienen correspondencia, en los datos nativos de entrenamiento las posiciones 77–152 valen siempre 1. Ante otra entrada sin columna mapeada, el valor se imputa con cero y su indicador permanece en 0.

La Tabla C.1 conserva el orden y muestra las dos correspondencias mantenidas por el proyecto, ambas completas para los 76 identificadores. En Flowmeter.py, la presencia efectiva depende de las columnas de cada CSV. La tabla no añade unidades ni descripciones ajenas a las fuentes.

Tabla C.1: Esquema canónico completo y correspondencias de columnas fuente. Fuente: elaboración propia a partir de `src/canonical_schema.py` y `scripts/predict_real_traffic_v2.py`.

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
1	<code>flow_duration</code>	Flow Duration	<code>flow_duration</code>	Estadísticas generales del flujo
2	<code>total_fwd_packets</code>	Total Fwd Packets	<code>tot_fwd_pkts</code>	Estadísticas generales del flujo
3	<code>total_bwd_packets</code>	Total Backward Packets	<code>tot_bwd_pkts</code>	Estadísticas generales del flujo
4	<code>total_length_of_fwd_packets</code>	Total Length of Fwd Packets	<code>totlen_fwd_pkts</code>	Estadísticas generales del flujo
5	<code>total_length_of_bwd_packets</code>	Total Length of Bwd Packets	<code>totlen_bwd_pkts</code>	Estadísticas generales del flujo
6	<code>fwd_packet_length_max</code>	Fwd Packet Length Max	<code>fwd_pkt_len_max</code>	Tamaño de paquetes (hacia adelante)
7	<code>fwd_packet_length_min</code>	Fwd Packet Length Min	<code>fwd_pkt_len_min</code>	Tamaño de paquetes (hacia adelante)
8	<code>fwd_packet_length_mean</code>	Fwd Packet Length Mean	<code>fwd_pkt_len_mean</code>	Tamaño de paquetes (hacia adelante)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- PY	Grupo lógico
9	fwd_packet_length_std	Fwd Packet Length Std	fwd_pkt_len_std	Tamaño de paquetes (hacia adelante)
10	bwd_packet_length_max	Bwd Packet Length Max	bwd_pkt_len_max	Tamaño de paquetes (hacia atrás)
11	bwd_packet_length_min	Bwd Packet Length Min	bwd_pkt_len_min	Tamaño de paquetes (hacia atrás)
12	bwd_packet_length_mean	Bwd Packet Length Mean	bwd_pkt_len_mean	Tamaño de paquetes (hacia atrás)
13	bwd_packet_length_std	Bwd Packet Length Std	bwd_pkt_len_std	Tamaño de paquetes (hacia atrás)
14	flow_bytes_per_s	Flow Bytes/s	flow_byts_s	Tasas de flujo
15	flow_packets_per_s	Flow Packets/s	flow_pkts_s	Tasas de flujo
16	flow_iat_mean	Flow IAT Mean	flow_iat_mean	Tiempo entre llegadas (flujo)
17	flow_iat_std	Flow IAT Std	flow_iat_std	Tiempo entre llegadas (flujo)
18	flow_iat_max	Flow IAT Max	flow_iat_max	Tiempo entre llegadas (flujo)
19	flow_iat_min	Flow IAT Min	flow_iat_min	Tiempo entre llegadas (flujo)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- PY	Grupo lógico
20	fwd_iat_total	Fwd IAT Total	fwd_iat_tot	Tiempo entre llegadas (hacia adelante)
21	fwd_iat_mean	Fwd IAT Mean	fwd_iat_mean	Tiempo entre llegadas (hacia adelante)
22	fwd_iat_std	Fwd IAT Std	fwd_iat_std	Tiempo entre llegadas (hacia adelante)
23	fwd_iat_max	Fwd IAT Max	fwd_iat_max	Tiempo entre llegadas (hacia adelante)
24	fwd_iat_min	Fwd IAT Min	fwd_iat_min	Tiempo entre llegadas (hacia adelante)
25	bwd_iat_total	Bwd IAT Total	bwd_iat_tot	Tiempo entre llegadas (hacia atrás)
26	bwd_iat_mean	Bwd IAT Mean	bwd_iat_mean	Tiempo entre llegadas (hacia atrás)
27	bwd_iat_std	Bwd IAT Std	bwd_iat_std	Tiempo entre llegadas (hacia atrás)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- PY	Grupo lógico
28	bwd_iat_max	Bwd IAT Max	bwd_iat_max	Tiempo entre llegadas (hacia atrás)
29	bwd_iat_min	Bwd IAT Min	bwd_iat_min	Tiempo entre llegadas (hacia atrás)
30	fwd_psh_flags	Fwd PSH Flags	fwd_psh_flags	Indicadores TCP
31	bwd_psh_flags	Bwd PSH Flags	bwd_psh_flags	Indicadores TCP
32	fwd_urg_flags	Fwd URG Flags	fwd_urg_flags	Indicadores TCP
33	bwd_urg_flags	Bwd URG Flags	bwd_urg_flags	Indicadores TCP
34	fin_flag_count	FIN Flag Count	fin_flag_cnt	Indicadores TCP
35	syn_flag_count	SYN Flag Count	syn_flag_cnt	Indicadores TCP
36	rst_flag_count	RST Flag Count	rst_flag_cnt	Indicadores TCP
37	psh_flag_count	PSH Flag Count	psh_flag_cnt	Indicadores TCP
38	ack_flag_count	ACK Flag Count	ack_flag_cnt	Indicadores TCP
39	urg_flag_count	URG Flag Count	urg_flag_cnt	Indicadores TCP
40	cwe_flag_count	CWE Flag Count	cwr_flag_count	Indicadores TCP
41	ece_flag_count	ECE Flag Count	ece_flag_cnt	Indicadores TCP

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- PY	Grupo lógico
42	fwd_header_length	Fwd Header Length	fwd_header_len	Longitud de cabecera
43	bwd_header_length	Bwd Header Length	bwd_header_len	Longitud de cabecera
44	fwd_packets_per_s	Fwd Packets/s	fwd_pkts_s	Paquetes por segundo (por di- rección)
45	bwd_packets_per_s	Bwd Packets/s	bwd_pkts_s	Paquetes por segundo (por di- rección)
46	min_packet_length	Min Packet Length	pkt_len_min	Longitud de paquete (global)
47	max_packet_length	Max Packet Length	pkt_len_max	Longitud de paquete (global)
48	packet_length_mean	Packet Length Mean	pkt_len_mean	Longitud de paquete (global)
49	packet_length_std	Packet Length Std	pkt_len_std	Longitud de paquete (global)
50	packet_length_variance	Packet Length Variance	pkt_len_var	Longitud de paquete (global)
51	down_up_ratio	Down/Up Ratio	down_up_ratio	Ratios y estadísticas derivadas
52	average_packet_size	Average Packet Size	pkt_size_avg	Ratios y estadísticas derivadas
53	avg_fwd_segment_size	Avg Fwd Segment Size	fwd_seg_size_avg	Ratios y estadísticas derivadas
54	avg_bwd_segment_size	Avg Bwd Segment Size	bwd_seg_size_avg	Ratios y estadísticas derivadas
55	fwd_avg_bytes_per_bulk	Fwd Avg Bytes/Bulk	fwd_byts_b_avg	Ráfagas (hacia adelante)

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- PY	Grupo lógico
56	fwd_avg_packets_per_bulk	Fwd Avg Packets/Bulk	fwd_pkts_b_avg	Ráfagas (hacia adelante)
57	fwd_avg_bulk_rate	Fwd Avg Bulk Rate	fwd_blk_rate_avg	Ráfagas (hacia adelante)
58	bwd_avg_bytes_per_bulk	Bwd Avg Bytes/Bulk	bwd_byts_b_avg	Ráfagas (hacia atrás)
59	bwd_avg_packets_per_bulk	Bwd Avg Packets/Bulk	bwd_pkts_b_avg	Ráfagas (hacia atrás)
60	bwd_avg_bulk_rate	Bwd Avg Bulk Rate	bwd_blk_rate_avg	Ráfagas (hacia atrás)
61	subflow_fwd_packets	Subflow Fwd Packets	subflow_fwd_pkts	Subflujos
62	subflow_fwd_bytes	Subflow Fwd Bytes	subflow_fwd_byts	Subflujos
63	subflow_bwd_packets	Subflow Bwd Packets	subflow_bwd_pkts	Subflujos
64	subflow_bwd_bytes	Subflow Bwd Bytes	subflow_bwd_byts	Subflujos
65	init_win_bytes_forward	Init_Win_bytes_forward	init_fwd_win_byts	Ventana TCP
66	init_win_bytes_backward	Init_Win_bytes_backward	init_bwd_win_byts	Ventana TCP
67	act_data_pkt_fwd	act_data_pkt_fwd	fwd_act_data_pkts	Datos activos
68	min_seg_size_forward	min_seg_size_forward	fwd_seg_size_min	Datos activos

Continúa en la página siguiente

Tabla C.1 (continuación)

Índice	Identificador canónico	Columna fuente CICIDS2017	Columna fuente Flowmeter- py	Grupo lógico
69	active_mean	Active Mean	active_mean	Tiempos activos/inactivos
70	active_std	Active Std	active_std	Tiempos activos/inactivos
71	active_max	Active Max	active_max	Tiempos activos/inactivos
72	active_min	Active Min	active_min	Tiempos activos/inactivos
73	idle_mean	Idle Mean	idle_mean	Tiempos activos/inactivos
74	idle_std	Idle Std	idle_std	Tiempos activos/inactivos
75	idle_max	Idle Max	idle_max	Tiempos activos/inactivos
76	idle_min	Idle Min	idle_min	Tiempos activos/inactivos

Catálogo de artefactos

El catálogo distingue evidencia histórica, contratos finales y productos agregados. Un nombre de ejecución no basta para acreditar una cifra: debe poder seguirse hasta configuración, datos, modelo, evaluación y manifiesto.

D.1 Esquema de ejecución final

El esquema 3.0 organiza cada intento físico bajo:

```
runs/final_campaign/<CAMPAIGN_ID>/attempts/<LOGICAL_RUN_ID>  
/attempt-<N>/
```

El estado de campaña registra identificador lógico, ejecución física, intento, dependencias, estado, alias y exportación. Un alias conserva `reuse_of`, el hash del manifiesto fuente y la fecha de validación; no copia el modelo.

Artefacto	Función científica
<code>config.json</code>	perfil resuelto, semillas, partición, tamaño, pasos, recompensa, hashes y dependencias
<code>metrics.json</code>	matriz de confusión, soporte y métricas con nulos explícitos cuando no son definibles
<code>environment.json</code>	hardware, sistema y paquetes observados
<code>timings.json</code>	tiempos por fase y duración total
<code>system_metrics.*</code>	uso de CPU, memoria, GPU y almacenamiento durante la ejecución
<code>run_summary.json</code>	resumen normalizado de configuración, progreso, métricas, tiempos y escalares
Modelo y preprocesamiento	política serializada, escalador, percentiles y nombres de características
Predicciones/validación	evidencia para reproducir métricas y contrastar evaluación directa
Manifiesto de artefactos	inventario, tamaño y SHA-256 de los ficheros producidos

Tabla D.1: Contenido científico mínimo de una ejecución primaria final. Fuente: esquema de artefactos 3.0 del repositorio.

Trabajo	Dependencia	Producto
Validación directa	MAIN física fresca	predicciones, conteos y comparación con métricas persistidas
Bootstrap MAIN	predicciones finales de MAIN	intervalos estratificados, 10 000 remuestreos, semilla 12345
Duplicados MAIN	partición y caché canónica de MAIN	duplicados internos y coincidencias train/test
Etiquetas barajadas	perfil, caché y split fijo	control separado de 10 000 pasos
Fase 2 fresca	modelo, escalador y percentiles de MAIN	predicciones, métricas y diagnósticos de dominio

Tabla D.2: Dependencias y salidas de los cinco trabajos auxiliares. Ninguno puede sustituirse por un artefacto histórico. Fuente: especificación final de campaña.

D.2 Artefactos auxiliares

Los cuatro primeros trabajos vinculados a MAIN deben referenciar la misma ejecución física y los hashes esperados. El control barajado permanece separado del agregado de rendimiento. Fase 2 añade el hash de la captura utilizada y deshabilita por defecto la exportación de metadatos sensibles.

D.3 Productos agregados

El agregador final produce resúmenes para MAIN, día completo, escalera de tamaños, sensibilidad a semilla, cuatro escenarios retenidos, RF emparejado y Fase 2. Conserva identificadores, semillas, soporte, prevalencia, hashes, tiempos y procedencia de alias. Las medias de holdouts se calculan solo sobre métricas definidas y reportan cuántos escenarios contribuyen; las métricas agrupadas responden a una pregunta distinta y se etiquetan como tales.

La Figura 6.1 representa la ruta hasta la tesis. Los generadores cuantitativos rechazan un agregado incompleto y escriben primero fuera de `memoria/`. Cada figura final tendrá un manifiesto de procedencia con los ficheros y valores representados.

D.4 Catálogo histórico previo a campaña

La Tabla D.3 conserva los artefactos usados para la sección histórica de Resultados. Su estado explícito impide que ocupen una celda final.

Tabla D.3: Artefactos previos a la campaña final citados en la memoria.
Fuente: repositorio.

Objeto	Ruta o identificador	Uso permitido
MAIN QRDQN histórica	runs/cicids2017/MAIN_qrdqn_cicids2017_canonical_full_random_20260609_193655/	Capacidad intradistribución y procedencia histórica
Referencia de test	runs/cicids2017/test_partition_reference_seed42.json	Reproducción de la partición histórica
Checks A/B/C	runs/validation/VAL_checks_A_20260212_235443, VAL_checks_B_20260212_235736, VAL_checks_C_20260213_004847	Controles y proxy temporal históricos
Bootstrap	runs/validation/bootstrap_ci_seed42.json	Precisión de muestreo histórica
Duplicados	runs/validation/duplicate_analysis_seed42.json	Continuidad de la partición histórica
Random Forest	runs/baselines/rf_cicids2017_canonical_20260628_024735	Comparación histórica, no RF final
Fase 2 histórica	runs/phase2/P2v2_pred_20260610_161231_MAIN/	Inferencia sobre una captura concreta con MAIN histórica

Los modelos exploratorios, sondas rápidas y ejecuciones fallidas no se citan

como evidencia científica. Los artefactos históricos no se modifican durante la campaña ni durante la escritura de la memoria.

D.5 Git, LFS y exportación

Los JSON, CSV agregados, resúmenes, diagnósticos y manifiestos pequeños son apropiados para revisión en Git. Los modelos, predicciones voluminosas y otros binarios rastreados utilizan Git LFS cuando la política del repositorio lo indica. Los CSV originales, cachés generadas y paquetes de recuperación no se añaden automáticamente al historial.

Tras cada ejecución física validada, la exportación externa contiene una copia restaurable del directorio y un `tar.gz` con SHA-256. La copia no demuestra por sí sola almacenamiento independiente duradero, pero reduce el riesgo de perder evidencia durante una campaña larga y permite reanudar una exportación sin reentrenar.

D.6 Estado antes de la campaña

El catálogo final no contiene todavía ninguna ejecución bajo un `CAMPAIGN_ID` aceptado. La Tabla [D.1](#) describe el contrato, no una lista de resultados existentes.

Evidencia final pendiente. Cuando aparezcan artefactos finales, este anexo deberá añadir el identificador de campaña, el índice de ejecuciones aceptadas, el hash del agregado y la localización del bundle verificado, sin copiar aquí todas las métricas del Capítulo de Resultados.